Open Influence Diagrams: Proofs

Simon Frost

2026-10-19

Contents

Boundary observability **2**

Reward kernels: kernels over the expectation semiring **3**

The expectation-semiring kernel category **8**

Expected utility in context and the contextual identity **13**

Decision and utility generators on the typed open-network syntax **14**

The fixed-strategy interpretation and its functoriality **17**

Optimisation does not compose component by component **24**

Lenses and open-game equilibria (ADR 0020) **27**

The tensor of open games (ADR 0020, amended) **32**

Perfect equilibria (ADR 0020, amended) **37**

Evidence across a boundary (ADR 0021) **39**

Behavioural strategies and the value of randomising (ADR 0022) **47**

Reward certificates: the record **55**

Reward certificates: decoding the JSON **57**
- Constant strings . . . 57
- Flags and numeric mode . . . 58
- Metadata and observability . . . 59
- Tables, utilities, policies and the result . . . 59
- The network . . . 61
- The certificate . . . 62

Reward certificates: the exact reward kernel of the certificate's numbers **63**
- Flat indices . . . 64
- Labels, tables and policies . . . 65
- Coordinates . . . 66
- The enumeration . . . 66
- Result coordinates . . . 67
- Diagnostics . . . 68

Reward certificates: a faster exact check **69**
- Accumulating into an array . . . 69
- The output coordinate read at `(x, z)` . . . 70
- A row of `K` and `W` in one pass . . . 71
- The errors, row by row . . . 72

Reward certificates: the checker **72**

Reward certificates: soundness **76**
Tables by label 78
The interpretation, the strategy and the diagram 79
The bridge: `interpret` is the exact enumeration 80
Acceptance 83

Boundary observability

```
import CategoricalBayesianNetworksProofs
import InfluenceDiagramsProofs
```

The first module of the proofs of `CategoricalInfluenceDiagrams.jl` (ADR 0018, decision 5). It imports the two proof projects this one builds on: the typed open-network syntax of `CategoricalBayesianNetworks.jl` (ADR 0010, namespace `OpenNet`) and the finite influence-diagram model of `InfluenceDiagrams.jl` (namespace `InfluenceDiagramsProofs`).

It states the one piece of structure that open influence diagrams add to the boundary of an open network: every boundary variable is **observed**, remembered information that a later decision may read, or **hidden**. An interface with observability is an `OpenNet.Interface` with a flag on every port, and the tensor of two such interfaces places ports and flags side by side. **Information locality**, rule 3 of the typed-interface rule, says that no decision of a finite influence diagram reads a variable that its boundary declares hidden.

The later modules prove the rest of ADR 0018, decision 5: the expectation-semiring kernel category with its symmetric monoidal structure (`RewardKernel.lean`, `Category.lean`), the contextual identity (`Context.lean`), decision and utility generators on the typed syntax (`Syntax.lean`), the functoriality of the fixed-strategy interpretation (`Interpretation.lean`) and the isolated-optimisation and hidden-input obstructions (`Obstruction.lean`). The observability flags of this module are not read by that semantics.

```
set_option autoImplicit false

namespace OpenInfluence

/-- The observability of a boundary variable (ADR 0018, decision 2). -/
inductive Observability where
  /-- Remembered information that a later decision may read. -/
  | observed
  /-- State that crosses the boundary but that no decision on the far side may read. -/
  | hidden
  deriving DecidableEq, Repr

/-- An interface of the open-network syntax with an observability flag on every port. -/
structure ObservedInterface (S : OpenNet.Signature) where
  /-- The underlying finite typed interface. -/
  iface : OpenNet.Interface S
  /-- The flag of each port. -/
  flag : iface.Port → Observability

namespace ObservedInterface

variable {S : OpenNet.Signature}

/-- The tensor of two interfaces with observability: ports and flags side by side. -/
def sum (X Y : ObservedInterface S) : ObservedInterface S :=
  ⟨X.iface.sum Y.iface, Sum.elim X.flag Y.flag⟩

/-- The empty interface, the unit of `sum`. -/
def empty (S : OpenNet.Signature) : ObservedInterface S :=
  ⟨OpenNet.Interface.empty S, Empty.elim⟩

/-- The tensor keeps the flags of the left interface. -/
theorem sum_flag_inl (X Y : ObservedInterface S) (p : X.iface.Port) :
    (X.sum Y).flag (Sum.inl p) = X.flag p := rfl
```

```
/-- The tensor keeps the flags of the right interface. -/
theorem sum_flag_inr (X Y : ObservedInterface S) (p : Y.iface.Port) :
    (X.sum Y).flag (Sum.inr p) = Y.flag p := rfl

end ObservedInterface

open InfluenceDiagramsProofs

/-- Information locality (rule 3 of ADR 0018, decision 2): given the observability of the
boundary variables of a finite influence diagram (`none` for an internal variable), no
decision has a hidden boundary variable in its information set. -/
def InformationLocal (D : FinInfluenceDiagram) (boundary : D.V → Option Observability) :
    Prop :=
  ∀ d v, v ∈ D.info d → boundary v ≠ some .hidden

/-- A diagram whose boundary declares nothing hidden is information-local. -/
theorem informationLocal_of_no_hidden (D : FinInfluenceDiagram)
    (boundary : D.V → Option Observability) (h : ∀ v, boundary v ≠ some .hidden) :
    InformationLocal D boundary :=
  fun _ v _ ⇒ h v

/-- Revealing boundary variables (turning hidden flags into observed or internal ones)
preserves information locality. -/
theorem InformationLocal.of_reveal {D : FinInfluenceDiagram}
    {b b' : D.V → Option Observability}
    (h : InformationLocal D b) (hb : ∀ v, b' v = some .hidden → b v = some .hidden) :
    InformationLocal D b' :=
  fun d v hv hid ⇒ h d v hv (hb v hid)

end OpenInfluence
```

Reward kernels: kernels over the expectation semiring

```
import FiniteKernelsProofs.Finite.Kernel
import Mathlib.Tactic.Abel
```

A **reward kernel** `X → Y` between finite types is a pair `(K, W)` of real tables. `K x y` is the probability of reaching boundary state `y` from `x`, and `W x y` is the probability-weighted reward earned on the way (ADR 0018, decision 3). Composition, identity and tensor are those of `CategoricalInfluenceDiagrams.jl`'s `RewardKernel` (`src/semantics.jl`):

- `(K₁, W₁) ; (K₂, W₂) = (K₁K₂, W₁K₂ + K₁W₂)`;
- `id = (id, 0)`;
- `(K₁, W₁) ⊗ (K₂, W₂) = (K₁ ⊗ K₂, W₁ ⊗ K₂ + K₁ ⊗ W₂)`.

This is kernel composition over the expectation semiring `(p, r)·(q, s) = (pq, ps + rq)`. `K` reuses `FiniteKernelsProofs.Finite.Kernel` unchanged; only the reward component is new.

This module proves, for arbitrary real tables (no nonnegativity or normalisation is needed for any of them):

- the category laws: `id_comp`, `comp_id` and `comp_assoc`;
- the functoriality of the tensor: `tensor_comp_tensor` (the interchange law `(f ⊗ g) ; (f' ⊗ g') = (f ; f') ⊗ (g ; g')`) and `tensor_id`;
- the naturality of the structural maps, all lifted from `FiniteKernels` with zero reward: `tensor_assoc` (associator), `tensor_unit_left` and `tensor_unit_right` (unitors) and `tensor_swap` (symmetry);
- that copy and discard lifted with zero reward satisfy the comonoid laws, and that stochasticity of `K` is preserved by composition and tensor (`Stochastic.comp`, `Stochastic.tensor`).

Discard is **not** natural: `comp_discard` computes `f ; discard = (K ; discard, row sums of W)`, and `Examples.discard_not_natural` exhibits a stochastic reward kernel whose composite with discard is not discard. The Mathlib packaging of these laws as a symmetric monoidal category is `Category.lean`.

```
set_option autoImplicit false

noncomputable section

namespace OpenInfluence

open FiniteKernelsProofs.Finite
```

Bilinearity of kernel composition and tensor

```
namespace KernelAlgebra

variable {X Y Z X₁ X₂ Y₁ Y₂ : Type}

theorem kcomp_add_left [Fintype Y] (k k' : Kernel X Y) (l : Kernel Y Z) :
    Kernel.comp (k + k') l = Kernel.comp k l + Kernel.comp k' l := by
  funext x z
  simp [Kernel.comp, add_mul, Finset.sum_add_distrib]

theorem kcomp_add_right [Fintype Y] (k : Kernel X Y) (l l' : Kernel Y Z) :
    Kernel.comp k (l + l') = Kernel.comp k l + Kernel.comp k l' := by
  funext x z
  simp [Kernel.comp, mul_add, Finset.sum_add_distrib]

theorem kcomp_zero_left [Fintype Y] (l : Kernel Y Z) :
    Kernel.comp (0 : Kernel X Y) l = 0 := by
  funext x z
  simp [Kernel.comp]

theorem kcomp_zero_right [Fintype Y] (k : Kernel X Y) :
    Kernel.comp k (0 : Kernel Y Z) = 0 := by
  funext x z
  simp [Kernel.comp]

theorem ktensor_add_left (k k' : Kernel X₁ Y₁) (l : Kernel X₂ Y₂) :
    Kernel.tensor (k + k') l = Kernel.tensor k l + Kernel.tensor k' l := by
  funext x y
  simp [Kernel.tensor, add_mul]

theorem ktensor_add_right (k : Kernel X₁ Y₁) (l l' : Kernel X₂ Y₂) :
    Kernel.tensor k (l + l') = Kernel.tensor k l + Kernel.tensor k l' := by
  funext x y
  simp [Kernel.tensor, mul_add]

theorem ktensor_zero_left (l : Kernel X₂ Y₂) :
    Kernel.tensor (0 : Kernel X₁ Y₁) l = 0 := by
  funext x y
  simp [Kernel.tensor]

theorem ktensor_zero_right (k : Kernel X₁ Y₁) :
    Kernel.tensor k (0 : Kernel X₂ Y₂) = 0 := by
  funext x y
  simp [Kernel.tensor]

end KernelAlgebra

open KernelAlgebra

/-- A reward kernel `X → Y`: a probability table `K` and a probability-weighted reward
table `W` of the same shape (Julia `RewardKernel(K, W)`). -/
@[ext] structure RewardKernel (X Y : Type) where
  /-- `K x y`: the probability of reaching `y` from `x`. -/
  K : Kernel X Y
  /-- `W x y`: the reward earned on the way from `x` to `y`, weighted by its probability. -/
  W : Kernel X Y
```

```
namespace RewardKernel

variable {X Y Z V X₁ X₂ X₃ Y₁ Y₂ Y₃ Z₁ Z₂ : Type}
```

Constructions

```
/-- Serial composition `(K₁, W₁) ; (K₂, W₂) = (K₁K₂, W₁K₂ + K₁W₂)` (diagrammatic order). -/
def comp [Fintype Y] (f : RewardKernel X Y) (g : RewardKernel Y Z) : RewardKernel X Z :=
  ⟨Kernel.comp f.K g.K, Kernel.comp f.W g.K + Kernel.comp f.K g.W⟩

/-- A kernel with zero reward. Identities and all structural maps are of this form. -/
def lift (k : Kernel X Y) : RewardKernel X Y := ⟨k, 0⟩

/-- The identity `(id, 0)`. -/
def id (X : Type) [DecidableEq X] : RewardKernel X X := lift (Kernel.idK X)

/-- The tensor `(K₁ ⊗ K₂, W₁ ⊗ K₂ + K₁ ⊗ W₂)`. -/
def tensor (f : RewardKernel X₁ Y₁) (g : RewardKernel X₂ Y₂) :
    RewardKernel (X₁ × X₂) (Y₁ × Y₂) :=
  ⟨Kernel.tensor f.K g.K, Kernel.tensor f.W g.K + Kernel.tensor f.K g.W⟩

/-- The deterministic reward kernel of a function, with zero reward. -/
def ofFun [DecidableEq Y] (e : X → Y) : RewardKernel X Y := lift (Kernel.ofFun e)

/-- Copy, lifted with zero reward. -/
def copy (X : Type) [DecidableEq X] : RewardKernel X (X × X) := lift (Kernel.copy X)

/-- Discard, lifted with zero reward. -/
def discard (X : Type) : RewardKernel X Unit := lift (Kernel.discard X)

/-- The probability component is stochastic (nonnegative and normalised); the reward
component is unconstrained. -/
def Stochastic [Fintype Y] (f : RewardKernel X Y) : Prop := Kernel.Stochastic f.K

@[simp] theorem comp_K [Fintype Y] (f : RewardKernel X Y) (g : RewardKernel Y Z) :
    (f.comp g).K = Kernel.comp f.K g.K := rfl

@[simp] theorem comp_W [Fintype Y] (f : RewardKernel X Y) (g : RewardKernel Y Z) :
    (f.comp g).W = Kernel.comp f.W g.K + Kernel.comp f.K g.W := rfl

@[simp] theorem tensor_K (f : RewardKernel X₁ Y₁) (g : RewardKernel X₂ Y₂) :
    (f.tensor g).K = Kernel.tensor f.K g.K := rfl

@[simp] theorem tensor_W (f : RewardKernel X₁ Y₁) (g : RewardKernel X₂ Y₂) :
    (f.tensor g).W = Kernel.tensor f.W g.K + Kernel.tensor f.K g.W := rfl

@[simp] theorem lift_K (k : Kernel X Y) : (lift k).K = k := rfl
@[simp] theorem lift_W (k : Kernel X Y) : (lift k).W = 0 := rfl
```

Category laws

```
theorem id_comp [Fintype X] [DecidableEq X] (f : RewardKernel X Y) : (id X).comp f = f := by
  ext1
  · exact Kernel.idK_comp f.K
  · change Kernel.comp 0 f.K + Kernel.comp (Kernel.idK X) f.W = f.W
    rw [kcomp_zero_left, Kernel.idK_comp, zero_add]

theorem comp_id [Fintype Y] [DecidableEq Y] (f : RewardKernel X Y) : f.comp (id Y) = f := by
  ext1
  · exact Kernel.comp_idK f.K
  · change Kernel.comp f.W (Kernel.idK Y) + Kernel.comp f.K 0 = f.W
    rw [kcomp_zero_right, Kernel.comp_idK, add_zero]
```

```
theorem comp_assoc [Fintype Y] [Fintype Z] (f : RewardKernel X Y) (g : RewardKernel Y Z)
    (h : RewardKernel Z V) : (f.comp g).comp h = f.comp (g.comp h) := by
  ext1
  · exact Kernel.comp_assoc f.K g.K h.K
  · simp only [comp_W, comp_K, kcomp_add_left, kcomp_add_right, Kernel.comp_assoc]
    abel
```

Lifted kernels

```
theorem lift_comp_lift [Fintype Y] (k : Kernel X Y) (l : Kernel Y Z) :
    (lift k).comp (lift l) = lift (Kernel.comp k l) := by
  ext1
  · rfl
  · simp [kcomp_zero_left, kcomp_zero_right]

theorem lift_tensor_lift (k : Kernel X₁ Y₁) (l : Kernel X₂ Y₂) :
    (lift k).tensor (lift l) = lift (Kernel.tensor k l) := by
  ext1
  · rfl
  · simp [ktensor_zero_left, ktensor_zero_right]
```

Functoriality of the tensor

```
/-- The interchange law `(f ⊗ g) ; (f' ⊗ g') = (f ; f') ⊗ (g ; g')`. -/
theorem tensor_comp_tensor [Fintype Y₁] [Fintype Y₂] (f : RewardKernel X₁ Y₁)
    (g : RewardKernel X₂ Y₂) (f' : RewardKernel Y₁ Z₁) (g' : RewardKernel Y₂ Z₂) :
    (f.tensor g).comp (f'.tensor g') = (f.comp f').tensor (g.comp g') := by
  ext1
  · exact Kernel.comp_tensor f.K g.K f'.K g'.K
  · simp only [comp_W, comp_K, tensor_W, tensor_K, kcomp_add_left, kcomp_add_right,
      ktensor_add_left, ktensor_add_right, Kernel.comp_tensor]
    abel

/-- The tensor of identities is the identity. -/
theorem tensor_id [DecidableEq X] [DecidableEq Y] :
    (id X).tensor (id Y) = id (X × Y) := by
  simp only [id, lift_tensor_lift, Kernel.tensor_idK]
```

Naturality of the structural maps

```
/-- Associativity of the tensor up to the associator (lifted with zero reward). -/
theorem tensor_assoc [Fintype X₁] [Fintype X₂] [Fintype X₃] [Fintype Y₁] [Fintype Y₂]
    [Fintype Y₃] [DecidableEq X₁] [DecidableEq X₂] [DecidableEq X₃] [DecidableEq Y₁]
    [DecidableEq Y₂] [DecidableEq Y₃] (f : RewardKernel X₁ Y₁) (g : RewardKernel X₂ Y₂)
    (h : RewardKernel X₃ Y₃) :
    ((f.tensor g).tensor h).comp (lift (Kernel.assoc Y₁ Y₂ Y₃)) =
      (lift (Kernel.assoc X₁ X₂ X₃)).comp (f.tensor (g.tensor h)) := by
  ext1
  · exact Kernel.tensor_assoc f.K g.K h.K
  · simp only [comp_W, tensor_W, tensor_K, lift_W, lift_K, kcomp_add_left,
      kcomp_add_right, kcomp_zero_left, kcomp_zero_right, ktensor_add_left, ktensor_add_right,
      Kernel.tensor_assoc]
    abel

/-- Left unit of the tensor up to the left unitor. -/
theorem tensor_unit_left [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y]
    (f : RewardKernel X Y) :
    ((id Unit).tensor f).comp (lift (Kernel.leftUnitor Y)) =
      (lift (Kernel.leftUnitor X)).comp f := by
  ext1
  · exact Kernel.tensor_unit_left f.K
```

```
  · simp only [id, comp_W, tensor_W, tensor_K, lift_W, lift_K, kcomp_add_left,
      kcomp_zero_left, kcomp_zero_right, ktensor_zero_left, Kernel.tensor_unit_left]
    simp

/-- Right unit of the tensor up to the right unitor. -/
theorem tensor_unit_right [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y]
    (f : RewardKernel X Y) :
    (f.tensor (id Unit)).comp (lift (Kernel.rightUnitor Y)) =
      (lift (Kernel.rightUnitor X)).comp f := by
  ext1
  · exact Kernel.tensor_unit_right f.K
  · simp only [id, comp_W, tensor_W, tensor_K, lift_W, lift_K, kcomp_add_left,
      kcomp_zero_left, kcomp_zero_right, ktensor_zero_right, Kernel.tensor_unit_right]
    simp

/-- Naturality of the symmetry. -/
theorem tensor_swap [Fintype X₁] [Fintype X₂] [Fintype Y₁] [Fintype Y₂] [DecidableEq X₁]
    [DecidableEq X₂] [DecidableEq Y₁] [DecidableEq Y₂] (f : RewardKernel X₁ Y₁)
    (g : RewardKernel X₂ Y₂) :
    (f.tensor g).comp (lift (Kernel.swap Y₁ Y₂)) =
      (lift (Kernel.swap X₁ X₂)).comp (g.tensor f) := by
  ext1
  · exact Kernel.tensor_swap f.K g.K
  · simp only [comp_W, tensor_W, tensor_K, lift_W, lift_K, kcomp_add_left,
      kcomp_add_right, kcomp_zero_left, kcomp_zero_right, Kernel.tensor_swap]
    abel
```

Copy and discard with zero reward

```
/-- Coassociativity of the lifted copy. -/
theorem copy_assoc [Fintype X] [DecidableEq X] :
    (copy X).comp ((id X).tensor (copy X)) =
      ((copy X).comp ((copy X).tensor (id X))).comp (lift (Kernel.assoc X X X)) := by
  simp only [copy, id, lift_tensor_lift, lift_comp_lift, Kernel.copy_assoc]

/-- Left counit law of the lifted copy and discard. -/
theorem copy_discard_left [Fintype X] [DecidableEq X] :
    (copy X).comp ((discard X).tensor (id X)) = lift (Kernel.leftUnitorInv X) := by
  simp only [copy, discard, id, lift_tensor_lift, lift_comp_lift, Kernel.copy_discard_left]

/-- Right counit law of the lifted copy and discard. -/
theorem copy_discard_right [Fintype X] [DecidableEq X] :
    (copy X).comp ((id X).tensor (discard X)) = lift (Kernel.rightUnitorInv X) := by
  simp only [copy, discard, id, lift_tensor_lift, lift_comp_lift, Kernel.copy_discard_right]

/-- Cocommutativity of the lifted copy. -/
theorem copy_swap [Fintype X] [DecidableEq X] :
    (copy X).comp (lift (Kernel.swap X X)) = copy X := by
  simp only [copy, lift_comp_lift, Kernel.copy_swap]

/-- Composing with discard keeps the discarded probability and sums the reward over the
output: `f ; discard = (K ; discard, x ↦ Σ_y W x y)`. -/
theorem comp_discard [Fintype Y] (f : RewardKernel X Y) :
    f.comp (discard Y) = ⟨Kernel.comp f.K (Kernel.discard Y), fun x _ ⇒ ∑ y, f.W x y⟩ := by
  ext1
  · rfl
  · funext x u
    simp [comp, discard, lift, Kernel.comp]

/-- Discard is natural exactly for the reward kernels with normalised `K` and zero total
reward from every input. In particular it is not natural in general. -/
theorem comp_discard_eq_discard_iff [Fintype Y] (f : RewardKernel X Y) :
    f.comp (discard Y) = discard X ↔ Kernel.Normalised f.K ∧ ∀ x, ∑ y, f.W x y = 0 := by
  rw [comp_discard]
  constructor
```

```
  · intro h
    have hK := congrArg RewardKernel.K h
    have hW := congrArg RewardKernel.W h
    refine ⟨fun x ⇒ ?_, fun x ⇒ ?_⟩
    · have := congrFun (congrFun hK x) ()
      simpa [Kernel.comp, discard, lift] using this
    · exact congrFun (congrFun hW x) ()
  · rintro ⟨hK, hW⟩
    ext1
    · funext x u
      simpa [Kernel.comp, discard, lift] using hK x
    · funext x u
      exact hW x
```

Stochasticity

```
theorem Stochastic.comp [Fintype Y] [Fintype Z] {f : RewardKernel X Y}
    {g : RewardKernel Y Z} (hf : f.Stochastic) (hg : g.Stochastic) : (f.comp g).Stochastic :=
  Kernel.Stochastic.comp hf hg

theorem Stochastic.tensor [Fintype Y₁] [Fintype Y₂] {f : RewardKernel X₁ Y₁}
    {g : RewardKernel X₂ Y₂} (hf : f.Stochastic) (hg : g.Stochastic) :
    (f.tensor g).Stochastic :=
  Kernel.Stochastic.tensor hf hg

theorem Stochastic.ofFun [Fintype Y] [DecidableEq Y] (e : X → Y) : (ofFun e).Stochastic :=
  Kernel.Stochastic.ofFun e

theorem Stochastic.id [Fintype X] [DecidableEq X] : (id X).Stochastic :=
  Kernel.Stochastic.ofFun _

theorem Stochastic.discard : (discard X).Stochastic :=
  ⟨Kernel.Nonneg.discard, Kernel.Normalised.discard⟩

end RewardKernel
```

Discard is not natural

```
namespace RewardKernel.Examples

/-- A one-point process that is certain to stay put and earns reward one. -/
def unitReward : RewardKernel Unit Unit := ⟨Kernel.idK Unit, fun _ _ ⇒ 1⟩

theorem unitReward_stochastic : unitReward.Stochastic :=
  Kernel.Stochastic.ofFun (fun u : Unit ⇒ u)

/-- Discard, lifted with zero reward, is not natural: a stochastic reward kernel that earns
reward one, followed by discard, still carries that reward, so it is not discard (as ADR 0010
recorded `discard_not_natural` for the open-network syntax). -/
theorem discard_not_natural :
    unitReward.Stochastic ∧ unitReward.comp (discard Unit) ≠ discard Unit := by
  refine ⟨unitReward_stochastic, fun h ⇒ ?_⟩
  have hW := congrFun (congrFun (congrArg RewardKernel.W h) ()) ()
  rw [comp_discard] at hW
  simp [unitReward, discard, lift] at hW

end RewardKernel.Examples

end OpenInfluence
```

The expectation-semiring kernel category

```
import CategoricalInfluenceDiagramsProofs.RewardKernel
import FiniteKernelsProofs.Theory.FinStoch
import Mathlib.CategoryTheory.CopyDiscardCategory.Basic
```

`FinRew` packages the laws of `RewardKernel.lean` as Mathlib instances. Objects are finite types with decidable equality (the empty type included), as in `FinStoch`. A morphism is a reward kernel whose probability component `K` is stochastic; the reward component `W` is an arbitrary real table, of either sign. Composition, identity and tensor are those of `RewardKernel`; the associator, unitors and symmetry are the deterministic reindexings of `FinStoch`, lifted with zero reward.

The instances are a `Category`, a `MonoidalCategory`, a `SymmetricCategory` and a `CopyDiscardCategory` (every object a commutative comonoid, copy and discard lifted with zero reward, coherent with the tensor). There is deliberately no `MarkovCategory` instance: discard is not natural, because `f ≫ ε` keeps the expected reward of `f` (`FinRew.discard_not_natural`).

`FinRew.lift` is the faithful embedding of `FinStoch` with zero reward; it preserves composition, identities and tensor, so `FinStoch` is the reward-free part of `FinRew`.

```
set_option autoImplicit false

noncomputable section

namespace OpenInfluence

open CategoryTheory MonoidalCategory FiniteKernelsProofs FiniteKernelsProofs.Finite

/-- Finite state types with decidable equality, including the empty type. -/
structure FinRew where
  /-- The finite set of boundary states. -/
  carrier : Type
  [fintype : Fintype carrier]
  [decEq : DecidableEq carrier]

attribute [instance] FinRew.fintype FinRew.decEq

namespace FinRew

instance instCategory : Category FinRew where
  Hom X Y := {r : RewardKernel X.carrier Y.carrier // r.Stochastic}
  id X := ⟨RewardKernel.id X.carrier, RewardKernel.Stochastic.id⟩
  comp f g := ⟨f.1.comp g.1, f.2.comp g.2⟩
  id_comp f := Subtype.ext (RewardKernel.id_comp f.1)
  comp_id f := Subtype.ext (RewardKernel.comp_id f.1)
  assoc f g h := Subtype.ext (RewardKernel.comp_assoc f.1 g.1 h.1)

variable {X Y Z : FinRew}

@[simp] theorem comp_val (f : X → Y) (g : Y → Z) : (f ≫ g).val = f.val.comp g.val := rfl

@[simp] theorem id_val (X : FinRew) : (𝟙 X : X → X).val = RewardKernel.id X.carrier := rfl

/-- A stochastic kernel with zero reward. -/
def lift (k : Kernel X.carrier Y.carrier) (hk : Kernel.Stochastic k) : X → Y :=
  ⟨RewardKernel.lift k, hk⟩

/-- The deterministic morphism of a function, with zero reward. -/
def det (f : X.carrier → Y.carrier) : X → Y := lift (Kernel.ofFun f) (Kernel.Stochastic.ofFun f)

/-- A state-space equivalence induces an isomorphism with zero reward. -/
def isoOfEquiv (e : X.carrier ≃ Y.carrier) : X ≅ Y where
  hom := det e
  inv := det e.symm
  hom_inv_id := Subtype.ext (by
    change (RewardKernel.lift _).comp (RewardKernel.lift _) = RewardKernel.lift _
```

```
    rw [RewardKernel.lift_comp_lift, Kernel.ofFun_comp_ofFun]
    exact congrArg RewardKernel.lift (Kernel.ofFun_congr fun x ⇒ e.symm_apply_apply x))
  inv_hom_id := Subtype.ext (by
    change (RewardKernel.lift _).comp (RewardKernel.lift _) = RewardKernel.lift _
    rw [RewardKernel.lift_comp_lift, Kernel.ofFun_comp_ofFun]
    exact congrArg RewardKernel.lift (Kernel.ofFun_congr fun y ⇒ e.apply_symm_apply y))

instance instMonoidalCategoryStruct : MonoidalCategoryStruct FinRew where
  tensorObj X Y := ⟨X.carrier × Y.carrier⟩
  tensorUnit := ⟨Unit⟩
  tensorHom f g := ⟨f.1.tensor g.1, f.2.tensor g.2⟩
  whiskerLeft X _ _ g := ⟨(RewardKernel.id X.carrier).tensor g.1,
    RewardKernel.Stochastic.id.tensor g.2⟩
  whiskerRight f Y := ⟨f.1.tensor (RewardKernel.id Y.carrier),
    f.2.tensor RewardKernel.Stochastic.id⟩
  associator X Y Z := isoOfEquiv (Equiv.prodAssoc X.carrier Y.carrier Z.carrier)
  leftUnitor X := isoOfEquiv (Equiv.punitProd X.carrier)
  rightUnitor X := isoOfEquiv (Equiv.prodPUnit X.carrier)

@[simp] theorem tensorHom_val {X Y X' Y' : FinRew} (f : X → Y) (g : X' → Y') :
    (f ⊗ₘ g).val = f.val.tensor g.val := rfl

instance instMonoidalCategory : MonoidalCategory FinRew :=
  MonoidalCategory.ofTensorHom
    (id_tensorHom_id := fun _ _ ⇒ Subtype.ext RewardKernel.tensor_id)
    (id_tensorHom := by intros; rfl)
    (tensorHom_id := by intros; rfl)
    (tensorHom_comp_tensorHom := fun f g f' g' ⇒
      Subtype.ext (RewardKernel.tensor_comp_tensor f.1 g.1 f'.1 g'.1))
    (associator_naturality := fun f g h ⇒ Subtype.ext (RewardKernel.tensor_assoc f.1 g.1 h.1))
    (leftUnitor_naturality := fun f ⇒ Subtype.ext (RewardKernel.tensor_unit_left f.1))
    (rightUnitor_naturality := fun f ⇒ Subtype.ext (RewardKernel.tensor_unit_right f.1))
    (pentagon := by
      intro W X Y Z
      apply Subtype.ext
      change ((RewardKernel.lift _).tensor (RewardKernel.id _)).comp
        ((RewardKernel.lift _).comp ((RewardKernel.id _).tensor (RewardKernel.lift _))) =
          (RewardKernel.lift _).comp (RewardKernel.lift _)
      simp only [RewardKernel.id, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift,
        Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun]
      rfl)
    (triangle := by
      intro X Y
      apply Subtype.ext
      change (RewardKernel.lift _).comp ((RewardKernel.id _).tensor (RewardKernel.lift _)) =
          (RewardKernel.lift _).tensor (RewardKernel.id _)
      simp only [RewardKernel.id, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift,
        Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun]
      rfl)

instance instSymmetricCategory : SymmetricCategory FinRew where
  braiding X Y := isoOfEquiv (Equiv.prodComm X.carrier Y.carrier)
  braiding_naturality_left := by
    intro X Y f Z
    exact Subtype.ext (RewardKernel.tensor_swap f.1 (RewardKernel.id _))
  braiding_naturality_right := by
    intro X Y Z f
    exact Subtype.ext (RewardKernel.tensor_swap (RewardKernel.id _) f.1)
  hexagon_forward := by
    intro X Y Z
    apply Subtype.ext
    change (RewardKernel.lift _).comp ((RewardKernel.lift _).comp (RewardKernel.lift _)) =
      ((RewardKernel.lift _).tensor (RewardKernel.id _)).comp
        ((RewardKernel.lift _).comp ((RewardKernel.id _).tensor (RewardKernel.lift _)))
    simp only [RewardKernel.id, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift,
      Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun]
```

```
    rfl
  hexagon_reverse := by
    intro X Y Z
    apply Subtype.ext
    change (RewardKernel.lift _).comp ((RewardKernel.lift _).comp (RewardKernel.lift _)) =
      ((RewardKernel.id _).tensor (RewardKernel.lift _)).comp
        ((RewardKernel.lift _).comp ((RewardKernel.lift _).tensor (RewardKernel.id _)))
    simp only [RewardKernel.id, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift,
      Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun]
    rfl
  symmetry := by
    intro X Y
    apply Subtype.ext
    change (RewardKernel.lift _).comp (RewardKernel.lift _) = RewardKernel.lift _
    simp only [RewardKernel.lift_comp_lift, Kernel.ofFun_comp_ofFun]
    rfl

/-- Copy and discard, lifted with zero reward. -/
instance instComonObj (X : FinRew) : ComonObj X where
  counit := lift (Kernel.discard X.carrier) ⟨Kernel.Nonneg.discard, Kernel.Normalised.discard⟩
  comul := det fun x ⇒ (x, x)
  counit_comul := Subtype.ext (by
    change (RewardKernel.lift _).comp ((RewardKernel.lift _).tensor (RewardKernel.id _)) =
      RewardKernel.lift _
    simp only [RewardKernel.id, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift]
    exact congrArg RewardKernel.lift Kernel.copy_discard_left)
  comul_counit := Subtype.ext (by
    change (RewardKernel.lift _).comp ((RewardKernel.id _).tensor (RewardKernel.lift _)) =
      RewardKernel.lift _
    simp only [RewardKernel.id, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift]
    exact congrArg RewardKernel.lift Kernel.copy_discard_right)
  comul_assoc := Subtype.ext (by
    change (RewardKernel.lift _).comp ((RewardKernel.id _).tensor (RewardKernel.lift _)) =
      (RewardKernel.lift _).comp
        (((RewardKernel.lift _).tensor (RewardKernel.id _)).comp (RewardKernel.lift _))
    simp only [RewardKernel.id, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift,
      Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun]
    rfl)

instance instIsCommComonObj (X : FinRew) : IsCommComonObj X where
  comul_comm := Subtype.ext (by
    change (RewardKernel.lift _).comp (RewardKernel.lift _) = RewardKernel.lift _
    simp only [RewardKernel.lift_comp_lift, Kernel.ofFun_comp_ofFun]
    rfl)

open scoped ComonObj

/-- The structural `tensorμ` of Mathlib is the middle-swap reindexing with zero reward. -/
theorem tensorμ_val (X Y : FinRew) :
    (MonoidalCategory.tensorμ X X Y Y).val =
      RewardKernel.lift (Kernel.tensorμ X.carrier Y.carrier) := by
  simp only [MonoidalCategory.tensorμ]
  change (RewardKernel.lift _).comp
    (((RewardKernel.id _).tensor (RewardKernel.lift _)).comp
      (((RewardKernel.id _).tensor ((RewardKernel.lift _).tensor (RewardKernel.id _))).comp
        (((RewardKernel.id _).tensor (RewardKernel.lift _)).comp (RewardKernel.lift _)))) = _
  simp only [RewardKernel.id, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift,
    Kernel.idK, Kernel.tensor_ofFun, Kernel.ofFun_comp_ofFun, Kernel.tensorμ]
  rfl

instance instCopyDiscardCategory : CopyDiscardCategory FinRew where
  copy_tensor X Y := Subtype.ext (by
    change RewardKernel.lift (Kernel.copy (X.carrier × Y.carrier)) =
      ((RewardKernel.lift (Kernel.copy X.carrier)).tensor
        (RewardKernel.lift (Kernel.copy Y.carrier))).comp
        (MonoidalCategory.tensorμ X X Y Y).val
```

```
    rw [tensorμ_val, RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift]
    exact congrArg RewardKernel.lift Kernel.copy_prod)
  discard_tensor X Y := Subtype.ext (by
    change RewardKernel.lift (Kernel.discard (X.carrier × Y.carrier)) =
      ((RewardKernel.lift (Kernel.discard X.carrier)).tensor
        (RewardKernel.lift (Kernel.discard Y.carrier))).comp
        (RewardKernel.lift (Kernel.ofFun (Equiv.punitProd Unit)))
    rw [RewardKernel.lift_tensor_lift, RewardKernel.lift_comp_lift]
    exact congrArg RewardKernel.lift Kernel.discard_prod)
  copy_unit := Subtype.ext rfl
  discard_unit := Subtype.ext (by
    change RewardKernel.lift (Kernel.discard Unit) = RewardKernel.lift (Kernel.idK Unit)
    exact congrArg RewardKernel.lift Kernel.discard_unit)

/-- Composing with discard sums the reward over the output. -/
theorem comp_discard_val (f : X → Y) :
    (f ≫ ε[Y]).val = ⟨Kernel.discard X.carrier, fun x _ ⇒ ∑ y, f.val.W x y⟩ := by
  change f.val.comp (RewardKernel.discard Y.carrier) = _
  rw [RewardKernel.comp_discard]
  congr 1
  exact Kernel.Normalised.comp_discard f.2.2

/-- Discard is natural exactly for the morphisms whose total reward from every input is
zero. -/
theorem comp_discard_eq_iff (f : X → Y) : f ≫ ε[Y] = ε[X] ↔ ∀ x, ∑ y, f.val.W x y = 0 := by
  rw [← Subtype.coe_inj, comp_discard_val]
  constructor
  · intro h x
    exact congrFun (congrFun (congrArg RewardKernel.W h) x) ()
  · intro h
    ext1
    · rfl
    · funext x u
      exact h x

/-- The one-point object. -/
abbrev unitObj : FinRew := ⟨Unit⟩

/-- The morphism that stays put and earns reward one. -/
def unitReward : unitObj → unitObj :=
  ⟨RewardKernel.Examples.unitReward, RewardKernel.Examples.unitReward_stochastic⟩

/-- `FinRew` is a copy-discard category but not a Markov category: discard is not natural.
The witness is the morphism `unitObj → unitObj` that stays put and earns reward one. -/
theorem discard_not_natural : unitReward ≫ ε[unitObj] ≠ ε[unitObj] := by
  rw [Ne, comp_discard_eq_iff]
  intro h
  have := h ()
  simp [unitReward, RewardKernel.Examples.unitReward] at this

/-- The embedding of `FinStoch` with zero reward. -/
def ofFinStoch (X : FinStoch) : FinRew := ⟨X.carrier⟩

/-- A stochastic kernel as a morphism with zero reward. -/
def liftHom {X Y : FinStoch} (f : X → Y) : ofFinStoch X → ofFinStoch Y := lift f.1 f.2

theorem liftHom_comp {X Y Z : FinStoch} (f : X → Y) (g : Y → Z) :
    liftHom (f ≫ g) = liftHom f ≫ liftHom g :=
  Subtype.ext (RewardKernel.lift_comp_lift f.1 g.1).symm

theorem liftHom_id (X : FinStoch) : liftHom (𝟙 X) = 𝟙 (ofFinStoch X) := rfl

theorem liftHom_tensor {X Y X' Y' : FinStoch} (f : X → Y) (g : X' → Y') :
    (liftHom (f ⊗ₘ g) : ofFinStoch (X ⊗ X') → ofFinStoch (Y ⊗ Y')) =
      (liftHom f ⊗ₘ liftHom g : ofFinStoch X ⊗ ofFinStoch X' → _) :=
  Subtype.ext (RewardKernel.lift_tensor_lift f.1 g.1).symm
```

```
theorem liftHom_injective {X Y : FinStoch} : Function.Injective (liftHom (X := X) (Y := Y)) :=
  fun _ _ h ⇒ Subtype.ext (congrArg (fun r : ofFinStoch X → ofFinStoch Y ⇒ r.1.K) h)

end FinRew

end OpenInfluence
```

Expected utility in context and the contextual identity

```
import CategoricalInfluenceDiagramsProofs.RewardKernel
```

For a reward kernel `f = (K, W) : X → Y`, a boundary prior `μ` on `X` and a terminal continuation value `c` on `Y` (both arbitrary real functions), the expected utility is

`EU_f(μ, c) = Σ_x μ x · Σ_y (W x y + K x y · c y)`,

the bundle's `Score_{μ,c}(K, W)`. A suffix `g : Y → Z` turns a continuation `c` on `Z` into the continuation `c_g y = Σ_z (W_g y z + K_g y z · c z)` on `Y`, its value function. A prefix `f` turns a prior `μ` on `X` into the induced boundary law `ν_f y = Σ_x μ x · K_f x y` and earns the prefix reward `L_f = Σ_x μ x · Σ_y W_f x y`.

The **contextual identity** (ADR 0018; the bundle's `INF-FIXED-CONTEXT`) is

`EU_{f;g}(μ, c) = EU_f(μ, c_g) = L_f + EU_g(ν_f, c)`.

The second equation holds for arbitrary tables (`eu_continuation`). The first needs the suffix's probability component to be normalised (`eu_comp`): the prefix reward `W_f x y` must survive the suffix with total mass one. `eu_comp` shows exactly that the defect is `Σ_x μ x Σ_y W_f x y (Σ_z K_g y z - 1)` (`eu_comp_general`), so the hypothesis cannot simply be dropped. For an interpreted diagram, where `f` and `g` are the reward kernels of the two components under fixed strategies, `Interpretation.lean` applies this through the functoriality of `interpret`.

```
set_option autoImplicit false

noncomputable section

namespace OpenInfluence

open FiniteKernelsProofs.Finite

namespace RewardKernel

variable {X Y Z : Type} [Fintype X] [Fintype Y] [Fintype Z]

/-- Expected utility of a reward kernel under the boundary prior `μ` and the terminal
continuation `c`. -/
def eu (f : RewardKernel X Y) (μ : X → ℝ) (c : Y → ℝ) : ℝ :=
  ∑ x, μ x * ∑ y, (f.W x y + f.K x y * c y)

/-- The value function `c_g` of a suffix `g` with terminal continuation `c`. -/
def continuation (g : RewardKernel Y Z) (c : Z → ℝ) : Y → ℝ :=
  fun y ⇒ ∑ z, (g.W y z + g.K y z * c z)

/-- The boundary law `ν_f = μ K_f` induced by the prefix `f` from the prior `μ`. -/
def pushforward (f : RewardKernel X Y) (μ : X → ℝ) : Y → ℝ :=
  fun y ⇒ ∑ x, μ x * f.K x y

/-- The reward `L_f = Σ μ W_f` earned by the prefix `f` from the prior `μ`. -/
def prefixReward (f : RewardKernel X Y) (μ : X → ℝ) : ℝ :=
  ∑ x, μ x * ∑ y, f.W x y

/-- The expected utility with terminal continuation `c` is the prior-weighted continuation
of `f`. -/
theorem eu_eq_continuation (f : RewardKernel X Y) (μ : X → ℝ) (c : Y → ℝ) :
    f.eu μ c = ∑ x, μ x * f.continuation c x := rfl
```

```
/-- Composition without normalisation: the composite's expected utility differs from the
contextual one by the prefix reward times the suffix's missing mass. -/
theorem eu_comp_general (f : RewardKernel X Y) (g : RewardKernel Y Z) (μ : X → ℝ)
    (c : Z → ℝ) :
    (f.comp g).eu μ c =
      f.eu μ (g.continuation c) + ∑ x, μ x * ∑ y, f.W x y * ((∑ z, g.K y z) - 1) := by
  have hL : ∀ x, (∑ z, ((f.comp g).W x z + (f.comp g).K x z * c z)) =
      ∑ y, ∑ z, (f.W x y * g.K y z + f.K x y * g.W y z + f.K x y * g.K y z * c z) := by
    intro x
    simp only [comp_W, comp_K, Kernel.comp, Pi.add_apply, Finset.sum_mul,
      ← Finset.sum_add_distrib]
    exact Finset.sum_comm
  have hR : ∀ x y, (f.W x y + f.K x y * g.continuation c y) +
      f.W x y * ((∑ z, g.K y z) - 1) =
      ∑ z, (f.W x y * g.K y z + f.K x y * g.W y z + f.K x y * g.K y z * c z) := by
    intro x y
    have e : (∑ z, (f.W x y * g.K y z + f.K x y * g.W y z + f.K x y * g.K y z * c z)) =
        f.W x y * ∑ z, g.K y z + f.K x y * g.continuation c y := by
      simp only [continuation, Finset.mul_sum, ← Finset.sum_add_distrib, mul_add, mul_assoc,
        add_assoc]
    rw [e]
    ring
  simp only [eu, ← Finset.sum_add_distrib, ← mul_add, hL, ← hR]

/-- First half of the contextual identity: with a normalised suffix, the composite's
expected utility is the prefix's expected utility against the suffix's value function. -/
theorem eu_comp (f : RewardKernel X Y) (g : RewardKernel Y Z) (hg : Kernel.Normalised g.K)
    (μ : X → ℝ) (c : Z → ℝ) : (f.comp g).eu μ c = f.eu μ (g.continuation c) := by
  rw [eu_comp_general]
  have h0 : ∀ y, (∑ z, g.K y z) - 1 = 0 := fun y ⇒ by rw [hg y, sub_self]
  simp [h0]

/-- Second half of the contextual identity: against any continuation `c'`, the prefix's
expected utility is its own reward plus the continuation's expectation under the induced
boundary law. -/
theorem eu_continuation (f : RewardKernel X Y) (g : RewardKernel Y Z) (μ : X → ℝ)
    (c : Z → ℝ) :
    f.eu μ (g.continuation c) = f.prefixReward μ + g.eu (f.pushforward μ) c := by
  rw [eu_eq_continuation g]
  generalize g.continuation c = C
  simp only [eu, prefixReward, pushforward, mul_add, Finset.sum_add_distrib, Finset.mul_sum,
    Finset.sum_mul]
  congr 1
  rw [Finset.sum_comm]
  exact Finset.sum_congr rfl fun y _ ⇒ Finset.sum_congr rfl fun x _ ⇒ by ring

/-- **Contextual identity** for reward kernels:
`EU_{f;g}(μ, c) = EU_f(μ, c_g) = L_f + EU_g(ν_f, c)`, for a suffix with normalised `K`. -/
theorem contextual_identity (f : RewardKernel X Y) (g : RewardKernel Y Z)
    (hg : Kernel.Normalised g.K) (μ : X → ℝ) (c : Z → ℝ) :
    (f.comp g).eu μ c = f.eu μ (g.continuation c) ∧
      f.eu μ (g.continuation c) = f.prefixReward μ + g.eu (f.pushforward μ) c :=
  ⟨eu_comp f g hg μ c, eu_continuation f g μ c⟩

/-- The normalisation of the suffix cannot be dropped from the first half: here the suffix
loses all mass, and the prefix's reward of one is lost with it. -/
theorem eu_comp_needs_normalised :
    ∃ (f g : RewardKernel Unit Unit) (μ : Unit → ℝ) (c : Unit → ℝ),
      (f.comp g).eu μ c ≠ f.eu μ (g.continuation c) := by
  refine ⟨⟨fun _ _ ⇒ 1, fun _ _ ⇒ 1⟩, ⟨fun _ _ ⇒ 0, fun _ _ ⇒ 0⟩, fun _ ⇒ 1,
    fun _ ⇒ 0, ?_⟩
  simp [eu, comp, continuation, Kernel.comp]

end RewardKernel

end OpenInfluence
```

Decision and utility generators on the typed open-network syntax

```
import CategoricalInfluenceDiagramsProofs.Basic
```

ADR 0010's typed open-network syntax (`OpenNet.Net X Y`) has one kind of generator: a mechanism with a label, an ordered list of typed parents and one typed output. This module extends it in the two ways ADR 0018 (decision 5) asks for.

- **Decision generators.** An influence-diagram signature (`IDSignature`) marks each generator label as a chance mechanism or a decision (`isDecision`). A decision generator is an ordinary generator of the underlying net whose label is a decision label; its ordered parent list is its list of **information ports**, so a policy for it is a function of the values on those ports (`Interpretation.lean`).
- **Utility generators.** A signature also has utility labels, each with an ordered list of input types (`uinputs`). An open influence diagram (`IDNet X Y`) is a net together with a finite set of utility generators, each with a label and an ordered, typed list of the net's variables (its **scope**). A utility reads its scope and produces no variable, so utilities never cross a boundary; their reward crosses only through the semantics, as ADR 0018 requires.

Serial composition (`IDNet.comp`) and tensor (`IDNet.tensor`) are those of the underlying nets, with the utility generators of the two components placed side by side and their scopes substituted along the nets' variable maps (`Net.left`, `Net.right`, `Net.tinl`, `Net.tinr`). Identities have no generators and no utilities. Every net embeds with no utility (`IDNet.ofNet`), and the atomic generators are `IDNet.generator` (one chance or decision generator on its own inputs) and `IDNet.utility` (one utility on a scope of an interface, passing the interface through).

Scope. The syntax is the full generality of `OpenNet.Net`: arbitrary finite nets, not only terms built from generators. It is **not** quotiented by structural isomorphism, so the functoriality theorems of `Interpretation.lean` are equations on representatives rather than a Mathlib `Functor` out of a syntax category. Observability flags (`Basic.lean`) are not part of this syntax: the fixed-strategy semantics does not read them, and information locality is a separate side condition on strategies.

```
set_option autoImplicit false

namespace OpenInfluence

/-- A signature for open influence diagrams: an `OpenNet` signature whose labels are
chance mechanisms or decisions, together with utility labels and their input types. -/
structure IDSignature extends OpenNet.Signature where
  /-- Whether generators with this label are decisions (`true`) or chance mechanisms. -/
  isDecision : Label → Bool
  /-- Utility labels. -/
  ULabel : Type
  /-- The ordered input types of a utility label. -/
  uinputs : ULabel → List Ty

/-- The finite typed interfaces of a signature, those of the underlying `OpenNet`
signature. -/
abbrev IDSignature.Iface (S : IDSignature) := OpenNet.Interface S.toSignature

variable {S : IDSignature}

/-- An open influence diagram `X → Y`: a typed open net (whose generators are chance
mechanisms and decisions) with a finite set of utility generators, each reading an ordered,
typed list of the net's variables. -/
structure IDNet (X Y : S.Iface) where
  /-- The underlying open net of chance and decision generators. -/
  net : OpenNet.Net X Y
  /-- The utility generators. -/
  UGen : Type
  /-- There are finitely many utility generators. -/
  ufinite : Fintype UGen
  /-- The label of each utility generator. -/
  ulabel : UGen → S.ULabel
  /-- The ordered scope of each utility generator: input ports or internal generators. -/
  uparents : UGen → List net.Var
  /-- Each scope has the input types of its label. -/
```

```
  uparents_typed : ∀ u, (uparents u).map net.varType = S.uinputs (ulabel u)

attribute [instance] IDNet.ufinite

namespace IDNet

variable {X Y Z X' Y' : S.Iface}

/-- A generator of the underlying net is a decision when its label is a decision label. -/
def IsDecision (f : IDNet X Y) (a : f.net.Gen) : Prop := S.isDecision (f.net.label a) = true

/-- The information ports of a generator: its ordered parents. -/
def info (f : IDNet X Y) (a : f.net.Gen) : List f.net.Var := f.net.parents a

/-- An open net with no utility generators. -/
def ofNet (n : OpenNet.Net X Y) : IDNet X Y where
  net := n
  UGen := Empty
  ufinite := inferInstance
  ulabel := Empty.elim
  uparents := Empty.elim
  uparents_typed u := u.elim

/-- The identity: pure wiring, with no generators and no utilities. -/
def identity (X : S.Iface) : IDNet X X := ofNet (OpenNet.Net.identity X)

/-- Serial composition: compose the nets, and keep the utilities of both components with
their scopes substituted along the gluing. -/
def comp (f : IDNet X Y) (g : IDNet Y Z) : IDNet X Z where
  net := f.net.comp g.net
  UGen := f.UGen ⊕ g.UGen
  ufinite := inferInstance
  ulabel := Sum.elim f.ulabel g.ulabel
  uparents := Sum.elim
    (fun u ⇒ (f.uparents u).map (OpenNet.Net.left f.net g.net))
    (fun u ⇒ (g.uparents u).map (OpenNet.Net.right f.net g.net))
  uparents_typed := by
    intro u
    cases u with
    | inl u ⇒
      change ((f.uparents u).map (OpenNet.Net.left f.net g.net)).map _ = _
      rw [List.map_map]
      exact (congrArg (fun k ⇒ (f.uparents u).map k)
        (funext (OpenNet.Net.left_typed f.net g.net))).trans (f.uparents_typed u)
    | inr u ⇒
      change ((g.uparents u).map (OpenNet.Net.right f.net g.net)).map _ = _
      rw [List.map_map]
      exact (congrArg (fun k ⇒ (g.uparents u).map k)
        (funext (OpenNet.Net.right_typed f.net g.net))).trans (g.uparents_typed u)

/-- Tensor: place the nets side by side, and keep the utilities of both components with
their scopes reindexed into the disjoint union. -/
def tensor (f : IDNet X Y) (g : IDNet X' Y') : IDNet (X.sum X') (Y.sum Y') where
  net := f.net.tensor g.net
  UGen := f.UGen ⊕ g.UGen
  ufinite := inferInstance
  ulabel := Sum.elim f.ulabel g.ulabel
  uparents := Sum.elim
    (fun u ⇒ (f.uparents u).map (OpenNet.Net.tinl f.net g.net))
    (fun u ⇒ (g.uparents u).map (OpenNet.Net.tinr f.net g.net))
  uparents_typed := by
    intro u
    cases u with
    | inl u ⇒
      change ((f.uparents u).map (OpenNet.Net.tinl f.net g.net)).map _ = _
      rw [List.map_map]
```

```
      exact (congrArg (fun k ⇒ (f.uparents u).map k)
        (funext (OpenNet.Net.tinl_typed f.net g.net))).trans (f.uparents_typed u)
    | inr u ⇒
      change ((g.uparents u).map (OpenNet.Net.tinr f.net g.net)).map _ = _
      rw [List.map_map]
      exact (congrArg (fun k ⇒ (g.uparents u).map k)
        (funext (OpenNet.Net.tinr_typed f.net g.net))).trans (g.uparents_typed u)

@[simp] theorem comp_net (f : IDNet X Y) (g : IDNet Y Z) : (f.comp g).net = f.net.comp g.net :=
  rfl

@[simp] theorem tensor_net (f : IDNet X Y) (g : IDNet X' Y') :
    (f.tensor g).net = f.net.tensor g.net := rfl
```

Atomic generators

```
/-- The interface of the ordered input types of a generator label. -/
def inputIface (l : S.Label) : S.Iface :=
  ⟨Fin (S.inputs l).length, inferInstance, fun i ⇒ (S.inputs l).get i⟩

/-- The one-port interface of the output type of a generator label. -/
def outputIface (l : S.Label) : S.Iface := ⟨Unit, inferInstance, fun _ ⇒ S.result l⟩

/-- A single chance or decision generator labelled `l`, reading its input ports in order
(for a decision, its information ports) and exposing its output. -/
def generator (l : S.Label) : IDNet (inputIface l) (outputIface l) :=
  ofNet {
    Gen := Unit
    finite := inferInstance
    label := fun _ ⇒ l
    parents := fun _ ⇒ List.ofFn (fun i ⇒ Sum.inl i)
    output := fun _ ⇒ Sum.inr ()
    parents_typed := fun _ ⇒ by
      rw [List.map_ofFn]
      exact List.ofFn_get (S.inputs l)
    output_typed := fun _ ⇒ rfl
    acyclic := ⟨fun _ ⇒ 0, fun _ _ h ⇒ by simp at h⟩ }

/-- A single utility generator labelled `u` reading the ports `scope` of `X`, which it
passes through unchanged. -/
def utility (X : S.Iface) (u : S.ULabel) (scope : List X.Port)
    (h : scope.map X.type = S.uinputs u) : IDNet X X where
  net := OpenNet.Net.identity X
  UGen := Unit
  ufinite := inferInstance
  ulabel _ := u
  uparents _ := scope.map Sum.inl
  uparents_typed _ := by
    rw [List.map_map]
    exact h

end IDNet

end OpenInfluence
```

The fixed-strategy interpretation and its functoriality

```
import CategoricalInfluenceDiagramsProofs.Syntax
import CategoricalInfluenceDiagramsProofs.Context
import CategoricalInfluenceDiagramsProofs.Category
```

An **interpretation** of an influence-diagram signature (`IDInterpretation`) gives every type a finite state space,

every chance label a nonnegative table normalised on every correctly typed parent row, and every utility label a real-valued table on its ordered inputs. A **strategy** (`IDInterpretation.Strategy`) gives every decision label a deterministic policy: a function from the values on the decision's information ports to a state of its output type. As in `CategoricalInfluenceDiagrams.jl`, policies are keyed by label, so decision generators with equal labels share a policy, and one strategy covers both components of a composite (the Julia package's disjoint union of the components' strategies, when their decision labels are disjoint).

A strategy turns the interpretation into an ordinary `OpenNet.Interpretation` (`IDInterpretation.interp`): a decision generator becomes the point mass at its policy's choice, a chance generator keeps its table. Its sum-product kernel (ADR 0010, unchanged) is the probability component `K` of the interpretation. The reward component `W` sums, over the same internal assignments, their probability times the total utility of the diagram's utility generators read on their scopes (`rewardTable`). So the fixed-strategy semantics of an open influence diagram `f : X → Y` is the reward kernel

`interpret σ f = (K_f, W_f) : RewardKernel (Boundary X) (Boundary Y)`,

ADR 0018's `interpret(o, strategy)`. Its probability component is stochastic (`interpret_stochastic`), so it is a morphism of `FinRew` (`interpretHom`).

Functoriality for a fixed strategy, on representatives (the syntax is not quotiented):

- `interpret_comp`: `interpret σ (f ; g) = interpret σ f ; interpret σ g`;
- `interpret_identity`: `interpret σ (id X) = (id, 0)`;
- `interpret_tensor`: `interpret σ (f ⊗ g)`, read through the canonical bijection between assignments of `X ⊔ X'` and pairs of assignments, is `interpret σ f ⊗ interpret σ g`; `interpret_tensor_natural` states the same as naturality of that bijection, the shape of `OpenNet.Interpretation.tensor_natural`.

The **contextual identity** then holds for interpreted diagrams (`contextual_identity`), with the normalisation of the suffix supplied by `interpret_stochastic`.

```
set_option autoImplicit false

noncomputable section

namespace OpenInfluence

open CategoryTheory FiniteKernelsProofs.Finite OpenNet

/-- A semantics for the chance and utility labels of an influence-diagram signature. -/
structure IDInterpretation (S : IDSignature) where
  /-- The finite state space of each type. -/
  states : S.Ty → Type
  /-- State spaces are finite. -/
  finite : ∀ t, Fintype (states t)
  /-- The table of a chance label, on ordered typed parent values and one child value. -/
  chance : S.Label → List (Sigma states) → Sigma states → ℝ
  /-- Chance tables are nonnegative. -/
  chance_nonneg : ∀ l p s, S.isDecision l = false → 0 ≤ chance l p s
  /-- Chance tables are normalised on every correctly typed parent row. -/
  chance_normalized : ∀ l p, S.isDecision l = false → p.map Sigma.fst = S.inputs l →
    @Finset.sum (states (S.result l)) ℝ _ (@Finset.univ _ (finite _))
      (fun s ⇒ chance l p ⟨S.result l, s⟩) = 1
  /-- The table of a utility label, on ordered typed input values. -/
  utility : S.ULabel → List (Sigma states) → ℝ

attribute [instance] IDInterpretation.finite

namespace IDInterpretation

variable {S : IDSignature} (M : IDInterpretation S)

/-- A deterministic strategy: for each decision label, a policy mapping the values on the
decision's information ports to a state of its output type. -/
def Strategy : Type :=
  (l : S.Label) → S.isDecision l = true → List (Sigma M.states) → M.states (S.result l)

open Classical in
```

```
/-- The `OpenNet` interpretation for a fixed strategy: decisions are the point masses of
their policies, chance mechanisms keep their tables. -/
def interp (σ : M.Strategy) : OpenNet.Interpretation S.toSignature where
  states := M.states
  finite := M.finite
  weight l p s :=
    if h : S.isDecision l = true then (if s = ⟨S.result l, σ l h p⟩ then 1 else 0)
    else M.chance l p s
  nonneg l p s := by
    by_cases h : S.isDecision l = true
    · simp only [h, dite_true]
      split_ifs <;> norm_num
    · simp only [h]
      exact M.chance_nonneg l p s (by simpa using h)
  normalized l p hp := by
    by_cases h : S.isDecision l = true
    · simp only [h, dite_true, Sigma.mk.injEq, heq_eq_eq, true_and]
      simp
    · simp only [h]
      exact M.chance_normalized l p (by simpa using h) hp

variable (σ : M.Strategy)
variable {X Y Z X' Y' : S.Iface}

/-- The total utility of the diagram's utility generators, read on their scopes, at the
input assignment `x` and the internal assignment `z`. -/
def reward (f : IDNet X Y) (x : (M.interp σ).Boundary X) (z : (M.interp σ).Internal f.net) :
    ℝ :=
  ∑ u, M.utility (f.ulabel u) ((f.uparents u).map ((M.interp σ).valuation f.net x z).value)

/-- The reward table: the probability-weighted total utility of the internal assignments
that produce each output. -/
def rewardTable (f : IDNet X Y) :
    Kernel ((M.interp σ).Boundary X) ((M.interp σ).Boundary Y) :=
  fun x y ⇒ ∑ z, if y = (M.interp σ).read f.net x z then
    (M.interp σ).joint f.net x z * M.reward σ f x z else 0

/-- The fixed-strategy semantics of an open influence diagram: the sum-product kernel of
its net and its reward table. -/
def interpret (f : IDNet X Y) :
    RewardKernel ((M.interp σ).Boundary X) ((M.interp σ).Boundary Y) :=
  ⟨(M.interp σ).kernel f.net, M.rewardTable σ f⟩

theorem interpret_stochastic (f : IDNet X Y) : (M.interpret σ f).Stochastic :=
  (M.interp σ).kernel_stochastic f.net

/-- The boundary assignments of an interface, as an object of `FinRew`. -/
def space (X : S.Iface) : FinRew := ⟨(M.interp σ).Boundary X⟩

/-- The interpretation as a morphism of `FinRew`. -/
def interpretHom (f : IDNet X Y) : M.space σ X → M.space σ Y :=
  ⟨M.interpret σ f, M.interpret_stochastic σ f⟩
```

The atomic generators

```
open Classical in
/-- A decision generator is the point mass at its policy's choice, read on the values of
its information ports. -/
theorem interp_weight_decision (l : S.Label) (h : S.isDecision l = true)
    (p : List (Sigma M.states)) (s : Sigma M.states) :
    (M.interp σ).weight l p s = if s = ⟨S.result l, σ l h p⟩ then 1 else 0 := by
  simp [interp, h]

/-- A chance generator keeps its table. -/
theorem interp_weight_chance (l : S.Label) (h : S.isDecision l = false)
```

```
    (p : List (Sigma M.states)) (s : Sigma M.states) :
    (M.interp σ).weight l p s = M.chance l p s := by
  simp [interp, h]

/-- A utility generator passes its interface through unchanged. -/
theorem interpret_utility_K (X : S.Iface) (u : S.ULabel) (scope : List X.Port)
    (h : scope.map X.type = S.uinputs u) :
    (M.interpret σ (IDNet.utility X u scope h)).K = Kernel.idK _ := by
  change (M.interp σ).kernel (Net.wire id (fun _ ⇒ rfl)) = Kernel.idK _
  rw [(M.interp σ).kernel_wire]
  apply Kernel.ofFun_congr
  intro x
  rfl

/-- A utility generator earns, on the diagonal, the utility of its label read on its
scope. -/
theorem interpret_utility_W (X : S.Iface) (u : S.ULabel) (scope : List X.Port)
    (h : scope.map X.type = S.uinputs u) (x y : (M.interp σ).Boundary X) :
    (M.interpret σ (IDNet.utility X u scope h)).W x y =
      if y = x then M.utility u (scope.map x.value) else 0 := by
  letI : Unique ((M.interp σ).Internal (Net.wire (id : X.Port → X.Port) (fun _ ⇒ rfl))) := by
    change Unique ((M.interp σ).Assignment (S.result ∘ (Empty.elim : Empty → S.Label)))
    infer_instance
  change (∑ z, if y = (M.interp σ).read (Net.wire id (fun _ ⇒ rfl)) x z then
      (M.interp σ).joint (Net.wire id (fun _ ⇒ rfl)) x z *
        M.reward σ (IDNet.utility X u scope h) x z else 0) = _
  rw [Fintype.sum_unique, (M.interp σ).joint_wire, one_mul]
  have hread : (M.interp σ).read (Net.wire id (fun _ ⇒ rfl)) x default = x := rfl
  have hrew : M.reward σ (IDNet.utility X u scope h) x
      (default : (M.interp σ).Internal (Net.wire (id : X.Port → X.Port) (fun _ ⇒ rfl))) =
      M.utility u (scope.map x.value) := by
    unfold reward
    change (∑ _v : Unit, M.utility u ((scope.map Sum.inl).map _)) = _
    rw [Fintype.sum_unique, List.map_map]
    rfl
  rw [hread, hrew]
```

Rewards of composites

```
theorem reward_comp (f : IDNet X Y) (g : IDNet Y Z) (x : (M.interp σ).Boundary X)
    (a : (M.interp σ).Internal f.net) (b : (M.interp σ).Internal g.net) :
    M.reward σ (f.comp g) x ((M.interp σ).combine f.net g.net a b) =
      M.reward σ f x a + M.reward σ g ((M.interp σ).read f.net x a) b := by
  unfold reward
  erw [Fintype.sum_sum_type]
  congr 1
  · apply Finset.sum_congr rfl
    intro u _
    change M.utility (f.ulabel u)
      (((f.uparents u).map (Net.left f.net g.net)).map _) = _
    rw [List.map_map]
    congr 1
    exact congrArg (fun k ⇒ (f.uparents u).map k)
      (funext ((M.interp σ).valuation_comp_left f.net g.net x a b))
  · apply Finset.sum_congr rfl
    intro u _
    change M.utility (g.ulabel u)
      (((g.uparents u).map (Net.right f.net g.net)).map _) = _
    rw [List.map_map]
    congr 1
    exact congrArg (fun k ⇒ (g.uparents u).map k)
      (funext ((M.interp σ).valuation_comp_right f.net g.net x a b))

theorem reward_tensor (f : IDNet X Y) (g : IDNet X' Y') (x : (M.interp σ).Boundary X)
    (x' : (M.interp σ).Boundary X') (a : (M.interp σ).Internal f.net)
```

```
    (b : (M.interp σ).Internal g.net) :
    M.reward σ (f.tensor g) (Interpretation.Assignment.join _ x x')
        ((M.interp σ).combine f.net g.net a b) =
      M.reward σ f x a + M.reward σ g x' b := by
  unfold reward
  erw [Fintype.sum_sum_type]
  congr 1
  · apply Finset.sum_congr rfl
    intro u _
    change M.utility (f.ulabel u)
      (((f.uparents u).map (Net.tinl f.net g.net)).map _) = _
    rw [List.map_map]
    congr 1
    exact congrArg (fun k ⇒ (f.uparents u).map k)
      (funext ((M.interp σ).valuation_tensor_left f.net g.net x x' a b))
  · apply Finset.sum_congr rfl
    intro u _
    change M.utility (g.ulabel u)
      (((g.uparents u).map (Net.tinr f.net g.net)).map _) = _
    rw [List.map_map]
    congr 1
    exact congrArg (fun k ⇒ (g.uparents u).map k)
      (funext ((M.interp σ).valuation_tensor_right f.net g.net x x' a b))
```

Functoriality under serial composition

```
/-- Summing a product of two output-fibre sums over the middle boundary substitutes the
first component's output into the second. -/
theorem sum_fibre_mul {f : Net X Y} {g : Net Y Z} (x : (M.interp σ).Boundary X)
    (w : (M.interp σ).Boundary Z) (F : (M.interp σ).Internal f → ℝ)
    (G : (M.interp σ).Boundary Y → (M.interp σ).Internal g → ℝ) :
    (∑ y, (∑ a, if y = (M.interp σ).read f x a then F a else 0) *
        (∑ b, if w = (M.interp σ).read g y b then G y b else 0)) =
      ∑ a, ∑ b, if w = (M.interp σ).read g ((M.interp σ).read f x a) b then
        F a * G ((M.interp σ).read f x a) b else 0 := by
  simp only [Finset.sum_mul]
  rw [Finset.sum_comm]
  apply Finset.sum_congr rfl
  intro a _
  rw [Finset.sum_eq_single ((M.interp σ).read f x a)]
  · simp only [if_true, Finset.mul_sum, mul_ite, mul_zero]
  · intro y _ hy
    simp [hy]
  · simp

theorem kernel_eq_sum (f : Net X Y) (x : (M.interp σ).Boundary X)
    (y : (M.interp σ).Boundary Y) :
    (M.interp σ).kernel f x y =
      ∑ z, if y = (M.interp σ).read f x z then (M.interp σ).joint f x z else 0 := by
  unfold Interpretation.kernel
  rfl

theorem rewardTable_comp (f : IDNet X Y) (g : IDNet Y Z) :
    M.rewardTable σ (f.comp g) =
      Kernel.comp (M.rewardTable σ f) ((M.interp σ).kernel g.net) +
        Kernel.comp ((M.interp σ).kernel f.net) (M.rewardTable σ g) := by
  funext x w
  simp only [Pi.add_apply, Kernel.comp, kernel_eq_sum]
  unfold rewardTable
  rw [M.sum_fibre_mul σ x w (fun a ⇒ (M.interp σ).joint f.net x a * M.reward σ f x a)
      (fun y b ⇒ (M.interp σ).joint g.net y b),
    M.sum_fibre_mul σ x w (fun a ⇒ (M.interp σ).joint f.net x a)
      (fun y b ⇒ (M.interp σ).joint g.net y b * M.reward σ g y b)]
  rw [← Finset.sum_add_distrib]
  change (∑ z : (M.interp σ).Internal (f.net.comp g.net),
```

```
      if w = (M.interp σ).read (f.net.comp g.net) x z then
        (M.interp σ).joint (f.net.comp g.net) x z * M.reward σ (f.comp g) x z else 0) = _
  erw [← ((M.interp σ).sumLabelsEquiv f.net.label g.net.label).symm.sum_comp]
  rw [Fintype.sum_prod_type]
  apply Finset.sum_congr rfl
  intro a _
  rw [← Finset.sum_add_distrib]
  apply Finset.sum_congr rfl
  intro b _
  have hr := (M.interp σ).read_comp f.net g.net x a b
  have hj := (M.interp σ).joint_comp f.net g.net x a b
  have hw := M.reward_comp σ f g x a b
  simp only [Interpretation.combine] at hr hj hw
  rw [hr, hj, hw]
  split_ifs <;> ring

/-- **Functoriality under serial composition**: the interpretation of a composite is the
composite, in the expectation-semiring kernel category, of the interpretations. -/
theorem interpret_comp (f : IDNet X Y) (g : IDNet Y Z) :
    M.interpret σ (f.comp g) = (M.interpret σ f).comp (M.interpret σ g) := by
  ext1
  · exact (M.interp σ).kernel_comp f.net g.net
  · exact M.rewardTable_comp σ f g

theorem interpretHom_comp (f : IDNet X Y) (g : IDNet Y Z) :
    M.interpretHom σ (f.comp g) = M.interpretHom σ f ≫ M.interpretHom σ g :=
  Subtype.ext (M.interpret_comp σ f g)

/-- **Identities**: pure wiring with no utilities is interpreted as `(id, 0)`. -/
theorem interpret_identity (X : S.Iface) :
    M.interpret σ (IDNet.identity X) = RewardKernel.id ((M.interp σ).Boundary X) := by
  ext1
  · change (M.interp σ).kernel (Net.wire id (fun _ ⇒ rfl)) = Kernel.idK _
    rw [(M.interp σ).kernel_wire]
    apply Kernel.ofFun_congr
    intro x
    rfl
  · funext x y
    have hr : ∀ z, M.reward σ (IDNet.identity X) x z = 0 :=
      fun z ⇒ Finset.sum_eq_zero (fun u _ ⇒ (u : Empty).elim)
    change M.rewardTable σ (IDNet.identity X) x y = 0
    unfold rewardTable
    simp [hr]

theorem interpretHom_identity (X : S.Iface) :
    M.interpretHom σ (IDNet.identity X) = 𝟙 (M.space σ X) :=
  Subtype.ext (M.interpret_identity σ X)
```

Functoriality under tensor

```
/-- The canonical bijection between pairs of boundary assignments and assignments of the
disjoint-union interface. -/
def join (X X' : S.Iface) :
    (M.interp σ).Boundary X × (M.interp σ).Boundary X' ≃ (M.interp σ).Boundary (X.sum X') :=
  (Interpretation.Assignment.sumEquiv (M.interp σ) X.type X'.type).symm

@[simp] theorem join_apply (X X' : S.Iface) (p : (M.interp σ).Boundary X ×
    (M.interp σ).Boundary X') :
    M.join σ X X' p = Interpretation.Assignment.join _ p.1 p.2 := rfl

theorem rewardTable_tensor (f : IDNet X Y) (g : IDNet X' Y') (x : (M.interp σ).Boundary X)
    (x' : (M.interp σ).Boundary X') (y : (M.interp σ).Boundary Y)
    (y' : (M.interp σ).Boundary Y') :
    M.rewardTable σ (f.tensor g) (Interpretation.Assignment.join _ x x')
        (Interpretation.Assignment.join _ y y') =
```

```
      M.rewardTable σ f x y * (M.interp σ).kernel g.net x' y' +
        (M.interp σ).kernel f.net x y * M.rewardTable σ g x' y' := by
  simp only [kernel_eq_sum]
  unfold rewardTable
  rw [Finset.sum_mul_sum, Finset.sum_mul_sum, ← Finset.sum_add_distrib]
  change (∑ z : (M.interp σ).Internal (f.net.tensor g.net),
      if Interpretation.Assignment.join _ y y' =
          (M.interp σ).read (f.net.tensor g.net) (Interpretation.Assignment.join _ x x') z then
        (M.interp σ).joint (f.net.tensor g.net) (Interpretation.Assignment.join _ x x') z *
          M.reward σ (f.tensor g) (Interpretation.Assignment.join _ x x') z else 0) = _
  erw [← ((M.interp σ).sumLabelsEquiv f.net.label g.net.label).symm.sum_comp]
  rw [Fintype.sum_prod_type]
  apply Finset.sum_congr rfl
  intro a _
  rw [← Finset.sum_add_distrib]
  apply Finset.sum_congr rfl
  intro b _
  have hr := (M.interp σ).read_tensor f.net g.net x x' a b
  have hj := (M.interp σ).joint_tensor f.net g.net x x' a b
  have hw := M.reward_tensor σ f g x x' a b
  simp only [Interpretation.combine] at hr hj hw
  rw [hr, hj, hw]
  by_cases h₁ : y = (M.interp σ).read f.net x a <;>
    by_cases h₂ : y' = (M.interp σ).read g.net x' b <;>
      simp [Interpretation.Assignment.join_injective, h₁, h₂]
  ring

theorem kernel_tensor_join (f : IDNet X Y) (g : IDNet X' Y') (x : (M.interp σ).Boundary X)
    (x' : (M.interp σ).Boundary X') (y : (M.interp σ).Boundary Y)
    (y' : (M.interp σ).Boundary Y') :
    (M.interp σ).kernel (f.tensor g).net (Interpretation.Assignment.join _ x x')
        (Interpretation.Assignment.join _ y y') =
      (M.interp σ).kernel f.net x y * (M.interp σ).kernel g.net x' y' :=
  (M.interp σ).kernel_tensor f.net g.net x x' y y'

/-- Read a reward kernel through maps on its input and output states. -/
def _root_.OpenInfluence.RewardKernel.reindex {A B C D : Type} (r : RewardKernel C D)
    (i : A → C) (o : B → D) : RewardKernel A B :=
  ⟨fun a b ⇒ r.K (i a) (o b), fun a b ⇒ r.W (i a) (o b)⟩

/-- **Functoriality under tensor**: read through the canonical bijections between
assignments of a disjoint-union interface and pairs of assignments, the interpretation of
a tensor is the tensor of the interpretations. -/
theorem interpret_tensor (f : IDNet X Y) (g : IDNet X' Y') :
    (M.interpret σ (f.tensor g)).reindex (M.join σ X X') (M.join σ Y Y') =
      (M.interpret σ f).tensor (M.interpret σ g) := by
  ext1
  · funext p q
    exact M.kernel_tensor_join σ f g p.1 p.2 q.1 q.2
  · funext p q
    exact M.rewardTable_tensor σ f g p.1 p.2 q.1 q.2

/-- **Functoriality under tensor**, as naturality of the canonical bijection (the shape of
`OpenNet.Interpretation.tensor_natural`). -/
theorem interpret_tensor_natural (f : IDNet X Y) (g : IDNet X' Y') :
    ((M.interpret σ f).tensor (M.interpret σ g)).comp (RewardKernel.ofFun (M.join σ Y Y')) =
      (RewardKernel.ofFun (M.join σ X X')).comp (M.interpret σ (f.tensor g)) := by
  rw [← M.interpret_tensor σ f g]
  ext1
  · simp only [RewardKernel.comp_K, RewardKernel.ofFun, RewardKernel.lift_K,
      Kernel.comp_ofEquiv_right, Kernel.comp_ofFun_left, RewardKernel.reindex]
    funext p q
    simp
  · simp only [RewardKernel.comp_W, RewardKernel.ofFun, RewardKernel.lift_K,
      RewardKernel.lift_W, KernelAlgebra.kcomp_zero_left, KernelAlgebra.kcomp_zero_right,
      add_zero, zero_add, Kernel.comp_ofEquiv_right, Kernel.comp_ofFun_left,
```

```
      RewardKernel.reindex]
    funext p q
    simp
```

The contextual identity for interpreted diagrams

```
/-- **Contextual identity** for open influence diagrams under a fixed strategy: for every
boundary prior `μ` and terminal continuation `c`,
`EU_{f;g}(μ, c) = EU_f(μ, c_g) = L_f + EU_g(ν_f, c)`, where `c_g` is the suffix's value
function, `ν_f` the boundary law induced by the prefix and `L_f` the prefix's reward. -/
theorem contextual_identity (f : IDNet X Y) (g : IDNet Y Z)
    (μ : (M.interp σ).Boundary X → ℝ) (c : (M.interp σ).Boundary Z → ℝ) :
    (M.interpret σ (f.comp g)).eu μ c =
        (M.interpret σ f).eu μ ((M.interpret σ g).continuation c) ∧
      (M.interpret σ f).eu μ ((M.interpret σ g).continuation c) =
        (M.interpret σ f).prefixReward μ +
          (M.interpret σ g).eu ((M.interpret σ f).pushforward μ) c := by
  rw [M.interpret_comp σ f g]
  exact RewardKernel.contextual_identity _ _ (M.interpret_stochastic σ g).2 μ c

end IDInterpretation

end OpenInfluence
```

Optimisation does not compose component by component

```
import CategoricalInfluenceDiagramsProofs.Context
import Mathlib.Tactic.NormNum
```

The contextual identity says each component's value depends on its context: the prefix sees the suffix only through the value function `c_g`, the suffix sees the prefix only through the induced boundary law `ν_f`. This module proves that the context cannot be dropped, with the finite witness of the sealed bundle's Lean track (`INF-ISOLATED-OBSTRUCTION`, `INF-HIDDEN-OBSTRUCTION`), restated in the reward-kernel semantics and not imported from the bundle.

The witness. A fair hidden Boolean `H` and an early action `A` that observes nothing form the prefix `early a : Unit → H × A`, which chooses `A = a` and pays utility `-[A]`. The suffix `late τ : H × A → Unit` is an action `B = τ A` that observes only `A` (so `H` is hidden at the boundary) and pays utilities `4[A]` and `2[B = H]`. The closed composite from the unit prior with zero continuation has expected utility `1 + 3[a]`.

Isolated optimisation (`isolated_optimisation_fails`). Optimised on its own (unit prior, zero continuation), the prefix chooses `a = false`, since its only utility is `-[A]`. Optimised on its own against the boundary law that this prefix actually induces, every suffix policy has value `1`. So **every** pair of separately optimal components has composite value `1`, while the composite with `a = true` attains `4`, the joint optimum: `1 < 4`.

Pointwise maximisation at a hidden boundary state (`hidden_input_obstruction`). Against the same boundary law, every legal suffix policy (a function of `A` only) has value `1`. Choosing `B` to maximise reward separately at each boundary state `(H, A)` gives the selector `B = H`, whose value is `2`; it reads the hidden `H` and is not a legal policy.

Both are statements about the fixed-strategy semantics; they are not instances of the typed syntax of `Syntax.lean`, whose interpretation they would be (the theorems there reduce such a diagram to these reward kernels by `interpret_comp`).

```
set_option autoImplicit false

noncomputable section

namespace OpenInfluence

namespace Obstruction
```

```
open FiniteKernelsProofs.Finite RewardKernel

/-- The boundary between the components: the hidden `H` and the observed `A`. -/
abbrev Boundary : Type := Bool × Bool

/-- The prefix with early action `A = a`: a fair hidden `H`, and utility `-[A]`. -/
def early (a : Bool) : RewardKernel Unit Boundary :=
  ⟨fun _ s ⇒ if s.2 = a then 1 / 2 else 0,
    fun _ s ⇒ if s.2 = a then 1 / 2 * (if s.2 then -1 else 0) else 0⟩

/-- The suffix whose action `B` is chosen by a selector on the whole boundary state, with
utilities `4[A]` and `2[B = H]`. -/
def lateSel (sel : Boundary → Bool) : RewardKernel Boundary Unit :=
  ⟨fun _ _ ⇒ 1, fun s _ ⇒ (if s.2 then 4 else 0) + (if sel s = s.1 then 2 else 0)⟩

/-- A legal suffix policy reads only the observed `A`. -/
def late (τ : Bool → Bool) : RewardKernel Boundary Unit := lateSel fun s ⇒ τ s.2

/-- A selector is a legal policy when it reads only the observed coordinate. -/
def Legal (sel : Boundary → Bool) : Prop := ∃ τ : Bool → Bool, ∀ s, sel s = τ s.2

/-- The unit prior on the closed input. -/
def unitPrior : Unit → ℝ := fun _ ⇒ 1

/-- The zero terminal continuation. -/
def zeroCont {Y : Type} : Y → ℝ := fun _ ⇒ 0

/-- The boundary law induced by the prefix `early a`. -/
def boundaryLaw (a : Bool) : Boundary → ℝ := (early a).pushforward unitPrior

/-- `p` is optimal among the members of a family of reward kernels for the prior `μ` and
the continuation `c`, the component considered on its own. -/
def IsolatedOptimal {X Y P : Type} [Fintype X] [Fintype Y] (family : P → RewardKernel X Y)
    (μ : X → ℝ) (c : Y → ℝ) (p : P) : Prop :=
  ∀ q, (family q).eu μ c ≤ (family p).eu μ c
```

Values

```
theorem early_eu (a : Bool) : (early a).eu unitPrior zeroCont = if a then -1 else 0 := by
  cases a <;>
    simp [eu, early, unitPrior, zeroCont, Fintype.sum_prod_type]

theorem early_prefixReward (a : Bool) :
    (early a).prefixReward unitPrior = if a then -1 else 0 := by
  cases a <;>
    simp [prefixReward, early, unitPrior, Fintype.sum_prod_type]

theorem late_normalised (sel : Boundary → Bool) : Kernel.Normalised (lateSel sel).K := by
  intro s
  simp [lateSel]

theorem lateSel_eu (sel : Boundary → Bool) (a : Bool) :
    (lateSel sel).eu (boundaryLaw a) zeroCont =
      (if a then 4 else 0) + ((if sel (true, a) = true then 1 else 0) +
        (if sel (false, a) = false then 1 else 0)) := by
  simp only [eu, lateSel, boundaryLaw, pushforward, early, unitPrior, zeroCont,
    Fintype.sum_prod_type, Fintype.sum_bool, Finset.univ_unique, Finset.sum_singleton]
  cases a <;> simp only [Bool.true_eq_false, Bool.false_eq_true, if_true, if_false] <;>
    split_ifs <;> norm_num

theorem late_eu (τ : Bool → Bool) (a : Bool) :
    (late τ).eu (boundaryLaw a) zeroCont = (if a then 4 else 0) + 1 := by
  rw [late, lateSel_eu]
  cases τ a <;> simp
```

```
theorem composite_eu (a : Bool) (τ : Bool → Bool) :
    ((early a).comp (late τ)).eu unitPrior zeroCont = 1 + (if a then 3 else 0) := by
  rw [eu_comp (early a) (late τ) (late_normalised _), eu_continuation, early_prefixReward]
  change _ + (late τ).eu (boundaryLaw a) zeroCont = _
  rw [late_eu]
  cases a <;> norm_num
```

The isolated-optimisation obstruction

```
theorem early_isolatedOptimal_iff (a : Bool) :
    IsolatedOptimal early unitPrior zeroCont a ↔ a = false := by
  constructor
  · intro h
    have := h false
    rw [early_eu, early_eu] at this
    cases a
    · rfl
    · norm_num at this
  · rintro rfl b
    rw [early_eu, early_eu]
    cases b <;> norm_num

theorem late_isolatedOptimal (a : Bool) (τ : Bool → Bool) :
    IsolatedOptimal late (boundaryLaw a) zeroCont τ := by
  intro τ'
  rw [late_eu, late_eu]

/-- **Isolated optimisation is not globally sound** (`INF-ISOLATED-OBSTRUCTION`). Every
prefix policy that is optimal on its own, followed by every suffix policy that is optimal
on its own against the boundary law that prefix induces, has composite expected utility
`1`; such pairs exist; another pair attains `4`; and `4` is the joint optimum. -/
theorem isolated_optimisation_fails :
    (∀ (a : Bool) (τ : Bool → Bool), IsolatedOptimal early unitPrior zeroCont a →
        IsolatedOptimal late (boundaryLaw a) zeroCont τ →
          ((early a).comp (late τ)).eu unitPrior zeroCont = 1) ∧
      (IsolatedOptimal early unitPrior zeroCont false ∧
        IsolatedOptimal late (boundaryLaw false) zeroCont (fun _ ⇒ false)) ∧
      ((early true).comp (late fun _ ⇒ false)).eu unitPrior zeroCont = 4 ∧
      (∀ (a : Bool) (τ : Bool → Bool),
        ((early a).comp (late τ)).eu unitPrior zeroCont ≤ 4) ∧
      (1 : ℝ) < 4 := by
  refine ⟨?_, ⟨((early_isolatedOptimal_iff false).mpr rfl, late_isolatedOptimal _ _⟩, ?_, ?_,
    by norm_num⟩
  · intro a τ ha _
    rw [(early_isolatedOptimal_iff a).mp ha, composite_eu]
    norm_num
  · rw [composite_eu]
    norm_num
  · intro a τ
    rw [composite_eu]
    cases a <;> norm_num
```

The hidden-input obstruction

```
/-- The selector that maximises the suffix's reward separately at each boundary state. -/
def peek : Boundary → Bool := Prod.fst

theorem peek_pointwise_max (sel : Boundary → Bool) (s : Boundary) :
    (lateSel sel).W s () ≤ (lateSel peek).W s () := by
  simp only [lateSel, peek]
  split_ifs <;> norm_num

theorem peek_not_legal : ¬ Legal peek := by
```

```
  rintro ⟨τ, h⟩
  have h1 := h (true, false)
  have h2 := h (false, false)
  simp only [peek] at h1 h2
  rw [← h1] at h2
  exact Bool.false_ne_true h2

/-- **Pointwise maximisation at a hidden boundary state is not a legal policy**
(`INF-HIDDEN-OBSTRUCTION`). Against the boundary law of the isolated-optimal prefix, every
legal suffix policy has value `1`, while the selector that maximises at each boundary state
(reading the hidden `H`) has value `2` and is not legal. -/
theorem hidden_input_obstruction :
    (∀ sel, Legal sel → (lateSel sel).eu (boundaryLaw false) zeroCont = 1) ∧
      (∀ sel s, (lateSel sel).W s () ≤ (lateSel peek).W s ()) ∧
      (lateSel peek).eu (boundaryLaw false) zeroCont = 2 ∧
      ¬ Legal peek ∧ (1 : ℝ) < 2 := by
  refine ⟨?_, peek_pointwise_max, ?_, peek_not_legal, by norm_num⟩
  · rintro sel ⟨τ, hτ⟩
    have : sel = fun s ⇒ τ s.2 := funext hτ
    subst this
    exact (late_eu τ false).trans (by norm_num)
  · rw [lateSel_eu]
    simp [peek]
    norm_num

end Obstruction

end OpenInfluence
```

Lenses and open-game equilibria (ADR 0020)

```
import CategoricalInfluenceDiagramsProofs.Obstruction
```

A reward kernel `f = (K, W) : X → Y` transports a context in two directions. Forward, a prior `μ` on `X` becomes the boundary law `ν = μ K` on `Y` (`pushforward`); backward, a continuation `c` on `Y` becomes the value function `x ↦ Σ_y (W x y + K x y · c y)` on `X` (`continuation`). The pair is a lens, and serial composition of reward kernels is sent to composition of lenses when the later kernel is normalised (`lens_comp`, `lens_id`). The expected utility pairs the two directions: `EU_f(μ, c) = L_f(μ) + Σ_y ν y · c y` (`eu_eq_prefixReward_add`).

Open games. Following open games, a `Game X Y` has strategies, a play (a normalised reward kernel for each strategy) and an equilibrium predicate over contexts `(μ, c)`. A *decision* game is one component whose decisions are controlled together; its equilibria are the strategies that maximise expected utility in the context. The serial composite `G.seq H` plays the composite kernel, and `(s, t)` is an equilibrium in `(μ, c)` when `s` is one of `G` in the context `(μ, c_t)` and `t` one of `H` in `(ν_s, c)`: each part sees the context the other induces, transported by the lenses. The tensor is in `GameTensor.lean` (ADR 0020, amended).

Results. - Serial composition is associative and unital on equilibria (`seq_assoc_eq`, `idGame_seq_eq`, `seq_idGame_eq`) and on plays (`seq_assoc_play`), because the induced contexts compose by the lens laws. - For two decision games an equilibrium is exactly a pair from which neither component gains by changing its own strategy, measured by the composite's expected utility (`seq_decision_eq_iff`). - A global optimum is an equilibrium, for every game built from decision games and identities by serial composition (`Sound`, `seq_sound`). - The converse fails (`coordination_equilibrium_not_optimal`): two decisions that must agree, the later one not observing the earlier, have an equilibrium worth `1` where the optimum is `2`. - In the obstruction witness of `Obstruction.lean` the equilibria are exactly the strategies that take the early action, which are the optimal ones (`obstruction_eq_iff`); the isolated choice is not an equilibrium.

Like `Obstruction.lean`, these are statements about the reward-kernel semantics; the play of a component under a strategy is its `interpret` by the theorems of `Interpretation.lean`.

```
set_option autoImplicit false

noncomputable section
```

```
namespace OpenInfluence

open FiniteKernelsProofs.Finite

namespace RewardKernel

variable {X Y Z : Type} [Fintype X] [Fintype Y] [Fintype Z]

omit [Fintype Z] in
/-- The forward map of the lens of a serial composite is the composite of the forward maps. -/
theorem pushforward_comp (f : RewardKernel X Y) (g : RewardKernel Y Z) (μ : X → ℝ) :
    (f.comp g).pushforward μ = g.pushforward (f.pushforward μ) := by
  funext z
  simp only [pushforward, comp_K, Kernel.comp_apply, Finset.mul_sum, Finset.sum_mul]
  rw [Finset.sum_comm]
  exact Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_congr rfl fun _ _ ⇒ by ring

/-- The backward map of the lens of a serial composite with a normalised suffix is the
composite of the backward maps, in the opposite order. -/
theorem continuation_comp (f : RewardKernel X Y) (g : RewardKernel Y Z)
    (hg : Kernel.Normalised g.K) (c : Z → ℝ) :
    (f.comp g).continuation c = f.continuation (g.continuation c) := by
  classical
  funext x₀
  have h := eu_comp f g hg (fun x ⇒ if x = x₀ then 1 else 0) c
  simpa [eu_eq_continuation, ite_mul, Finset.sum_ite_eq'] using h

theorem pushforward_id [DecidableEq X] (μ : X → ℝ) : (id X).pushforward μ = μ := by
  funext y
  simp [pushforward, id, lift, Kernel.idK_apply, mul_ite, Finset.sum_ite_eq]

theorem continuation_id [DecidableEq X] (c : X → ℝ) : (id X).continuation c = c := by
  funext x
  simp [continuation, id, lift, Kernel.idK_apply, ite_mul]

/-- The expected utility pairs the two directions of the lens: the prefix reward plus the
continuation integrated against the boundary law. -/
theorem eu_eq_prefixReward_add (f : RewardKernel X Y) (μ : X → ℝ) (c : Y → ℝ) :
    f.eu μ c = f.prefixReward μ + ∑ y, f.pushforward μ y * c y := by
  simp only [eu, prefixReward, pushforward, mul_add, Finset.sum_add_distrib, Finset.mul_sum,
    Finset.sum_mul]
  congr 1
  rw [Finset.sum_comm]
  exact Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_congr rfl fun _ _ ⇒ by ring

end RewardKernel

/-- A lens on real functions: forward on priors, backward on continuations. -/
structure Lens (X Y : Type) where
  /-- The forward map, a prior on `X` to a law on `Y`. -/
  fwd : (X → ℝ) → (Y → ℝ)
  /-- The backward map, a continuation on `Y` to a value function on `X`. -/
  bwd : (Y → ℝ) → (X → ℝ)

namespace Lens

variable {X Y Z : Type}

/-- Composition of lenses: forward maps in order, backward maps in reverse. -/
def comp (l : Lens X Y) (m : Lens Y Z) : Lens X Z := ⟨m.fwd ∘ l.fwd, l.bwd ∘ m.bwd⟩

/-- The identity lens. -/
def id (X : Type) : Lens X X := ⟨fun μ ⇒ μ, fun c ⇒ c⟩

end Lens
```

```
namespace RewardKernel

variable {X Y Z : Type} [Fintype X] [Fintype Y] [Fintype Z]

/-- The lens of a reward kernel: the boundary law forward, the value function backward. -/
def lens (f : RewardKernel X Y) : Lens X Y := ⟨f.pushforward, f.continuation⟩

/-- **Lens functoriality**: the lens of a serial composite with a normalised suffix is the
composite of the lenses. -/
theorem lens_comp (f : RewardKernel X Y) (g : RewardKernel Y Z) (hg : Kernel.Normalised g.K) :
    (f.comp g).lens = f.lens.comp g.lens := by
  simp only [lens, Lens.comp, Lens.mk.injEq]
  exact ⟨funext (pushforward_comp f g), funext (continuation_comp f g hg)⟩

/-- The lens of an identity is the identity lens. -/
theorem lens_id [DecidableEq X] : (id X).lens = Lens.id X := by
  simp only [lens, Lens.id, Lens.mk.injEq]
  exact ⟨funext pushforward_id, funext continuation_id⟩

end RewardKernel

/-- An open game `X → Y` over reward kernels: strategies, a normalised play for each, and an
equilibrium predicate over contexts `(μ, c)`. -/
structure Game (X Y : Type) [Fintype X] [Fintype Y] where
  /-- The strategies. -/
  Strat : Type
  /-- The reward kernel played by each strategy. -/
  play : Strat → RewardKernel X Y
  /-- Every play has a normalised probability component. -/
  normalised : ∀ s, Kernel.Normalised (play s).K
  /-- `Eq μ c s`: the strategy `s` is an equilibrium in the context `(μ, c)`. -/
  Eq : (X → ℝ) → (Y → ℝ) → Strat → Prop

namespace Game

open RewardKernel

variable {X Y Z V : Type} [Fintype X] [Fintype Y] [Fintype Z] [Fintype V]

/-- A decision game: one component whose decisions are controlled together, in equilibrium
exactly when its strategy maximises expected utility in the context. -/
def decision {S : Type} (play : S → RewardKernel X Y)
    (h : ∀ s, Kernel.Normalised (play s).K) : Game X Y where
  Strat := S
  play := play
  normalised := h
  Eq μ c s := ∀ s', (play s').eu μ c ≤ (play s).eu μ c

/-- The identity game, with one strategy, always in equilibrium. -/
def idGame (X : Type) [Fintype X] [DecidableEq X] : Game X X where
  Strat := Unit
  play _ := RewardKernel.id X
  normalised _ := Kernel.Normalised.idK
  Eq _ _ _ := True

/-- Serial composition: each part is in equilibrium in the context the other induces, the
suffix's value function for the prefix and the prefix's boundary law for the suffix. -/
def seq (G : Game X Y) (H : Game Y Z) : Game X Z where
  Strat := G.Strat × H.Strat
  play p := (G.play p.1).comp (H.play p.2)
  normalised p := Kernel.Normalised.comp (G.normalised p.1) (H.normalised p.2)
  Eq μ c p := G.Eq μ ((H.play p.2).continuation c) p.1 ∧
    H.Eq ((G.play p.1).pushforward μ) c p.2
```

Serial composition is a category on equilibria

```
theorem seq_assoc_play (G : Game X Y) (H : Game Y Z) (J : Game Z V) (s : G.Strat)
    (t : H.Strat) (u : J.Strat) :
    ((G.seq H).seq J).play ((s, t), u) = (G.seq (H.seq J)).play (s, (t, u)) :=
  RewardKernel.comp_assoc _ _ _

/-- **A chain of three games.** A strategy of `(G ; H) ; J` is an equilibrium exactly when
each game's strategy is an equilibrium in the context the lenses of the others transport to
it: `G` against the value functions of `H` and `J` in turn, `H` against the boundary law of
`G` and the value function of `J`, `J` against the boundary laws of `G` and `H` in turn. -/
theorem seq_seq_eq_iff (G : Game X Y) (H : Game Y Z) (J : Game Z V) (μ : X → ℝ) (c : V → ℝ)
    (s : G.Strat) (t : H.Strat) (u : J.Strat) :
    ((G.seq H).seq J).Eq μ c ((s, t), u) ↔
      G.Eq μ ((H.play t).continuation ((J.play u).continuation c)) s ∧
        H.Eq ((G.play s).pushforward μ) ((J.play u).continuation c) t ∧
        J.Eq ((H.play t).pushforward ((G.play s).pushforward μ)) c u := by
  change (G.Eq μ ((H.play t).continuation ((J.play u).continuation c)) s ∧
      H.Eq ((G.play s).pushforward μ) ((J.play u).continuation c) t) ∧
      J.Eq (((G.play s).comp (H.play t)).pushforward μ) c u ↔ _
  rw [pushforward_comp, and_assoc]

/-- **Associativity on equilibria.** Bracketing a chain of three games differently changes
neither its plays nor its equilibria, because the induced contexts compose by the lens laws. -/
theorem seq_assoc_eq (G : Game X Y) (H : Game Y Z) (J : Game Z V) (μ : X → ℝ) (c : V → ℝ)
    (s : G.Strat) (t : H.Strat) (u : J.Strat) :
    ((G.seq H).seq J).Eq μ c ((s, t), u) ↔ (G.seq (H.seq J)).Eq μ c (s, (t, u)) := by
  rw [seq_seq_eq_iff]
  change _ ↔ G.Eq μ (((H.play t).comp (J.play u)).continuation c) s ∧
      (H.Eq ((G.play s).pushforward μ) ((J.play u).continuation c) t ∧
        J.Eq ((H.play t).pushforward ((G.play s).pushforward μ)) c u)
  rw [continuation_comp _ _ (J.normalised u)]

theorem idGame_seq_eq [DecidableEq X] (G : Game X Y) (μ : X → ℝ) (c : Y → ℝ) (s : G.Strat) :
    ((idGame X).seq G).Eq μ c ((), s) ↔ G.Eq μ c s := by
  change True ∧ G.Eq ((RewardKernel.id X).pushforward μ) c s ↔ _
  rw [pushforward_id, true_and]

theorem seq_idGame_eq [DecidableEq Y] (G : Game X Y) (μ : X → ℝ) (c : Y → ℝ) (s : G.Strat) :
    (G.seq (idGame Y)).Eq μ c (s, ()) ↔ G.Eq μ c s := by
  change G.Eq μ ((RewardKernel.id Y).continuation c) s ∧ True ↔ _
  rw [continuation_id, and_true]
```

Equilibria of two decisions are person-by-person optima

```
/-- **Equilibrium of two decision games.** `(s, t)` is an equilibrium of the serial composite
of two decision games exactly when neither component can raise the composite's expected
utility by changing its own strategy alone. -/
theorem seq_decision_eq_iff {S T : Type} (f : S → RewardKernel X Y) (g : T → RewardKernel Y Z)
    (hf : ∀ s, Kernel.Normalised (f s).K) (hg : ∀ t, Kernel.Normalised (g t).K)
    (μ : X → ℝ) (c : Z → ℝ) (s : S) (t : T) :
    ((decision f hf).seq (decision g hg)).Eq μ c (s, t) ↔
      (∀ s', ((f s').comp (g t)).eu μ c ≤ ((f s).comp (g t)).eu μ c) ∧
        (∀ t', ((f s).comp (g t')).eu μ c ≤ ((f s).comp (g t)).eu μ c) := by
  change (∀ s', (f s').eu μ ((g t).continuation c) ≤ (f s).eu μ ((g t).continuation c)) ∧
      (∀ t', (g t').eu ((f s).pushforward μ) c ≤ (g t).eu ((f s).pushforward μ) c) ↔ _
  simp only [eu_comp _ _ (hg _), eu_continuation, add_le_add_iff_left]
```

Optima are equilibria

```
/-- A game is sound when every strategy that maximises expected utility in a context is an
equilibrium there. -/
def Sound (G : Game X Y) : Prop :=
  ∀ μ c s, (∀ s', (G.play s').eu μ c ≤ (G.play s).eu μ c) → G.Eq μ c s
```

```
theorem decision_sound {S : Type} (play : S → RewardKernel X Y)
    (h : ∀ s, Kernel.Normalised (play s).K) : (decision play h).Sound :=
  fun _ _ _ hs ⇒ hs

theorem idGame_sound [DecidableEq X] : (idGame X).Sound := fun _ _ _ _ ⇒ trivial

/-- **Optima are equilibria, compositionally.** Serial composition preserves soundness, so a
global optimum of any game built from decision games and identities by serial composition is
an equilibrium of it. -/
theorem seq_sound {G : Game X Y} {H : Game Y Z} (hG : G.Sound) (hH : H.Sound) :
    (G.seq H).Sound := by
  rintro μ c ⟨s, t⟩ hopt
  refine ⟨hG _ _ _ fun s' ⇒ ?_, hH _ _ _ fun t' ⇒ ?_⟩
  · have := hopt (s', t)
    change ((G.play s').comp (H.play t)).eu μ c ≤ ((G.play s).comp (H.play t)).eu μ c at this
    rwa [eu_comp _ _ (H.normalised t), eu_comp _ _ (H.normalised t)] at this
  · have := hopt (s, t')
    change ((G.play s).comp (H.play t')).eu μ c ≤ ((G.play s).comp (H.play t)).eu μ c at this
    rwa [eu_comp _ _ (H.normalised t'), eu_comp _ _ (H.normalised t), eu_continuation,
      eu_continuation, add_le_add_iff_left] at this

end Game
```

The converse fails: a coordination witness

```
namespace Coordination

open RewardKernel Obstruction

/-- The early decision chooses `A = a` and earns nothing. -/
def first (a : Bool) : RewardKernel Unit Bool :=
  ⟨fun _ y ⇒ if y = a then 1 else 0, fun _ _ ⇒ 0⟩

/-- The payoff of the late decision `B = b` against the early `A`: 2 when both are taken,
1 when neither is, 0 otherwise. -/
def payoff (a b : Bool) : ℝ :=
  if a = true ∧ b = true then 2 else if a = false ∧ b = false then 1 else 0

/-- The late decision chooses `B = b` without observing `A`, and earns `payoff A b`. -/
def second (b : Bool) : RewardKernel Bool Unit := ⟨fun _ _ ⇒ 1, fun y _ ⇒ payoff y b⟩

theorem first_normalised (a : Bool) : Kernel.Normalised (first a).K := by
  intro u
  cases a <;> simp [first]

theorem second_normalised (b : Bool) : Kernel.Normalised (second b).K := by
  intro y
  simp [second]

/-- The two decisions in series, as a game. -/
def game : Game Unit Unit :=
  (Game.decision first first_normalised).seq (Game.decision second second_normalised)

theorem composite_eu (a b : Bool) :
    ((first a).comp (second b)).eu unitPrior zeroCont = payoff a b := by
  rw [eu_comp _ _ (second_normalised b), eu_continuation]
  cases a <;> cases b <;>
    simp [eu, prefixReward, pushforward, first, second, payoff, unitPrior, zeroCont]

theorem game_eq_iff (a b : Bool) :
    game.Eq unitPrior zeroCont (a, b) ↔
      (∀ a', payoff a' b ≤ payoff a b) ∧ (∀ b', payoff a b' ≤ payoff a b) := by
  refine (Game.seq_decision_eq_iff first second first_normalised second_normalised unitPrior
    zeroCont a b).trans ?_
```

```
  simp only [composite_eu]

/-- **An equilibrium need not be optimal.** Not taking either action is an equilibrium worth
`1`, taking both is an equilibrium worth `2`, and `2` is the optimum. -/
theorem coordination_equilibrium_not_optimal :
    game.Eq unitPrior zeroCont (false, false) ∧ game.Eq unitPrior zeroCont (true, true) ∧
      ((first false).comp (second false)).eu unitPrior zeroCont = 1 ∧
      ((first true).comp (second true)).eu unitPrior zeroCont = 2 ∧
      (∀ a b, ((first a).comp (second b)).eu unitPrior zeroCont ≤ 2) ∧ (1 : ℝ) < 2 := by
  refine ⟨(game_eq_iff _ _).mpr ⟨fun a ⇒ ?_, fun b ⇒ ?_⟩,
    (game_eq_iff _ _).mpr ⟨fun a ⇒ ?_, fun b ⇒ ?_⟩, by rw [composite_eu]; simp [payoff],
    by rw [composite_eu]; simp [payoff], fun a b ⇒ ?_, by norm_num⟩
  · cases a <;> norm_num [payoff]
  · cases b <;> norm_num [payoff]
  · cases a <;> norm_num [payoff]
  · cases b <;> norm_num [payoff]
  · rw [composite_eu]
    cases a <;> cases b <;> norm_num [payoff]

end Coordination
```

Equilibria of the obstruction witness

```
namespace Obstruction

open RewardKernel

theorem early_normalised (a : Bool) : Kernel.Normalised (early a).K := by
  intro u
  cases a <;> simp [early, Fintype.sum_prod_type]

/-- The obstruction witness as a game: the early decision, then the late policy. -/
def game : Game Unit Unit :=
  (Game.decision early early_normalised).seq (Game.decision late fun _ ⇒ late_normalised _)

/-- **The equilibria of the obstruction witness** are exactly the strategies that take the
early action, which are its optima. The isolated choice `a = false` is not an equilibrium:
against the late component's value function, taking the action gains `3`. -/
theorem obstruction_eq_iff (a : Bool) (τ : Bool → Bool) :
    game.Eq unitPrior zeroCont (a, τ) ↔ a = true := by
  refine (Game.seq_decision_eq_iff early late early_normalised (fun _ ⇒ late_normalised _)
    unitPrior zeroCont a τ).trans ?_
  simp only [composite_eu]
  constructor
  · rintro ⟨h, -⟩
    have := h true
    cases a
    · norm_num at this
    · rfl
  · rintro rfl
    exact ⟨fun a' ⇒ by cases a' <;> norm_num, fun _ ⇒ le_rfl⟩

end Obstruction

end OpenInfluence
```

The tensor of open games (ADR 0020, amended)

```
import CategoricalInfluenceDiagramsProofs.Lens
import Mathlib.Tactic.Linarith
```

ADR 0020 left the tensor of games undefined: the context a factor of `G ⊗ H` receives can depend on the other

factor's input state. This module makes that precise in both directions.

The tensor. `Game.tensor G H` plays `play s ⊗ play t`, and `(s, t)` is an equilibrium in `(μ, c)` when `s` is one of `G` in the context of the first marginal of `μ` and the continuation the second factor induces (`leftCont`: its reward, and the continuation `c`, averaged over its input under the second marginal and over its output), and symmetrically for `t`. When the prior is a product of probability vectors, this context is exact: the composite's expected utility is the first factor's expected utility in it (`tensor_eu_left`, `tensor_eu_right`). So for two decision games and a product prior, an equilibrium of the tensor is exactly a pair from which neither factor gains alone, measured by the composite's expected utility (`tensor_decision_eq_iff`).

Correlated priors. No such context exists in general. With two binary inputs that are equal, a first factor that copies its input and a second that passes its input through, and a payoff for matching outputs, the best responses of the first factor are exactly the policy that copies (`correlated_best_responses`). But in no context of the first factor alone, a probability vector on its input and a continuation on its output, are the decision game's equilibria exactly that policy (`correlated_no_factor_context`): the continuation would have to depend on the input, through the correlation. A tensor that is exact for correlated priors needs contexts in which the continuation reads the input, which ADR 0020's `Game` does not have.

```
set_option autoImplicit false

noncomputable section

namespace OpenInfluence

open FiniteKernelsProofs.Finite

namespace Game

open RewardKernel

variable {X Y X' Y' : Type} [Fintype X] [Fintype Y] [Fintype X'] [Fintype Y']

/-- The first marginal of a prior on a product. -/
def marg₁ (μ : X × X' → ℝ) : X → ℝ := fun x ⇒ ∑ x', μ (x, x')

/-- The second marginal of a prior on a product. -/
def marg₂ (μ : X × X' → ℝ) : X' → ℝ := fun x' ⇒ ∑ x, μ (x, x')

omit [Fintype Y] [Fintype X] in
/-- The continuation the second factor `g` induces for the first, with law `ν` on its input:
its reward and the continuation `c`, averaged over its input and its output. -/
def leftCont (g : RewardKernel X' Y') (ν : X' → ℝ) (c : Y × Y' → ℝ) : Y → ℝ :=
  fun y ⇒ ∑ x', ν x' * ∑ y', (g.W x' y' + g.K x' y' * c (y, y'))

omit [Fintype Y'] [Fintype X'] in
/-- The continuation the first factor `f` induces for the second, with law `ν` on its input. -/
def rightCont (f : RewardKernel X Y) (ν : X → ℝ) (c : Y × Y' → ℝ) : Y' → ℝ :=
  fun y' ⇒ ∑ x, ν x * ∑ y, (f.W x y + f.K x y * c (y, y'))

/-- The tensor of open games: each factor in equilibrium in the context of its marginal prior
and the continuation the other factor induces. Exact for product priors
(`tensor_decision_eq_iff`); no context of a factor alone is exact for correlated priors
(`correlated_no_factor_context`). -/
def tensor (G : Game X Y) (H : Game X' Y') : Game (X × X') (Y × Y') where
  Strat := G.Strat × H.Strat
  play p := (G.play p.1).tensor (H.play p.2)
  normalised p := Kernel.Normalised.tensor (G.normalised p.1) (H.normalised p.2)
  Eq μ c p := G.Eq (marg₁ μ) (leftCont (H.play p.2) (marg₂ μ) c) p.1 ∧
    H.Eq (marg₂ μ) (rightCont (G.play p.1) (marg₁ μ) c) p.2

/-- The product of two priors. -/
def prodPrior (μ₁ : X → ℝ) (μ₂ : X' → ℝ) : X × X' → ℝ := fun p ⇒ μ₁ p.1 * μ₂ p.2

omit [Fintype X] in
theorem marg₁_prod (μ₁ : X → ℝ) (μ₂ : X' → ℝ) (h₂ : ∑ x', μ₂ x' = 1) :
    marg₁ (prodPrior μ₁ μ₂) = μ₁ := by
```

```
  funext x
  simp [marg₁, prodPrior, ← Finset.mul_sum, h₂]

omit [Fintype X'] in
theorem marg₂_prod (μ₁ : X → ℝ) (μ₂ : X' → ℝ) (h₁ : ∑ x, μ₁ x = 1) :
    marg₂ (prodPrior μ₁ μ₂) = μ₂ := by
  funext x'
  simp [marg₂, prodPrior, ← Finset.sum_mul, h₁]

omit [Fintype X] [Fintype Y] [Fintype X'] [Fintype Y'] in
/-- Reordering a fourfold sum. -/
theorem sum_comm4 {A B C D : Type} [Fintype A] [Fintype B] [Fintype C] [Fintype D]
    (F : A → B → C → D → ℝ) :
    ∑ a, ∑ b, ∑ c, ∑ d, F a b c d = ∑ b, ∑ d, ∑ a, ∑ c, F a b c d := by
  calc ∑ a, ∑ b, ∑ c, ∑ d, F a b c d = ∑ a, ∑ b, ∑ d, ∑ c, F a b c d :=
       Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_comm
    _ = ∑ b, ∑ a, ∑ d, ∑ c, F a b c d := Finset.sum_comm
    _ = ∑ b, ∑ d, ∑ a, ∑ c, F a b c d := Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_comm

/-- The expected utility of a tensor with a product prior, as a fourfold sum. -/
theorem tensor_eu_sum (f : RewardKernel X Y) (g : RewardKernel X' Y') (μ₁ : X → ℝ)
    (μ₂ : X' → ℝ) (c : Y × Y' → ℝ) :
    (f.tensor g).eu (prodPrior μ₁ μ₂) c = ∑ x, ∑ x', ∑ y, ∑ y', μ₁ x * μ₂ x' *
      (f.W x y * g.K x' y' + f.K x y * g.W x' y' + f.K x y * g.K x' y' * c (y, y')) := by
  unfold eu prodPrior
  simp only [tensor_K, tensor_W, Kernel.tensor_apply, Pi.add_apply, Fintype.sum_prod_type,
    Finset.mul_sum]

/-- With a product prior whose second factor is a probability vector, the expected utility of a
tensor is the first factor's expected utility in the context the second induces. -/
theorem tensor_eu_left (f : RewardKernel X Y) (g : RewardKernel X' Y')
    (hg : Kernel.Normalised g.K) (μ₁ : X → ℝ) (μ₂ : X' → ℝ) (h₂ : ∑ x', μ₂ x' = 1)
    (c : Y × Y' → ℝ) :
    (f.tensor g).eu (prodPrior μ₁ μ₂) c = f.eu μ₁ (leftCont g μ₂ c) := by
  have hone : ∑ x', μ₂ x' * ∑ y', g.K x' y' = 1 := by simp [hg _, h₂]
  have rhs : f.eu μ₁ (leftCont g μ₂ c) = ∑ x, ∑ y, ∑ x', ∑ y', μ₁ x * μ₂ x' *
      (f.W x y * g.K x' y' + f.K x y * g.W x' y' + f.K x y * g.K x' y' * c (y, y')) := by
    unfold eu leftCont
    refine Finset.sum_congr rfl fun x _ ⇒ ?_
    rw [Finset.mul_sum]
    refine Finset.sum_congr rfl fun y _ ⇒ ?_
    conv_lhs ⇒ rw [← mul_one (f.W x y), ← hone]
    simp only [Finset.mul_sum, mul_add, ← Finset.sum_add_distrib]
    refine Finset.sum_congr rfl fun x' _ ⇒ Finset.sum_congr rfl fun y' _ ⇒ by ring
  rw [rhs, tensor_eu_sum]
  exact Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_comm

/-- With a product prior whose first factor is a probability vector, the expected utility of a
tensor is the second factor's expected utility in the context the first induces. -/
theorem tensor_eu_right (f : RewardKernel X Y) (g : RewardKernel X' Y')
    (hf : Kernel.Normalised f.K) (μ₁ : X → ℝ) (μ₂ : X' → ℝ) (h₁ : ∑ x, μ₁ x = 1)
    (c : Y × Y' → ℝ) :
    (f.tensor g).eu (prodPrior μ₁ μ₂) c = g.eu μ₂ (rightCont f μ₁ c) := by
  have hone : ∑ x, μ₁ x * ∑ y, f.K x y = 1 := by simp [hf _, h₁]
  have rhs : g.eu μ₂ (rightCont f μ₁ c) = ∑ x', ∑ y', ∑ x, ∑ y, μ₁ x * μ₂ x' *
      (f.W x y * g.K x' y' + f.K x y * g.W x' y' + f.K x y * g.K x' y' * c (y, y')) := by
    unfold eu rightCont
    refine Finset.sum_congr rfl fun x' _ ⇒ ?_
    rw [Finset.mul_sum]
    refine Finset.sum_congr rfl fun y' _ ⇒ ?_
    conv_lhs ⇒ rw [← mul_one (g.W x' y'), ← hone]
    simp only [Finset.mul_sum, mul_add, ← Finset.sum_add_distrib]
    refine Finset.sum_congr rfl fun x _ ⇒ Finset.sum_congr rfl fun y _ ⇒ by ring
  rw [rhs, tensor_eu_sum, sum_comm4]

/-- **The tensor of two decision games, with a product prior**: an equilibrium is exactly a
```

```
/-- pair from which neither factor can raise the composite's expected utility alone. -/
theorem tensor_decision_eq_iff {S T : Type} (pf : S → RewardKernel X Y)
    (hf : ∀ s, Kernel.Normalised (pf s).K) (pg : T → RewardKernel X' Y')
    (hg : ∀ t, Kernel.Normalised (pg t).K) (μ₁ : X → ℝ) (μ₂ : X' → ℝ) (h₁ : ∑ x, μ₁ x = 1)
    (h₂ : ∑ x', μ₂ x' = 1) (c : Y × Y' → ℝ) (s : S) (t : T) :
    ((decision pf hf).tensor (decision pg hg)).Eq (prodPrior μ₁ μ₂) c (s, t) ↔
      (∀ s', ((pf s').tensor (pg t)).eu (prodPrior μ₁ μ₂) c ≤
        ((pf s).tensor (pg t)).eu (prodPrior μ₁ μ₂) c) ∧
      ∀ t', ((pf s).tensor (pg t')).eu (prodPrior μ₁ μ₂) c ≤
        ((pf s).tensor (pg t)).eu (prodPrior μ₁ μ₂) c := by
  have L : ∀ s' t', ((pf s').tensor (pg t')).eu (prodPrior μ₁ μ₂) c =
      (pf s').eu μ₁ (leftCont (pg t') μ₂ c) := fun s' t' ⇒ tensor_eu_left _ _ (hg t') μ₁ μ₂ h₂ c
  have R : ∀ s' t', ((pf s').tensor (pg t')).eu (prodPrior μ₁ μ₂) c =
      (pg t').eu μ₂ (rightCont (pf s') μ₁ c) := fun s' t' ⇒ tensor_eu_right _ _ (hf s') μ₁ μ₂ h₁ c
  change (∀ s', (pf s').eu (marg₁ (prodPrior μ₁ μ₂)) (leftCont (pg t) (marg₂ (prodPrior μ₁ μ₂)) c) ≤
      (pf s).eu (marg₁ (prodPrior μ₁ μ₂)) (leftCont (pg t) (marg₂ (prodPrior μ₁ μ₂)) c)) ∧
    (∀ t', (pg t').eu (marg₂ (prodPrior μ₁ μ₂)) (rightCont (pf s) (marg₁ (prodPrior μ₁ μ₂)) c) ≤
      (pg t).eu (marg₂ (prodPrior μ₁ μ₂)) (rightCont (pf s) (marg₁ (prodPrior μ₁ μ₂)) c)) ↔ _
  rw [marg₁_prod μ₁ μ₂ h₂, marg₂_prod μ₁ μ₂ h₁]
  constructor
  · rintro ⟨h1, h2⟩
    exact ⟨fun s' ⇒ by rw [L, L]; exact h1 s', fun t' ⇒ by rw [R, R]; exact h2 t'⟩
  · rintro ⟨h1, h2⟩
    refine ⟨fun s' ⇒ ?_, fun t' ⇒ ?_⟩
    · have := h1 s'
      rwa [L, L] at this
    · have := h2 t'
      rwa [R, R] at this

end Game
```

Correlated priors: no context of a factor alone

```
namespace CorrelatedTensor

open RewardKernel

/-- The first factor's play: output the state the policy `s` assigns to the input. -/
def copyPlay (s : Bool → Bool) : RewardKernel Bool Bool :=
  ⟨fun x y ⇒ if y = s x then 1 else 0, fun _ _ ⇒ 0⟩

theorem copyPlay_normalised (s : Bool → Bool) : Kernel.Normalised (copyPlay s).K := by
  intro x
  simp [copyPlay]

/-- The first factor as a decision game over all policies. -/
def first : Game Bool Bool := Game.decision copyPlay copyPlay_normalised

/-- The two inputs are equal, each value with probability one half. -/
def prior : Bool × Bool → ℝ := fun p ⇒ if p.1 = p.2 then 1 / 2 else 0

/-- One for matching outputs. -/
def payoff : Bool × Bool → ℝ := fun p ⇒ if p.1 = p.2 then 1 else 0

/-- The composite's value: one half for each input the policy copies. -/
theorem joint_eu (s : Bool → Bool) :
    ((copyPlay s).tensor (RewardKernel.id Bool)).eu prior payoff =
      (if s true = true then 1 / 2 else 0) + (if s false = false then 1 / 2 else 0) := by
  simp only [eu, copyPlay, prior, payoff, tensor_K, tensor_W, RewardKernel.id, lift,
    Kernel.tensor_apply, Pi.add_apply, Fintype.sum_prod_type, Fintype.sum_bool,
    Kernel.idK_apply]
  rcases Bool.eq_false_or_eq_true (s true) with h1 | h1 <;>
    rcases Bool.eq_false_or_eq_true (s false) with h2 | h2 <;> simp [h1, h2]

/-- With the correlated prior, the first factor's best responses are exactly the policy that
```

```
copies its input. -/
theorem correlated_best_responses (s : Bool → Bool) :
    (∀ s', ((copyPlay s').tensor (RewardKernel.id Bool)).eu prior payoff ≤
        ((copyPlay s).tensor (RewardKernel.id Bool)).eu prior payoff) ↔ ∀ x, s x = x := by
  constructor
  · intro h
    have hid := h id
    rw [joint_eu, joint_eu] at hid
    intro x
    cases x <;> cases h1 : s true <;> cases h2 : s false <;> simp_all <;> norm_num at hid
  · intro hs s'
    rw [joint_eu, joint_eu, hs true, hs false]
    cases s' true <;> cases s' false <;> norm_num

/-- The first factor's expected utility in a context of its own. -/
theorem first_eu (s : Bool → Bool) (μ₁ c₁ : Bool → ℝ) :
    (copyPlay s).eu μ₁ c₁ = μ₁ true * c₁ (s true) + μ₁ false * c₁ (s false) := by
  simp only [eu, copyPlay, Fintype.sum_bool]
  cases s true <;> cases s false <;> simp

/-- **No context of a factor alone is exact for a correlated prior.** For every probability
context `(μ₁, c₁)` of the first factor, its equilibria as a decision game differ from its best
responses in the composite. -/
theorem correlated_no_factor_context :
    ¬ ∃ μ₁ c₁ : Bool → ℝ, (∀ x, 0 ≤ μ₁ x) ∧ ∀ s, (first.Eq μ₁ c₁ s ↔ ∀ x, s x = x) := by
  rintro ⟨μ₁, c₁, hμ, h⟩
  have eq_iff : ∀ s : Bool → Bool, first.Eq μ₁ c₁ s ↔
      ∀ s' : Bool → Bool, μ₁ true * c₁ (s' true) + μ₁ false * c₁ (s' false) ≤
        μ₁ true * c₁ (s true) + μ₁ false * c₁ (s false) := by
    intro s
    change (∀ s', (copyPlay s').eu μ₁ c₁ ≤ (copyPlay s).eu μ₁ c₁) ↔ _
    simp only [first_eu]
  have hid := (eq_iff _root_.id).mp ((h _root_.id).mpr fun _ ⇒ rfl)
  -- The copying policy beats both constant policies.
  have h_t := hid (fun _ ⇒ true)
  have h_f := hid (fun _ ⇒ false)
  simp only [_root_.id_eq] at h_t h_f
  have a0 := hμ true
  have b0 := hμ false
  -- If the constant `true` policy ties, it is an equilibrium that does not copy.
  by_cases hb : μ₁ false * (c₁ false - c₁ true) = 0
  · have heq : first.Eq μ₁ c₁ (fun _ ⇒ true) := by
      rw [eq_iff]
      intro s'
      have := hid s'
      simp only [_root_.id_eq] at this
      have hb' : μ₁ false * c₁ false = μ₁ false * c₁ true := by
        rw [mul_sub, sub_eq_zero] at hb
        exact hb
      linarith
    have := ((h _).mp heq) false
    simp at this
  · -- Otherwise the second input has positive weight and `c₁ false > c₁ true`, so the first
    -- input has weight zero and the constant `false` policy ties.
    have hpos : 0 < μ₁ false * (c₁ false - c₁ true) := lt_of_le_of_ne (by nlinarith) (Ne.symm hb)
    have hq : c₁ true < c₁ false := by
      by_contra hcon
      push Not at hcon
      nlinarith
    have ha : μ₁ true = 0 := by nlinarith
    have heq : first.Eq μ₁ c₁ (fun _ ⇒ false) := by
      rw [eq_iff]
      intro s'
      have := hid s'
      simp only [_root_.id_eq] at this
      rw [ha] at this ⊢
```

```
      linarith
    have := ((h _).mp heq) true
    simp at this

end CorrelatedTensor

end OpenInfluence
```

Perfect equilibria (ADR 0020, amended)

```
import CategoricalInfluenceDiagramsProofs.Lens
import Mathlib.Tactic.Linarith
```

An equilibrium of decision games can rest on choices at information states the equilibrium never reaches: a later component's policy there is a best response whatever it is, because those states have probability zero, and it can still deter an earlier component. This is the equilibrium refinement problem that subgame perfection answers.

Perfect decision games. A *perfect* decision game (`Game.perfect`) is in equilibrium in a context when its strategy maximises the value from *every* input state, against the context's continuation, whatever the prior. It refines the decision game: for a nonnegative prior a perfect strategy is a best response (`perfect_eq_decision`). Serial composites of perfect games ask the suffix to be perfect against the continuation and the prefix against the suffix's value function: backward induction, independent of the prior (`seq_perfect_eq_iff`).

The threat witness (`Threat`). An earlier decision chooses whether to act, worth `1` if it does not; a later one, observing it, chooses whether to cooperate, worth `4` if both act and `0` if only the first does. Not acting, with a later policy that would refuse to cooperate, is an equilibrium of the decision games worth `1` (`threat_equilibrium`): the refusal is never tested. The perfect equilibria are exactly the strategies in which the first acts and the second cooperates when it does, worth `4`, the optimum (`threat_perfect_iff`, `threat_optimum`).

```
set_option autoImplicit false

noncomputable section

namespace OpenInfluence

open FiniteKernelsProofs.Finite

namespace Game

open RewardKernel

variable {X Y Z : Type} [Fintype X] [Fintype Y] [Fintype Z]

/-- A perfect decision game: in equilibrium when its strategy maximises the value from every
input state against the continuation, whatever the prior. -/
def perfect {S : Type} (play : S → RewardKernel X Y) (h : ∀ s, Kernel.Normalised (play s).K) :
    Game X Y where
  Strat := S
  play := play
  normalised := h
  Eq _ c s := ∀ x s', (play s').continuation c x ≤ (play s).continuation c x

/-- **Perfection refines equilibrium**: with a nonnegative prior, a perfect strategy is a best
response. -/
theorem perfect_eq_decision {S : Type} (play : S → RewardKernel X Y)
    (h : ∀ s, Kernel.Normalised (play s).K) (μ : X → ℝ) (hμ : ∀ x, 0 ≤ μ x) (c : Y → ℝ)
    (s : S) (hs : (perfect play h).Eq μ c s) : (decision play h).Eq μ c s := by
  intro s'
  rw [eu_eq_continuation, eu_eq_continuation]
  exact Finset.sum_le_sum fun x _ ⇒ mul_le_mul_of_nonneg_left (hs x s') (hμ x)

/-- **Backward induction**: a strategy of two perfect games in series is an equilibrium exactly
when the suffix is perfect against the continuation and the prefix against the suffix's value
```

```
function, whatever the prior. -/
theorem seq_perfect_eq_iff {S T : Type} (pf : S → RewardKernel X Y)
    (hf : ∀ s, Kernel.Normalised (pf s).K) (pg : T → RewardKernel Y Z)
    (hg : ∀ t, Kernel.Normalised (pg t).K) (μ : X → ℝ) (c : Z → ℝ) (s : S) (t : T) :
    ((perfect pf hf).seq (perfect pg hg)).Eq μ c (s, t) ↔
      (∀ x s', (pf s').continuation ((pg t).continuation c) x ≤
        (pf s).continuation ((pg t).continuation c) x) ∧
      ∀ y t', (pg t').continuation c y ≤ (pg t).continuation c y :=
  Iff.rfl

end Game
```

The threat witness

```
namespace Threat

open RewardKernel

/-- The first decision: act (`true`) or not, a point mass on the choice. -/
def first (a : Bool) : RewardKernel Unit Bool :=
  ⟨fun _ y ⇒ if y = a then 1 else 0, fun _ _ ⇒ 0⟩

/-- The payoff: `1` if the first does not act, `4` if both act, `0` if only the first does. -/
def payoff (a b : Bool) : ℝ := if a then (if b then 4 else 0) else 1

/-- The second decision, observing the first: a policy `b` from the first's choice to
cooperating, paying the payoff. -/
def second (b : Bool → Bool) : RewardKernel Bool Unit :=
  ⟨fun _ _ ⇒ 1, fun a _ ⇒ payoff a (b a)⟩

theorem first_normalised (a : Bool) : Kernel.Normalised (first a).K := by
  intro x
  simp [first]

theorem second_normalised (b : Bool → Bool) : Kernel.Normalised (second b).K := by
  intro x
  simp [second]

/-- The composite's value. -/
theorem composite_eu (a : Bool) (b : Bool → Bool) :
    ((first a).comp (second b)).eu (fun _ ⇒ 1) (fun _ ⇒ 0) = payoff a (b a) := by
  cases a <;> simp [eu, first, second, comp, Kernel.comp]

/-- The first decision's value against the second's value function. -/
theorem first_value (a : Bool) (b : Bool → Bool) :
    (first a).eu (fun _ ⇒ 1) ((second b).continuation (fun _ ⇒ 0)) = payoff a (b a) := by
  simp only [eu, continuation, first, second, Fintype.sum_bool, Fintype.univ_unit,
    Finset.sum_singleton]
  cases a <;> simp

/-- The second decision's value against the first's boundary law. -/
theorem second_value (a : Bool) (b : Bool → Bool) :
    (second b).eu ((first a).pushforward (fun _ ⇒ 1)) (fun _ ⇒ 0) = payoff a (b a) := by
  simp only [eu, pushforward, first, second, Fintype.sum_bool, Fintype.univ_unit,
    Finset.sum_singleton]
  cases a <;> simp

/-- The threat: refuse to cooperate if the first acts. -/
def threat : Bool → Bool := fun _ ⇒ false

/-- **An equilibrium sustained by an untested threat.** Not acting, against a second decision
that would refuse to cooperate, is an equilibrium of the decision games, worth `1`. -/
theorem threat_equilibrium :
    ((Game.decision first first_normalised).seq
        (Game.decision second second_normalised)).Eq (fun _ ⇒ 1) (fun _ ⇒ 0)
```

```
        (false, threat) ∧
      ((first false).comp (second threat)).eu (fun _ ⇒ 1) (fun _ ⇒ 0) = 1 := by
  refine ⟨⟨fun a' ⇒ ?_, fun b' ⇒ ?_⟩, by rw [composite_eu]; simp [payoff]⟩
  · show (first a').eu (fun _ ⇒ 1) ((second threat).continuation (fun _ ⇒ 0)) ≤
      (first false).eu (fun _ ⇒ 1) ((second threat).continuation (fun _ ⇒ 0))
    rw [first_value, first_value]
    cases a' <;> simp [payoff, threat]
  · show (second b').eu ((first false).pushforward (fun _ ⇒ 1)) (fun _ ⇒ 0) ≤
      (second threat).eu ((first false).pushforward (fun _ ⇒ 1)) (fun _ ⇒ 0)
    rw [second_value, second_value]
    simp [payoff]

/-- **Perfect equilibria**: the first acts and the second cooperates when it does. -/
theorem threat_perfect_iff (a : Bool) (b : Bool → Bool) :
    ((Game.perfect first first_normalised).seq
        (Game.perfect second second_normalised)).Eq (fun _ ⇒ 1) (fun _ ⇒ 0) (a, b) ↔
      a = true ∧ b true = true := by
  rw [Game.seq_perfect_eq_iff]
  have hsecond : ∀ (b' : Bool → Bool) (y : Bool),
      (second b').continuation (fun _ ⇒ 0) y = payoff y (b' y) := by
    intro b' y
    simp [continuation, second]
  have hfirst : ∀ (x : Unit) (a' : Bool) (b' : Bool → Bool),
      (first a').continuation ((second b').continuation (fun _ ⇒ 0)) x = payoff a' (b' a') := by
    intro x a' b'
    simp only [continuation, first, second, Fintype.sum_bool]
    cases a' <;> simp
  simp only [hsecond, hfirst]
  constructor
  · rintro ⟨hf, hs⟩
    have hb : b true = true := by
      have h4 := hs true (fun _ ⇒ true)
      revert h4
      cases b true <;> norm_num [payoff]
    refine ⟨?_, hb⟩
    have h1 := hf () true
    rw [hb] at h1
    revert h1
    cases a <;> norm_num [payoff]
  · rintro ⟨rfl, hb⟩
    refine ⟨fun _ a' ⇒ ?_, fun y b' ⇒ ?_⟩
    · rw [hb]
      cases a' <;> norm_num [payoff, hb]
    · cases y
      · norm_num [payoff]
      · rw [hb]
        cases b' true <;> norm_num [payoff]

/-- The perfect equilibria attain the optimum, `4`. -/
theorem threat_optimum :
    (∀ a b, ((first a).comp (second b)).eu (fun _ ⇒ 1) (fun _ ⇒ 0) ≤ 4) ∧
      ((first true).comp (second fun _ ⇒ true)).eu (fun _ ⇒ 1) (fun _ ⇒ 0) = 4 := by
  refine ⟨fun a b ⇒ ?_, by rw [composite_eu]; simp [payoff]⟩
  rw [composite_eu]
  cases a <;> cases b _ <;> norm_num [payoff]

end Threat

end OpenInfluence
```

Evidence across a boundary (ADR 0021)

```
import CategoricalInfluenceDiagramsProofs.Lens
import CategoricalInfluenceDiagramsProofs.Interpretation
```

Evidence makes the probability component of a reward kernel substochastic: conditioning an open influence diagram on observed states keeps, for each input `x` and output `y`, the probability `K x y` of reaching `y` *and* seeing the evidence, and the reward `W x y` earned on those paths. The category of reward kernels does not need normalisation, so serial composition still holds; what fails is the normalised lens of ADR 0020, whose backward map loses the reward of the prefix when the suffix loses mass (`eu_comp_needs_normalised`).

The repair is to carry the mass. A context on the boundary becomes a pair in the expectation semiring on each side:

- an **effect** `(m, c)` on the outputs, the likelihood `m y` of the evidence still to come and the evidence-weighted value `c y` of what follows (`Effect`), carried backward by `pull`;
- a **state** `(ν, ρ)` on the inputs, the unnormalised law `ν x` of reaching `x` with the evidence seen so far and the reward-weighted law `ρ x` (`State`), carried forward by `push`.

Results. - Both actions are functorial with no hypothesis (`pull_comp`, `push_comp`, `pull_id`, `push_id`), and the pairing of a state with an effect, the evidence mass `Σ_x ν x m x` and the evidence-weighted value `Σ_x (ρ x m x + ν x c x)`, is the same at every boundary (`mass_push`, `value_push`). So the composite's evidence mass is the pairing of the prefix's unnormalised boundary law with the suffix's likelihood (`mass_comp`), and its conditional value, their ratio, is computed at any cut (`conditionalValue_comp`). - The plain contexts are the case of mass one: a prior is a state with zero reward, a continuation an effect with mass one, and against them the pairing is the expected utility of ADR 0018 (`value_ofPrior`). With a normalised suffix, pulling a continuation back keeps the mass one and is the value function of ADR 0020 (`pull_ofValue`); without normalisation the first half of the contextual identity still holds once the mass is carried (`eu_comp_pull`). - Hard evidence on a boundary variable is composition with a diagonal indicator (`observe`), which is idempotent and commutes with other evidence (`observe_idem`, `observe_comm`): stating the same evidence on both sides of a boundary changes nothing. - Normalising each component on its own evidence before composing is not sound (`local_normalisation_obstruction`): with a uniform prior on a binary state worth `1` when true, and evidence nine times likelier when it is true, the conditional value is `9/10`, while the components normalised one by one give `1/2`. The evidence mass must cross the boundary. - For the syntax, a likelihood on the values of the generators, keyed by label as strategies are, weights the sum-product interpretation (`interpretE`). Weighting is functorial under serial composition (`interpretE_comp`) and tensor (`interpretE_tensor`), and the constant likelihood one gives back `interpret` (`interpretE_one`). Hard evidence is the indicator likelihood.

```
set_option autoImplicit false

noncomputable section

namespace OpenInfluence

open FiniteKernelsProofs.Finite

namespace RewardKernel

variable {X Y Z : Type}

/-- A context on the output side carrying evidence: the likelihood `m` of the evidence still
to come, and the evidence-weighted value `c` of what follows. -/
@[ext] structure Effect (Y : Type) where
  /-- `m y`: the probability of the evidence still to come, from the boundary state `y`. -/
  m : Y → ℝ
  /-- `c y`: the expected value still to come times the indicator of that evidence. -/
  c : Y → ℝ

/-- A context on the input side carrying evidence: the unnormalised law `ν` and the
reward-weighted law `ρ`. -/
@[ext] structure State (X : Type) where
  /-- `ν x`: the probability of reaching `x` with the evidence seen so far. -/
  ν : X → ℝ
  /-- `ρ x`: the reward earned so far, weighted as `ν x` is. -/
  ρ : X → ℝ

/-- A continuation with no evidence still to come. -/
def Effect.ofValue (c : Y → ℝ) : Effect Y := ⟨fun _ ⇒ 1, c⟩
```

```
/-- A prior with no reward earned yet. -/
def State.ofPrior (μ : X → ℝ) : State X := ⟨μ, fun _ ⇒ 0⟩

/-- The backward action of a reward kernel on effects. -/
def pull [Fintype Z] (g : RewardKernel Y Z) (e : Effect Z) : Effect Y :=
  ⟨fun y ⇒ ∑ z, g.K y z * e.m z, fun y ⇒ ∑ z, (g.W y z * e.m z + g.K y z * e.c z)⟩

/-- The forward action of a reward kernel on states. -/
def push [Fintype X] (f : RewardKernel X Y) (s : State X) : State Y :=
  ⟨fun y ⇒ ∑ x, s.ν x * f.K x y, fun y ⇒ ∑ x, (s.ρ x * f.K x y + s.ν x * f.W x y)⟩

/-- The evidence mass of a state against an effect. -/
def mass [Fintype X] (s : State X) (e : Effect X) : ℝ := ∑ x, s.ν x * e.m x

/-- The evidence-weighted value of a state against an effect. -/
def value [Fintype X] (s : State X) (e : Effect X) : ℝ := ∑ x, (s.ρ x * e.m x + s.ν x * e.c x)

/-- The conditional value: the evidence-weighted value divided by the evidence mass. -/
def conditionalValue [Fintype X] (s : State X) (e : Effect X) : ℝ := value s e / mass s e

variable [Fintype X] [Fintype Y] [Fintype Z]

omit [Fintype X] in
/-- The backward action is functorial, with no normalisation hypothesis. -/
theorem pull_comp (f : RewardKernel X Y) (g : RewardKernel Y Z) (e : Effect Z) :
    (f.comp g).pull e = f.pull (g.pull e) := by
  ext x
  · simp only [pull, comp_K, Kernel.comp_apply, Finset.sum_mul, Finset.mul_sum]
    rw [Finset.sum_comm]
    exact Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_congr rfl fun _ _ ⇒ by ring
  · simp only [pull, comp_K, comp_W, Kernel.comp_apply, Pi.add_apply, Finset.sum_mul,
      Finset.mul_sum, add_mul, mul_add, Finset.sum_add_distrib]
    rw [Finset.sum_comm (f := fun z y ⇒ f.W x y * g.K y z * e.m z),
      Finset.sum_comm (f := fun z y ⇒ f.K x y * g.W y z * e.m z),
      Finset.sum_comm (f := fun z y ⇒ f.K x y * g.K y z * e.c z)]
    simp only [mul_assoc, add_assoc]

omit [Fintype Z] in
/-- The forward action is functorial, with no normalisation hypothesis. -/
theorem push_comp (f : RewardKernel X Y) (g : RewardKernel Y Z) (s : State X) :
    (f.comp g).push s = g.push (f.push s) := by
  ext z
  · simp only [push, comp_K, Kernel.comp_apply, Finset.sum_mul, Finset.mul_sum]
    rw [Finset.sum_comm]
    exact Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_congr rfl fun _ _ ⇒ by ring
  · simp only [push, comp_K, comp_W, Kernel.comp_apply, Pi.add_apply, Finset.sum_mul,
      Finset.mul_sum, add_mul, mul_add, Finset.sum_add_distrib]
    rw [Finset.sum_comm (f := fun x y ⇒ s.ρ x * (f.K x y * g.K y z)),
      Finset.sum_comm (f := fun x y ⇒ s.ν x * (f.W x y * g.K y z)),
      Finset.sum_comm (f := fun x y ⇒ s.ν x * (f.K x y * g.W y z))]
    simp only [mul_assoc, add_assoc]

omit [Fintype X] [Fintype Y] [Fintype Z] in
theorem pull_id [Fintype X] [DecidableEq X] (e : Effect X) : (id X).pull e = e := by
  ext x <;> simp [pull, id, lift, Kernel.idK_apply, ite_mul]

omit [Fintype X] [Fintype Y] [Fintype Z] in
theorem push_id [Fintype X] [DecidableEq X] (s : State X) : (id X).push s = s := by
  ext x <;> simp [push, id, lift, Kernel.idK_apply, mul_ite]

omit [Fintype Z] in
/-- The evidence mass is the same on either side of a reward kernel. -/
theorem mass_push (f : RewardKernel X Y) (s : State X) (e : Effect Y) :
    mass (f.push s) e = mass s (f.pull e) := by
  simp only [mass, push, pull, Finset.sum_mul, Finset.mul_sum]
  rw [Finset.sum_comm]
```

```
  exact Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_congr rfl fun _ _ ⇒ by ring

omit [Fintype Z] in
/-- The evidence-weighted value is the same on either side of a reward kernel. -/
theorem value_push (f : RewardKernel X Y) (s : State X) (e : Effect Y) :
    value (f.push s) e = value s (f.pull e) := by
  simp only [value, push, pull, Finset.sum_mul, Finset.mul_sum, add_mul, mul_add,
    Finset.sum_add_distrib]
  rw [Finset.sum_comm (f := fun y x ⇒ s.ρ x * f.K x y * e.m y),
    Finset.sum_comm (f := fun y x ⇒ s.ν x * f.W x y * e.m y),
    Finset.sum_comm (f := fun y x ⇒ s.ν x * f.K x y * e.c y)]
  simp only [mul_assoc, add_assoc]

/-- The evidence mass of a composite is the pairing, at the boundary, of the prefix's
unnormalised boundary law with the suffix's likelihood. -/
theorem mass_comp (f : RewardKernel X Y) (g : RewardKernel Y Z) (s : State X) (e : Effect Z) :
    mass s ((f.comp g).pull e) = ∑ y, (f.push s).ν y * (g.pull e).m y := by
  rw [pull_comp, ← mass_push]
  rfl

/-- The conditional value of a composite can be computed at the boundary between its parts:
the prefix's state pushed forward, against the suffix's effect pulled back. -/
theorem conditionalValue_comp (f : RewardKernel X Y) (g : RewardKernel Y Z) (s : State X)
    (e : Effect Z) :
    conditionalValue s ((f.comp g).pull e) = conditionalValue (f.push s) (g.pull e) := by
  simp only [conditionalValue, pull_comp, mass_push, value_push]

omit [Fintype Z] in
/-- Against a prior and a continuation, the pairing is the expected utility of ADR 0018. -/
theorem value_ofPrior (f : RewardKernel X Y) (μ : X → ℝ) (c : Y → ℝ) :
    value (State.ofPrior μ) (f.pull (Effect.ofValue c)) = f.eu μ c := by
  simp [value, State.ofPrior, Effect.ofValue, pull, eu, mul_one]

omit [Fintype X] [Fintype Y] [Fintype Z] in
/-- With a normalised suffix, pulling a continuation back keeps the mass one and gives the
value function of ADR 0020. -/
theorem pull_ofValue [Fintype Z] (g : RewardKernel Y Z) (hg : Kernel.Normalised g.K)
    (c : Z → ℝ) : g.pull (Effect.ofValue c) = Effect.ofValue (g.continuation c) := by
  ext y
  · simp [pull, Effect.ofValue, hg y]
  · simp [pull, Effect.ofValue, continuation]

/-- The first half of the contextual identity without normalisation: carrying the mass, the
composite's expected utility is the prefix's value against the suffix's effect. -/
theorem eu_comp_pull (f : RewardKernel X Y) (g : RewardKernel Y Z) (μ : X → ℝ)
    (c : Z → ℝ) :
    (f.comp g).eu μ c = value (State.ofPrior μ) (f.pull (g.pull (Effect.ofValue c))) := by
  rw [← pull_comp, value_ofPrior]
```

Hard evidence on a boundary variable

```
omit [Fintype X] [Fintype Y] [Fintype Z] in
/-- Observing that the boundary state satisfies `P`: the diagonal indicator, with zero
reward. -/
def observe (P : X → Prop) [DecidablePred P] [DecidableEq X] : RewardKernel X X :=
  lift fun x x' ⇒ if x = x' ∧ P x then 1 else 0

omit [Fintype Y] [Fintype Z] in
theorem observe_comp_observe (P Q : X → Prop) [DecidablePred P] [DecidablePred Q]
    [DecidableEq X] :
    (observe P).comp (observe Q) = observe fun x ⇒ P x ∧ Q x := by
  ext x x'
  · simp only [observe, lift, comp_K, Kernel.comp_apply]
    rw [Finset.sum_eq_single x']
    · by_cases h : x = x' <;> by_cases hp : P x <;> by_cases hq : Q x' <;> simp_all
```

```
    · intro b _ hb
      simp [hb]
    · simp
  · simp [observe, lift, comp_W, Kernel.comp]

omit [Fintype Y] [Fintype Z] in
/-- Observing twice is observing once: the same evidence stated on both sides of a boundary
changes nothing. -/
theorem observe_idem (P : X → Prop) [DecidablePred P] [DecidableEq X] :
    (observe P).comp (observe P) = observe P := by
  rw [observe_comp_observe]
  simp

omit [Fintype Y] [Fintype Z] in
/-- Evidence on a boundary commutes. -/
theorem observe_comm (P Q : X → Prop) [DecidablePred P] [DecidablePred Q] [DecidableEq X] :
    (observe P).comp (observe Q) = (observe Q).comp (observe P) := by
  rw [observe_comp_observe, observe_comp_observe]
  congr 1
  funext x
  exact propext and_comm

end RewardKernel
```

Normalising each component is not sound

```
namespace LocalNormalisation

open RewardKernel

/-- Normalise each row of a reward kernel by its probability mass: the component conditioned
on its own evidence, in isolation. -/
def normalise {X Y : Type} [Fintype Y] (f : RewardKernel X Y) : RewardKernel X Y :=
  ⟨fun x y ⇒ f.K x y / ∑ y', f.K x y', fun x y ⇒ f.W x y / ∑ y', f.K x y'⟩

/-- The prefix: a uniform binary state, worth `1` when it is true. -/
def prior : RewardKernel Unit Bool :=
  ⟨fun _ _ ⇒ 1 / 2, fun _ b ⇒ if b then 1 / 2 else 0⟩

/-- The suffix: evidence nine times likelier when the state is true. -/
def evidence : RewardKernel Bool Unit :=
  ⟨fun b _ ⇒ if b then 9 / 10 else 1 / 10, fun _ _ ⇒ 0⟩

/-- The whole context of the closed composite: the unit state and the unit effect. -/
def start : RewardKernel.State Unit := RewardKernel.State.ofPrior fun _ ⇒ 1

def finish : RewardKernel.Effect Unit := RewardKernel.Effect.ofValue fun _ ⇒ 0

theorem composite_conditionalValue :
    conditionalValue start ((prior.comp evidence).pull finish) = 9 / 10 := by
  simp only [conditionalValue, value, mass, start, finish, RewardKernel.State.ofPrior,
    RewardKernel.Effect.ofValue,
    pull, prior, evidence, comp_K, comp_W, Kernel.comp_apply, Pi.add_apply,
    Fintype.sum_bool, Fintype.univ_unit, Finset.sum_singleton]
  norm_num

theorem normalised_conditionalValue :
    conditionalValue start (((normalise prior).comp (normalise evidence)).pull finish) =
      1 / 2 := by
  simp only [conditionalValue, value, mass, start, finish, RewardKernel.State.ofPrior,
    RewardKernel.Effect.ofValue,
    pull, normalise, prior, evidence, comp_K, comp_W, Kernel.comp_apply, Pi.add_apply,
    Fintype.sum_bool, Fintype.univ_unit, Finset.sum_singleton]
  norm_num
```

```
/-- **Local normalisation is not sound.** Conditioning each component on its own evidence
and composing gives `1/2`; the composite conditioned on all of it gives `9/10`. -/
theorem local_normalisation_obstruction :
    conditionalValue start ((prior.comp evidence).pull finish) = 9 / 10 ∧
      conditionalValue start (((normalise prior).comp (normalise evidence)).pull finish) =
        1 / 2 :=
  ⟨composite_conditionalValue, normalised_conditionalValue⟩

end LocalNormalisation
```

Evidence in the syntax

```
namespace IDInterpretation

open OpenNet

variable {S : IDSignature} (M : IDInterpretation S) (σ : M.Strategy)
variable {X Y Z X' Y' : S.Iface}

/-- A likelihood on the values of generators, keyed by label as strategies are. Hard
evidence that a generator takes the value `s` is the indicator of `s`. -/
abbrev Likelihood : Type := S.Label → Sigma M.states → ℝ

variable (ev : M.Likelihood)

/-- The likelihood of an internal assignment: the product over generators. -/
def lik (f : IDNet X Y) (z : (M.interp σ).Internal f.net) : ℝ :=
  ∏ a, ev (f.net.label a) (z a)

/-- The evidence-weighted probability table. -/
def kernelE (f : IDNet X Y) : Kernel ((M.interp σ).Boundary X) ((M.interp σ).Boundary Y) :=
  fun x y ⇒ ∑ z, if y = (M.interp σ).read f.net x z then
    (M.interp σ).joint f.net x z * M.lik σ ev f z else 0

/-- The evidence-weighted reward table. -/
def rewardTableE (f : IDNet X Y) :
    Kernel ((M.interp σ).Boundary X) ((M.interp σ).Boundary Y) :=
  fun x y ⇒ ∑ z, if y = (M.interp σ).read f.net x z then
    (M.interp σ).joint f.net x z * M.lik σ ev f z * M.reward σ f x z else 0

/-- The fixed-strategy semantics of an open influence diagram weighted by the likelihood
`ev`: a substochastic reward kernel for hard evidence. -/
def interpretE (f : IDNet X Y) :
    RewardKernel ((M.interp σ).Boundary X) ((M.interp σ).Boundary Y) :=
  ⟨M.kernelE σ ev f, M.rewardTableE σ ev f⟩

theorem lik_comp (f : IDNet X Y) (g : IDNet Y Z) (a : (M.interp σ).Internal f.net)
    (b : (M.interp σ).Internal g.net) :
    M.lik σ ev (f.comp g) ((M.interp σ).combine f.net g.net a b) =
      M.lik σ ev f a * M.lik σ ev g b := by
  unfold lik
  erw [Fintype.prod_sum_type]
  rfl

theorem kernelE_comp (f : IDNet X Y) (g : IDNet Y Z) :
    M.kernelE σ ev (f.comp g) = Kernel.comp (M.kernelE σ ev f) (M.kernelE σ ev g) := by
  funext x w
  simp only [Kernel.comp]
  unfold kernelE
  rw [M.sum_fibre_mul σ x w (fun a ⇒ (M.interp σ).joint f.net x a * M.lik σ ev f a)
      (fun y b ⇒ (M.interp σ).joint g.net y b * M.lik σ ev g b)]
  change (∑ z : (M.interp σ).Internal (f.net.comp g.net),
      if w = (M.interp σ).read (f.net.comp g.net) x z then
        (M.interp σ).joint (f.net.comp g.net) x z * M.lik σ ev (f.comp g) z else 0) = _
  erw [← ((M.interp σ).sumLabelsEquiv f.net.label g.net.label).symm.sum_comp]
```

```
  rw [Fintype.sum_prod_type]
  apply Finset.sum_congr rfl
  intro a _
  apply Finset.sum_congr rfl
  intro b _
  have hr := (M.interp σ).read_comp f.net g.net x a b
  have hj := (M.interp σ).joint_comp f.net g.net x a b
  have hl := M.lik_comp σ ev f g a b
  simp only [Interpretation.combine] at hr hj hl
  rw [hr, hj, hl]
  split_ifs <;> ring

theorem rewardTableE_comp (f : IDNet X Y) (g : IDNet Y Z) :
    M.rewardTableE σ ev (f.comp g) =
      Kernel.comp (M.rewardTableE σ ev f) (M.kernelE σ ev g) +
        Kernel.comp (M.kernelE σ ev f) (M.rewardTableE σ ev g) := by
  funext x w
  simp only [Pi.add_apply, Kernel.comp]
  unfold rewardTableE kernelE
  rw [M.sum_fibre_mul σ x w
      (fun a ⇒ (M.interp σ).joint f.net x a * M.lik σ ev f a * M.reward σ f x a)
      (fun y b ⇒ (M.interp σ).joint g.net y b * M.lik σ ev g b),
    M.sum_fibre_mul σ x w (fun a ⇒ (M.interp σ).joint f.net x a * M.lik σ ev f a)
      (fun y b ⇒ (M.interp σ).joint g.net y b * M.lik σ ev g b * M.reward σ g y b)]
  rw [← Finset.sum_add_distrib]
  change (∑ z : (M.interp σ).Internal (f.net.comp g.net),
      if w = (M.interp σ).read (f.net.comp g.net) x z then
        (M.interp σ).joint (f.net.comp g.net) x z * M.lik σ ev (f.comp g) z *
          M.reward σ (f.comp g) x z else 0) = _
  erw [← ((M.interp σ).sumLabelsEquiv f.net.label g.net.label).symm.sum_comp]
  rw [Fintype.sum_prod_type]
  apply Finset.sum_congr rfl
  intro a _
  rw [← Finset.sum_add_distrib]
  apply Finset.sum_congr rfl
  intro b _
  have hr := (M.interp σ).read_comp f.net g.net x a b
  have hj := (M.interp σ).joint_comp f.net g.net x a b
  have hl := M.lik_comp σ ev f g a b
  have hw := M.reward_comp σ f g x a b
  simp only [Interpretation.combine] at hr hj hl hw
  rw [hr, hj, hl, hw]
  split_ifs <;> ring

/-- **Evidence composes**: the evidence-weighted interpretation of a serial composite is the
composite, in the category of reward kernels, of the evidence-weighted interpretations. -/
theorem interpretE_comp (f : IDNet X Y) (g : IDNet Y Z) :
    M.interpretE σ ev (f.comp g) = (M.interpretE σ ev f).comp (M.interpretE σ ev g) := by
  ext1
  · exact M.kernelE_comp σ ev f g
  · exact M.rewardTableE_comp σ ev f g

theorem lik_tensor (f : IDNet X Y) (g : IDNet X' Y') (a : (M.interp σ).Internal f.net)
    (b : (M.interp σ).Internal g.net) :
    M.lik σ ev (f.tensor g) ((M.interp σ).combine f.net g.net a b) =
      M.lik σ ev f a * M.lik σ ev g b := by
  unfold lik
  erw [Fintype.prod_sum_type]
  rfl

theorem kernelE_tensor (f : IDNet X Y) (g : IDNet X' Y') (x : (M.interp σ).Boundary X)
    (x' : (M.interp σ).Boundary X') (y : (M.interp σ).Boundary Y)
    (y' : (M.interp σ).Boundary Y') :
    M.kernelE σ ev (f.tensor g) (Interpretation.Assignment.join _ x x')
        (Interpretation.Assignment.join _ y y') =
      M.kernelE σ ev f x y * M.kernelE σ ev g x' y' := by
```

```
  unfold kernelE
  rw [Finset.sum_mul_sum]
  change (∑ z : (M.interp σ).Internal (f.net.tensor g.net),
      if Interpretation.Assignment.join _ y y' =
          (M.interp σ).read (f.net.tensor g.net) (Interpretation.Assignment.join _ x x') z then
        (M.interp σ).joint (f.net.tensor g.net) (Interpretation.Assignment.join _ x x') z *
          M.lik σ ev (f.tensor g) z else 0) = _
  erw [← ((M.interp σ).sumLabelsEquiv f.net.label g.net.label).symm.sum_comp]
  rw [Fintype.sum_prod_type]
  apply Finset.sum_congr rfl
  intro a _
  apply Finset.sum_congr rfl
  intro b _
  have hr := (M.interp σ).read_tensor f.net g.net x x' a b
  have hj := (M.interp σ).joint_tensor f.net g.net x x' a b
  have hl := M.lik_tensor σ ev f g a b
  simp only [Interpretation.combine] at hr hj hl
  rw [hr, hj, hl]
  by_cases h₁ : y = (M.interp σ).read f.net x a <;>
    by_cases h₂ : y' = (M.interp σ).read g.net x' b <;>
      simp [Interpretation.Assignment.join_injective, h₁, h₂]
  ring

theorem rewardTableE_tensor (f : IDNet X Y) (g : IDNet X' Y') (x : (M.interp σ).Boundary X)
    (x' : (M.interp σ).Boundary X') (y : (M.interp σ).Boundary Y)
    (y' : (M.interp σ).Boundary Y') :
    M.rewardTableE σ ev (f.tensor g) (Interpretation.Assignment.join _ x x')
        (Interpretation.Assignment.join _ y y') =
      M.rewardTableE σ ev f x y * M.kernelE σ ev g x' y' +
        M.kernelE σ ev f x y * M.rewardTableE σ ev g x' y' := by
  unfold rewardTableE kernelE
  rw [Finset.sum_mul_sum, Finset.sum_mul_sum, ← Finset.sum_add_distrib]
  change (∑ z : (M.interp σ).Internal (f.net.tensor g.net),
      if Interpretation.Assignment.join _ y y' =
          (M.interp σ).read (f.net.tensor g.net) (Interpretation.Assignment.join _ x x') z then
        (M.interp σ).joint (f.net.tensor g.net) (Interpretation.Assignment.join _ x x') z *
          M.lik σ ev (f.tensor g) z *
            M.reward σ (f.tensor g) (Interpretation.Assignment.join _ x x') z else 0) = _
  erw [← ((M.interp σ).sumLabelsEquiv f.net.label g.net.label).symm.sum_comp]
  rw [Fintype.sum_prod_type]
  apply Finset.sum_congr rfl
  intro a _
  rw [← Finset.sum_add_distrib]
  apply Finset.sum_congr rfl
  intro b _
  have hr := (M.interp σ).read_tensor f.net g.net x x' a b
  have hj := (M.interp σ).joint_tensor f.net g.net x x' a b
  have hl := M.lik_tensor σ ev f g a b
  have hw := M.reward_tensor σ f g x x' a b
  simp only [Interpretation.combine] at hr hj hl hw
  rw [hr, hj, hl, hw]
  by_cases h₁ : y = (M.interp σ).read f.net x a <;>
    by_cases h₂ : y' = (M.interp σ).read g.net x' b <;>
      simp [Interpretation.Assignment.join_injective, h₁, h₂]
  ring

/-- **Evidence composes in parallel**: read through the canonical bijections between
assignments of a disjoint-union interface and pairs of assignments, the evidence-weighted
interpretation of a tensor is the tensor of the evidence-weighted interpretations. -/
theorem interpretE_tensor (f : IDNet X Y) (g : IDNet X' Y') :
    (M.interpretE σ ev (f.tensor g)).reindex (M.join σ X X') (M.join σ Y Y') =
      (M.interpretE σ ev f).tensor (M.interpretE σ ev g) := by
  ext1
  · funext p q
    exact M.kernelE_tensor σ ev f g p.1 p.2 q.1 q.2
  · funext p q
```

```
    exact M.rewardTableE_tensor σ ev f g p.1 p.2 q.1 q.2

/-- With the likelihood one everywhere, there is no evidence. -/
theorem interpretE_one (f : IDNet X Y) :
    M.interpretE σ (fun _ _ ⇒ 1) f = M.interpret σ f := by
  ext x y
  · simp [interpretE, kernelE, interpret, lik, kernel_eq_sum]
  · simp [interpretE, rewardTableE, interpret, rewardTable, lik]

end IDInterpretation

end OpenInfluence
```

Behavioural strategies and the value of randomising (ADR 0022)

```
import CategoricalInfluenceDiagramsProofs.Evidence
```

ADR 0018 kept randomised policies out of the first release "until a theorem covers them". This module supplies the theorems.

Behavioural strategies. A behavioural strategy (`Behaviour`) gives every decision label a distribution over its result for each list of values on its information ports, normalised on correctly typed rows. Decisions become generators with those tables (`interpB`), and the semantics of an open influence diagram is again a reward kernel (`interpretB`), stochastic, functorial under serial composition (`interpretB_comp`), identities (`interpretB_identity`) and tensor (`interpretB_tensor`), and the contextual identity holds (`contextual_identityB`). A deterministic strategy is the behavioural strategy of point masses, with the same interpretation (`interpB_pointMass`, `interpretB_pointMass`).

Mixtures. A mixture of reward kernels (`mix`) composes affinely on either side (`mix_comp`, `comp_mix`), its expected utility is the mixture of the expected utilities (`eu_mix`), so it never exceeds the best of them (`eu_mix_le`), and its conditional value never exceeds the best conditional value among the components of positive mass (`conditionalValue_mix_le`, the mediant inequality). Randomising over whole strategies of a component therefore never helps, with or without evidence.

Shared policies. Decisions that share a label share one policy. Then randomising can help: the absent-minded driver (`AbsentMinded`), two decisions with one policy, continue with probability `p` at both, pays `4p - 3p²`, which is `4/3` at `p = 2/3` and at most `1` for either deterministic policy (`randomising_helps`). With a policy each, the same payoffs `p (4 (1 - q) + q)` are maximised by a deterministic pair (`independent_deterministic_optimal`).

```
set_option autoImplicit false

noncomputable section

namespace OpenInfluence

open CategoryTheory FiniteKernelsProofs.Finite OpenNet

namespace IDInterpretation

variable {S : IDSignature} (M : IDInterpretation S)

/-- A behavioural strategy: for every decision label a table of probabilities of its result
given the values on its information ports, nonnegative and normalised on correctly typed
rows. -/
structure Behaviour where
  /-- `policy l h p s`: the probability of the result `s` given the information values `p`. -/
  policy : (l : S.Label) → S.isDecision l = true → List (Sigma M.states) → Sigma M.states → ℝ
  /-- Probabilities are nonnegative. -/
  nonneg : ∀ l h p s, 0 ≤ policy l h p s
  /-- Probabilities sum to one on every correctly typed row. -/
  normalized : ∀ l h p, p.map Sigma.fst = S.inputs l →
    @Finset.sum (M.states (S.result l)) ℝ _ (@Finset.univ _ (M.finite _))
      (fun s ⇒ policy l h p ⟨S.result l, s⟩) = 1
```

```
/-- The `OpenNet` interpretation for a behavioural strategy: decisions draw from their
policies, chance mechanisms keep their tables. -/
def interpB (β : M.Behaviour) : OpenNet.Interpretation S.toSignature where
  states := M.states
  finite := M.finite
  weight l p s :=
    if h : S.isDecision l = true then β.policy l h p s else M.chance l p s
  nonneg l p s := by
    by_cases h : S.isDecision l = true
    · simp only [h, dite_true]
      exact β.nonneg l h p s
    · simp only [h]
      exact M.chance_nonneg l p s (by simpa using h)
  normalized l p hp := by
    by_cases h : S.isDecision l = true
    · simp only [h, dite_true]
      exact β.normalized l h p hp
    · simp only [h]
      exact M.chance_normalized l p (by simpa using h) hp

open Classical in
/-- A deterministic strategy as a behavioural one: the point masses of its policies. -/
def pointMass (σ : M.Strategy) : M.Behaviour where
  policy l h p s := if s = ⟨S.result l, σ l h p⟩ then 1 else 0
  nonneg l h p s := by split_ifs <;> norm_num
  normalized l h p _ := by
    simp only [Sigma.mk.injEq, heq_eq_eq, true_and]
    simp

/-- The point masses of a deterministic strategy interpret decisions as `interp` does. -/
theorem interpB_pointMass (σ : M.Strategy) : M.interpB (M.pointMass σ) = M.interp σ := rfl

variable (β : M.Behaviour)
variable {X Y Z X' Y' : S.Iface}

/-- The total utility of the diagram's utility generators, read on their scopes, at the
input assignment `x` and the internal assignment `z`. -/
def rewardB (f : IDNet X Y) (x : (M.interpB β).Boundary X) (z : (M.interpB β).Internal f.net) :
    ℝ :=
  ∑ u, M.utility (f.ulabel u) ((f.uparents u).map ((M.interpB β).valuation f.net x z).value)

/-- The reward table: the probability-weighted total utility of the internal assignments
that produce each output. -/
def rewardTableB (f : IDNet X Y) :
    Kernel ((M.interpB β).Boundary X) ((M.interpB β).Boundary Y) :=
  fun x y ⇒ ∑ z, if y = (M.interpB β).read f.net x z then
    (M.interpB β).joint f.net x z * M.rewardB β f x z else 0

/-- The semantics of an open influence diagram under a behavioural strategy: the
sum-product kernel of its net and its reward table. -/
def interpretB (f : IDNet X Y) :
    RewardKernel ((M.interpB β).Boundary X) ((M.interpB β).Boundary Y) :=
  ⟨(M.interpB β).kernel f.net, M.rewardTableB β f⟩

theorem interpretB_stochastic (f : IDNet X Y) : (M.interpretB β f).Stochastic :=
  (M.interpB β).kernel_stochastic f.net
```

Rewards of composites

```
theorem rewardB_comp (f : IDNet X Y) (g : IDNet Y Z) (x : (M.interpB β).Boundary X)
    (a : (M.interpB β).Internal f.net) (b : (M.interpB β).Internal g.net) :
    M.rewardB β (f.comp g) x ((M.interpB β).combine f.net g.net a b) =
      M.rewardB β f x a + M.rewardB β g ((M.interpB β).read f.net x a) b := by
  unfold rewardB
```

```
  erw [Fintype.sum_sum_type]
  congr 1
  · apply Finset.sum_congr rfl
    intro u _
    change M.utility (f.ulabel u)
      (((f.uparents u).map (Net.left f.net g.net)).map _) = _
    rw [List.map_map]
    congr 1
    exact congrArg (fun k ⇒ (f.uparents u).map k)
      (funext ((M.interpB β).valuation_comp_left f.net g.net x a b))
  · apply Finset.sum_congr rfl
    intro u _
    change M.utility (g.ulabel u)
      (((g.uparents u).map (Net.right f.net g.net)).map _) = _
    rw [List.map_map]
    congr 1
    exact congrArg (fun k ⇒ (g.uparents u).map k)
      (funext ((M.interpB β).valuation_comp_right f.net g.net x a b))

theorem rewardB_tensor (f : IDNet X Y) (g : IDNet X' Y') (x : (M.interpB β).Boundary X)
    (x' : (M.interpB β).Boundary X') (a : (M.interpB β).Internal f.net)
    (b : (M.interpB β).Internal g.net) :
    M.rewardB β (f.tensor g) (Interpretation.Assignment.join _ x x')
        ((M.interpB β).combine f.net g.net a b) =
      M.rewardB β f x a + M.rewardB β g x' b := by
  unfold rewardB
  erw [Fintype.sum_sum_type]
  congr 1
  · apply Finset.sum_congr rfl
    intro u _
    change M.utility (f.ulabel u)
      (((f.uparents u).map (Net.tinl f.net g.net)).map _) = _
    rw [List.map_map]
    congr 1
    exact congrArg (fun k ⇒ (f.uparents u).map k)
      (funext ((M.interpB β).valuation_tensor_left f.net g.net x x' a b))
  · apply Finset.sum_congr rfl
    intro u _
    change M.utility (g.ulabel u)
      (((g.uparents u).map (Net.tinr f.net g.net)).map _) = _
    rw [List.map_map]
    congr 1
    exact congrArg (fun k ⇒ (g.uparents u).map k)
      (funext ((M.interpB β).valuation_tensor_right f.net g.net x x' a b))
```

Functoriality under serial composition

```
/-- Summing a product of two output-fibre sums over the middle boundary substitutes the
first component's output into the second. -/
theorem sum_fibre_mulB {f : Net X Y} {g : Net Y Z} (x : (M.interpB β).Boundary X)
    (w : (M.interpB β).Boundary Z) (F : (M.interpB β).Internal f → ℝ)
    (G : (M.interpB β).Boundary Y → (M.interpB β).Internal g → ℝ) :
    (∑ y, (∑ a, if y = (M.interpB β).read f x a then F a else 0) *
        (∑ b, if w = (M.interpB β).read g y b then G y b else 0)) =
      ∑ a, ∑ b, if w = (M.interpB β).read g ((M.interpB β).read f x a) b then
        F a * G ((M.interpB β).read f x a) b else 0 := by
  simp only [Finset.sum_mul]
  rw [Finset.sum_comm]
  apply Finset.sum_congr rfl
  intro a _
  rw [Finset.sum_eq_single ((M.interpB β).read f x a)]
  · simp only [if_true, Finset.mul_sum, mul_ite, mul_zero]
  · intro y _ hy
    simp [hy]
  · simp
```

```
theorem kernel_eq_sumB (f : Net X Y) (x : (M.interpB β).Boundary X)
    (y : (M.interpB β).Boundary Y) :
    (M.interpB β).kernel f x y =
      ∑ z, if y = (M.interpB β).read f x z then (M.interpB β).joint f x z else 0 := by
  unfold Interpretation.kernel
  rfl

theorem rewardTableB_comp (f : IDNet X Y) (g : IDNet Y Z) :
    M.rewardTableB β (f.comp g) =
      Kernel.comp (M.rewardTableB β f) ((M.interpB β).kernel g.net) +
        Kernel.comp ((M.interpB β).kernel f.net) (M.rewardTableB β g) := by
  funext x w
  simp only [Pi.add_apply, Kernel.comp, kernel_eq_sumB]
  unfold rewardTableB
  rw [M.sum_fibre_mulB β x w (fun a ⇒ (M.interpB β).joint f.net x a * M.rewardB β f x a)
      (fun y b ⇒ (M.interpB β).joint g.net y b),
    M.sum_fibre_mulB β x w (fun a ⇒ (M.interpB β).joint f.net x a)
      (fun y b ⇒ (M.interpB β).joint g.net y b * M.rewardB β g y b)]
  rw [← Finset.sum_add_distrib]
  change (∑ z : (M.interpB β).Internal (f.net.comp g.net),
      if w = (M.interpB β).read (f.net.comp g.net) x z then
        (M.interpB β).joint (f.net.comp g.net) x z * M.rewardB β (f.comp g) x z else 0) = _
  erw [← ((M.interpB β).sumLabelsEquiv f.net.label g.net.label).symm.sum_comp]
  rw [Fintype.sum_prod_type]
  apply Finset.sum_congr rfl
  intro a _
  rw [← Finset.sum_add_distrib]
  apply Finset.sum_congr rfl
  intro b _
  have hr := (M.interpB β).read_comp f.net g.net x a b
  have hj := (M.interpB β).joint_comp f.net g.net x a b
  have hw := M.rewardB_comp β f g x a b
  simp only [Interpretation.combine] at hr hj hw
  rw [hr, hj, hw]
  split_ifs <;> ring

/-- **Functoriality under serial composition**: the interpretation of a composite is the
composite, in the expectation-semiring kernel category, of the interpretations. -/
theorem interpretB_comp (f : IDNet X Y) (g : IDNet Y Z) :
    M.interpretB β (f.comp g) = (M.interpretB β f).comp (M.interpretB β g) := by
  ext1
  · exact (M.interpB β).kernel_comp f.net g.net
  · exact M.rewardTableB_comp β f g

theorem interpretB_identity (X : S.Iface) :
    M.interpretB β (IDNet.identity X) = RewardKernel.id ((M.interpB β).Boundary X) := by
  ext1
  · change (M.interpB β).kernel (Net.wire id (fun _ ⇒ rfl)) = Kernel.idK _
    rw [(M.interpB β).kernel_wire]
    apply Kernel.ofFun_congr
    intro x
    rfl
  · funext x y
    have hr : ∀ z, M.rewardB β (IDNet.identity X) x z = 0 :=
      fun z ⇒ Finset.sum_eq_zero (fun u _ ⇒ (u : Empty).elim)
    change M.rewardTableB β (IDNet.identity X) x y = 0
    unfold rewardTableB
    simp [hr]
```

Functoriality under tensor

```
/-- The canonical bijection between pairs of boundary assignments and assignments of the
disjoint-union interface. -/
def joinB (X X' : S.Iface) :
```

```
    (M.interpB β).Boundary X × (M.interpB β).Boundary X' ≃ (M.interpB β).Boundary (X.sum X') :=
  (Interpretation.Assignment.sumEquiv (M.interpB β) X.type X'.type).symm

@[simp] theorem joinB_apply (X X' : S.Iface) (p : (M.interpB β).Boundary X ×
    (M.interpB β).Boundary X') :
    M.joinB β X X' p = Interpretation.Assignment.join _ p.1 p.2 := rfl

theorem rewardTableB_tensor (f : IDNet X Y) (g : IDNet X' Y') (x : (M.interpB β).Boundary X)
    (x' : (M.interpB β).Boundary X') (y : (M.interpB β).Boundary Y)
    (y' : (M.interpB β).Boundary Y') :
    M.rewardTableB β (f.tensor g) (Interpretation.Assignment.join _ x x')
        (Interpretation.Assignment.join _ y y') =
      M.rewardTableB β f x y * (M.interpB β).kernel g.net x' y' +
        (M.interpB β).kernel f.net x y * M.rewardTableB β g x' y' := by
  simp only [kernel_eq_sumB]
  unfold rewardTableB
  rw [Finset.sum_mul_sum, Finset.sum_mul_sum, ← Finset.sum_add_distrib]
  change (∑ z : (M.interpB β).Internal (f.net.tensor g.net),
      if Interpretation.Assignment.join _ y y' =
          (M.interpB β).read (f.net.tensor g.net) (Interpretation.Assignment.join _ x x') z then
        (M.interpB β).joint (f.net.tensor g.net) (Interpretation.Assignment.join _ x x') z *
          M.rewardB β (f.tensor g) (Interpretation.Assignment.join _ x x') z else 0) = _
  erw [← ((M.interpB β).sumLabelsEquiv f.net.label g.net.label).symm.sum_comp]
  rw [Fintype.sum_prod_type]
  apply Finset.sum_congr rfl
  intro a _
  rw [← Finset.sum_add_distrib]
  apply Finset.sum_congr rfl
  intro b _
  have hr := (M.interpB β).read_tensor f.net g.net x x' a b
  have hj := (M.interpB β).joint_tensor f.net g.net x x' a b
  have hw := M.rewardB_tensor β f g x x' a b
  simp only [Interpretation.combine] at hr hj hw
  rw [hr, hj, hw]
  by_cases h₁ : y = (M.interpB β).read f.net x a <;>
    by_cases h₂ : y' = (M.interpB β).read g.net x' b <;>
      simp [Interpretation.Assignment.join_injective, h₁, h₂]
  ring

theorem kernel_tensor_joinB (f : IDNet X Y) (g : IDNet X' Y') (x : (M.interpB β).Boundary X)
    (x' : (M.interpB β).Boundary X') (y : (M.interpB β).Boundary Y)
    (y' : (M.interpB β).Boundary Y') :
    (M.interpB β).kernel (f.tensor g).net (Interpretation.Assignment.join _ x x')
        (Interpretation.Assignment.join _ y y') =
      (M.interpB β).kernel f.net x y * (M.interpB β).kernel g.net x' y' :=
  (M.interpB β).kernel_tensor f.net g.net x x' y y'


/-- **Functoriality under tensor** for a behavioural strategy. -/
theorem interpretB_tensor (f : IDNet X Y) (g : IDNet X' Y') :
    (M.interpretB β (f.tensor g)).reindex (M.joinB β X X') (M.joinB β Y Y') =
      (M.interpretB β f).tensor (M.interpretB β g) := by
  ext1
  · funext p q
    exact M.kernel_tensor_joinB β f g p.1 p.2 q.1 q.2
  · funext p q
    exact M.rewardTableB_tensor β f g p.1 p.2 q.1 q.2
```

The contextual identity for interpreted diagrams

```
/-- **Contextual identity** for open influence diagrams under a behavioural strategy: for every
boundary prior `μ` and terminal continuation `c`,
`EU_{f;g}(μ, c) = EU_f(μ, c_g) = L_f + EU_g(ν_f, c)`, where `c_g` is the suffix's value
function, `ν_f` the boundary law induced by the prefix and `L_f` the prefix's reward. -/
theorem contextual_identityB (f : IDNet X Y) (g : IDNet Y Z)
```

```
    (μ : (M.interpB β).Boundary X → ℝ) (c : (M.interpB β).Boundary Z → ℝ) :
    (M.interpretB β (f.comp g)).eu μ c =
        (M.interpretB β f).eu μ ((M.interpretB β g).continuation c) ∧
      (M.interpretB β f).eu μ ((M.interpretB β g).continuation c) =
        (M.interpretB β f).prefixReward μ +
          (M.interpretB β g).eu ((M.interpretB β f).pushforward μ) c := by
  rw [M.interpretB_comp β f g]
  exact RewardKernel.contextual_identity _ _ (M.interpretB_stochastic β g).2 μ c

/-- A deterministic strategy has the same semantics as the behavioural strategy of its point
masses. -/
theorem interpretB_pointMass (σ : M.Strategy) (f : IDNet X Y) :
    M.interpretB (M.pointMass σ) f = M.interpret σ f := rfl

end IDInterpretation
```

Mixtures of reward kernels

```
namespace RewardKernel

variable {X Y Z ι : Type} [Fintype X] [Fintype Y] [Fintype Z] [Fintype ι]

/-- The mixture `Σ_i q i · f i` of a finite family of reward kernels. -/
def mix (q : ι → ℝ) (f : ι → RewardKernel X Y) : RewardKernel X Y :=
  ⟨fun x y ⇒ ∑ i, q i * (f i).K x y, fun x y ⇒ ∑ i, q i * (f i).W x y⟩

omit [Fintype X] [Fintype Z] in
/-- Composition after a mixture is the mixture of the composites. -/
theorem mix_comp (q : ι → ℝ) (f : ι → RewardKernel X Y) (g : RewardKernel Y Z) :
    (mix q f).comp g = mix q (fun i ⇒ (f i).comp g) := by
  ext x z
  · simp only [mix, comp_K, Kernel.comp_apply, Finset.sum_mul, Finset.mul_sum]
    rw [Finset.sum_comm]
    exact Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_congr rfl fun _ _ ⇒ by ring
  · simp only [mix, comp_W, comp_K, Kernel.comp_apply, Pi.add_apply, Finset.sum_mul,
      Finset.mul_sum, mul_add, Finset.sum_add_distrib]
    rw [Finset.sum_comm (f := fun y i ⇒ q i * (f i).W x y * g.K y z),
      Finset.sum_comm (f := fun y i ⇒ q i * (f i).K x y * g.W y z)]
    simp only [mul_assoc]

omit [Fintype X] [Fintype Z] in
/-- Composition before a mixture is the mixture of the composites. -/
theorem comp_mix [Fintype X] (f : RewardKernel X Y) (q : ι → ℝ) (g : ι → RewardKernel Y Z) :
    f.comp (mix q g) = mix q (fun i ⇒ f.comp (g i)) := by
  ext x z
  · simp only [mix, comp_K, Kernel.comp_apply, Finset.mul_sum]
    rw [Finset.sum_comm]
    exact Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_congr rfl fun _ _ ⇒ by ring
  · simp only [mix, comp_W, comp_K, Kernel.comp_apply, Pi.add_apply, Finset.mul_sum,
      mul_add, Finset.sum_add_distrib]
    rw [Finset.sum_comm (f := fun y i ⇒ f.W x y * (q i * (g i).K y z)),
      Finset.sum_comm (f := fun y i ⇒ f.K x y * (q i * (g i).W y z))]
    congr 1 <;> exact Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_congr rfl fun _ _ ⇒ by ring

omit [Fintype Z] in
/-- The expected utility of a mixture is the mixture of the expected utilities. -/
theorem eu_mix (q : ι → ℝ) (f : ι → RewardKernel X Y) (μ : X → ℝ) (c : Y → ℝ) :
    (mix q f).eu μ c = ∑ i, q i * (f i).eu μ c := by
  have h1 : ∀ x y, (∑ i, q i * (f i).W x y) + (∑ i, q i * (f i).K x y) * c y =
      ∑ i, q i * ((f i).W x y + (f i).K x y * c y) := by
    intro x y
    rw [Finset.sum_mul, ← Finset.sum_add_distrib]
    exact Finset.sum_congr rfl fun _ _ ⇒ by ring
  calc (mix q f).eu μ c
      = ∑ x, ∑ y, ∑ i, q i * (μ x * ((f i).W x y + (f i).K x y * c y)) := by
```

```
        simp only [eu, mix, h1, Finset.mul_sum]
        exact Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_congr rfl fun _ _ ⇒
          Finset.sum_congr rfl fun _ _ ⇒ by ring
    _ = ∑ x, ∑ i, ∑ y, q i * (μ x * ((f i).W x y + (f i).K x y * c y)) :=
        Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_comm
    _ = ∑ i, ∑ x, ∑ y, q i * (μ x * ((f i).W x y + (f i).K x y * c y)) := Finset.sum_comm
    _ = ∑ i, q i * (f i).eu μ c := by
        simp only [eu, Finset.mul_sum]

omit [Fintype Z] in
/-- **Randomising over whole strategies does not help**: the expected utility of a mixture
with nonnegative weights summing to one is at most the best expected utility among its
components. -/
theorem eu_mix_le [Nonempty ι] (q : ι → ℝ) (hq0 : ∀ i, 0 ≤ q i) (hq : ∑ i, q i = 1)
    (f : ι → RewardKernel X Y) (μ : X → ℝ) (c : Y → ℝ) :
    ∃ i, (mix q f).eu μ c ≤ (f i).eu μ c := by
  classical
  obtain ⟨i, _, hi⟩ := Finset.exists_max_image Finset.univ (fun i ⇒ (f i).eu μ c)
    Finset.univ_nonempty
  refine ⟨i, ?_⟩
  rw [eu_mix q]
  calc ∑ j, q j * (f j).eu μ c ≤ ∑ j, q j * (f i).eu μ c :=
        Finset.sum_le_sum fun j hj ⇒ mul_le_mul_of_nonneg_left (hi j hj) (hq0 j)
    _ = (f i).eu μ c := by rw [← Finset.sum_mul, hq, one_mul]

/-- **The mediant inequality**: a ratio of nonnegatively weighted sums, with positive total
denominator, is at most the best ratio among the terms of positive denominator, provided a
term of zero denominator has zero numerator. -/
theorem mediant_le (w N D : ι → ℝ) (hw : ∀ i, 0 ≤ w i) (hD : ∀ i, 0 ≤ D i)
    (hND : ∀ i, D i = 0 → N i = 0) (hpos : 0 < ∑ i, w i * D i) :
    ∃ i, 0 < D i ∧ (∑ j, w j * N j) / (∑ j, w j * D j) ≤ N i / D i := by
  classical
  by_contra hcon
  push Not at hcon
  set T := (∑ j, w j * N j) / (∑ j, w j * D j) with hTdef
  have hT : T * (∑ j, w j * D j) = ∑ j, w j * N j := div_mul_cancel₀ _ hpos.ne'
  have hle : ∀ j, w j * N j ≤ T * (w j * D j) := by
    intro j
    rcases (hD j).lt_or_eq with h | h
    · have h' := hcon j h
      rw [div_lt_iff₀ h] at h'
      nlinarith [hw j]
    · rw [hND j h.symm, ← h]
      simp
  obtain ⟨j, hj⟩ : ∃ j, 0 < w j * D j := by
    by_contra hno
    push Not at hno
    exact absurd hpos (not_lt.mpr (Finset.sum_nonpos fun j _ ⇒ hno j))
  have hDj : 0 < D j := by
    rcases (hD j).lt_or_eq with h | h
    · exact h
    · rw [← h, mul_zero] at hj
      exact absurd hj (lt_irrefl 0)
  have hwj : 0 < w j := by
    rcases (hw j).lt_or_eq with h | h
    · exact h
    · rw [← h, zero_mul] at hj
      exact absurd hj (lt_irrefl 0)
  have hstrict : w j * N j < T * (w j * D j) := by
    have h' := hcon j hDj
    rw [div_lt_iff₀ hDj] at h'
    nlinarith
  have hsum : ∑ j, w j * N j < T * ∑ j, w j * D j := by
    rw [Finset.mul_sum]
    exact Finset.sum_lt_sum (fun j _ ⇒ hle j) ⟨j, Finset.mem_univ _, hstrict⟩
  linarith
```

```
omit [Fintype X] [Fintype Z] in
theorem pull_mix (q : ι → ℝ) (f : ι → RewardKernel X Y) (e : Effect Y) (x : X) :
    ((mix q f).pull e).m x = ∑ i, q i * ((f i).pull e).m x ∧
      ((mix q f).pull e).c x = ∑ i, q i * ((f i).pull e).c x := by
  constructor
  · simp only [pull, mix, Finset.sum_mul, Finset.mul_sum]
    rw [Finset.sum_comm]
    exact Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_congr rfl fun _ _ ⇒ by ring
  · simp only [pull, mix, Finset.sum_mul, Finset.mul_sum, ← Finset.sum_add_distrib]
    rw [Finset.sum_comm]
    exact Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_congr rfl fun _ _ ⇒ by ring

omit [Fintype Z] in
/-- The evidence mass and the evidence-weighted value of a mixture are the mixtures. -/
theorem mass_value_mix (q : ι → ℝ) (f : ι → RewardKernel X Y) (s : State X) (e : Effect Y) :
    mass s ((mix q f).pull e) = ∑ i, q i * mass s ((f i).pull e) ∧
      value s ((mix q f).pull e) = ∑ i, q i * value s ((f i).pull e) := by
  constructor
  · simp only [mass, (pull_mix q f e _).1, Finset.mul_sum]
    rw [Finset.sum_comm]
    exact Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_congr rfl fun _ _ ⇒ by ring
  · simp only [value, (pull_mix q f e _).1, (pull_mix q f e _).2, Finset.mul_sum,
      ← Finset.sum_add_distrib]
    rw [Finset.sum_comm]
    exact Finset.sum_congr rfl fun _ _ ⇒ Finset.sum_congr rfl fun _ _ ⇒ by ring

omit [Fintype Z] in
/-- **Randomising does not help given evidence either**: the conditional value of a mixture
is at most the conditional value of one of its components of positive evidence mass, when
masses are nonnegative and a component of zero mass has zero evidence-weighted value. -/
theorem conditionalValue_mix_le (q : ι → ℝ) (hq0 : ∀ i, 0 ≤ q i)
    (f : ι → RewardKernel X Y) (s : State X) (e : Effect Y)
    (hmass : ∀ i, 0 ≤ mass s ((f i).pull e))
    (hzero : ∀ i, mass s ((f i).pull e) = 0 → value s ((f i).pull e) = 0)
    (hpos : 0 < mass s ((mix q f).pull e)) :
    ∃ i, 0 < mass s ((f i).pull e) ∧
      conditionalValue s ((mix q f).pull e) ≤ conditionalValue s ((f i).pull e) := by
  rw [(mass_value_mix q f s e).1] at hpos
  unfold conditionalValue
  rw [(mass_value_mix q f s e).1, (mass_value_mix q f s e).2]
  exact mediant_le q _ _ hq0 hmass hzero hpos

end RewardKernel
```

Shared policies: the absent-minded driver

```
namespace AbsentMinded

open RewardKernel

/-- The first junction: continue with probability `p`; exiting here pays nothing. -/
def first (p : ℝ) : RewardKernel Unit Bool :=
  ⟨fun _ b ⇒ if b then p else 1 - p, fun _ _ ⇒ 0⟩

/-- The second junction, reached only by continuing: continue with probability `q` (paying
`1`) or exit (paying `4`). -/
def second (q : ℝ) : RewardKernel Bool Unit :=
  ⟨fun _ _ ⇒ 1, fun b _ ⇒ if b then 4 * (1 - q) + q else 0⟩

/-- The value of the drive with continuation probabilities `p` and `q`. -/
theorem drive_eu (p q : ℝ) :
    ((first p).comp (second q)).eu (fun _ ⇒ 1) (fun _ ⇒ 0) = p * (4 * (1 - q) + q) := by
  simp [eu, first, second, comp, Kernel.comp]
```

```
/-- **Randomising helps when two decisions share a policy.** Continuing with probability
`2/3` at both junctions is worth `4/3`; either deterministic policy is worth at most `1`. -/
theorem randomising_helps :
    ((first (2 / 3)).comp (second (2 / 3))).eu (fun _ ⇒ 1) (fun _ ⇒ 0) = 4 / 3 ∧
      ∀ p ∈ ({0, 1} : Set ℝ), ((first p).comp (second p)).eu (fun _ ⇒ 1) (fun _ ⇒ 0) ≤ 1 := by
  refine ⟨by rw [drive_eu]; norm_num, fun p hp ⇒ ?_⟩
  rw [drive_eu]
  rcases hp with rfl | rfl <;> norm_num

/-- **With a policy each, a deterministic pair is optimal**: continuing at the first junction
and exiting at the second is worth `4`, and no pair of probabilities does better. -/
theorem independent_deterministic_optimal :
    ((first 1).comp (second 0)).eu (fun _ ⇒ 1) (fun _ ⇒ 0) = 4 ∧
      ∀ p q : ℝ, 0 ≤ p → p ≤ 1 → 0 ≤ q → q ≤ 1 →
        ((first p).comp (second q)).eu (fun _ ⇒ 1) (fun _ ⇒ 0) ≤ 4 := by
  refine ⟨by rw [drive_eu]; norm_num, fun p q hp0 hp1 hq0 hq1 ⇒ ?_⟩
  rw [drive_eu]
  nlinarith

end AbsentMinded

end OpenInfluence
```

Reward certificates: the record

```
import CategoricalInfluenceDiagramsProofs.Interpretation
```

ADR 0019 adds a certificate format, `OpenInfluence.RewardCertificate/v1`, that records one open influence diagram as `CategoricalInfluenceDiagrams.jl` holds it, together with its observability flags, its numerical tables, one deterministic strategy and the reward kernel `(K, W)` that Julia's `interpret` computed for that strategy. The format is specified in the workspace plan `docs/REWARD-CERTIFICATES-PLAN-2026-10-17.md` ("The format"), which is the contract between the Julia exporter and this checker.

This module defines the Lean record of a decoded certificate (`Certificate`). It reuses the two parent projects instead of restating them:

- the structural part is CategoricalBayesianNetworks' raw network, `OpenNet.RawCertificate.Network`, unchanged: dense variable indices, ranks, positioned state rows, generators with tagged references and positioned input rows, and attributed boundary rows. A generator is a **decision** exactly when its `kernelRef` is a `policy` reference (`isPolicy`); every other generator is a **chance** generator;
- a number cell is InfluenceDiagrams' certificate value (`InfluenceDiagramsProofs.DVECertificate.Value`): a binary64 word, optionally with an exact rational companion, read exactly by `Value.toRat`.

What the format adds is recorded here as plain data: one observability flag per boundary row, one chance table per chance generator, the utilities (a name, positioned scope rows that have the shape of input rows, and a table), one policy per distinct decision name, and the recorded `K` and `W`. Every table is a flat list of cells in lexicographic order of its axes, the last axis fastest (`Exact.lean` defines that index). Decoding is `Json.lean`, the checker `Check.lean`, the exact reward kernel `Exact.lean` and the soundness theorems `Sound.lean`.

```
set_option autoImplicit false

namespace OpenInfluence.RewardCertificate

open OpenNet.RawCertificate

/-- A number cell, as in InfluenceDiagrams' DVE certificate. -/
abbrev Value := InfluenceDiagramsProofs.DVECertificate.Value

/-- The numeric mode: binary64 words only, or exact rational companions on every chance and
utility cell. -/
inductive NumericMode where
  | binary64Exact
  | rationalExact
```

```
  deriving DecidableEq, Repr

/-- The JSON spelling of a numeric mode. -/
def NumericMode.toString : NumericMode → String
  | .binary64Exact ⇒ "binary64_exact"
  | .rationalExact ⇒ "rational_exact"

/-- Provenance strings; the checker does not read them. -/
structure Metadata where
  producer : String
  producerVersion : String
  juliaVersion : String
  description : String
  deriving DecidableEq, Repr

/-- The table of one chance generator: its 0-based generator index and its cells (axes: the
parents in input-position order, then the child). -/
structure ChanceTable where
  mechanism : ℕ
  cells : List Value

/-- One utility: its name, its scope rows (`varId` and 1-based `position`, the shape of an
input row) and its cells (axes: the scope rows in position order). -/
structure Utility (n : ℕ) where
  name : String
  scopeRows : List (InputRow n)
  cells : List Value

/-- The policy of one decision name: the 1-based action state position at every cell of the
information axes of the first decision generator with that name. -/
structure Policy where
  decision : String
  actions : List ℕ
  deriving DecidableEq, Repr

/-- Julia's reward kernel at the boundary coordinates (axes: the input rows, then the output
rows). `W` is signed. The arithmetic is always `"binary64"`. -/
structure Result where
  K : List Value
  W : List Value

/-- A decoded `OpenInfluence.RewardCertificate/v1`. -/
structure Certificate where
  numericMode : NumericMode
  metadata : Metadata
  network : Network
  inputFlags : List Observability
  outputFlags : List Observability
  chanceTables : List ChanceTable
  utilities : List (Utility network.variableCount)
  strategy : List Policy
  result : Result

/-- The format string of version 1. -/
def formatName : String := "OpenInfluence.RewardCertificate/v1"

/-- A reference marks a decision exactly when it is a `policy` reference. -/
def isPolicy : Reference → Bool
  | .policy _ ⇒ true
  | _ ⇒ false

/-- The decision name of a `policy` reference. -/
def policyName? : Reference → Option String
  | .policy d ⇒ some d
  | _ ⇒ none
```

```
theorem isPolicy_iff (r : Reference) : isPolicy r = true ↔ ∃ d, r = .policy d := by
  cases r <;> simp [isPolicy]

/-- A utility label of the decoded diagram: the utility's name and the attributes of its
scope variables in position order (`Sound.lean` uses these as the signature's utility
labels). -/
structure ULabel where
  name : String
  inputs : List VariableAttrs
  deriving DecidableEq, Repr

end OpenInfluence.RewardCertificate
```

Reward certificates: decoding the JSON

```
import CategoricalInfluenceDiagramsProofs.Certificate.Record
```

This module decodes a parsed `Lean.Json` tree into a `Certificate` (`decodeCertificate`) and a certificate text into one (`read`). It reuses the readers of the two parent projects:

- the `network` object is decoded by CategoricalBayesianNetworks' record readers (`OpenNet.RawCertificate.JSON.variableData`, `mechanism`, `boundary`, with its `keys`, `field`, `list` and `natural`), exactly as `OpenNet.RawCertificate.JSON.network` decodes the body of `OpenNet.RawCertificate/v1`, except that there is no `format` key (`networkBody`). Utility scope rows have the shape of input rows and are decoded by its `inputRow`;
- number cells are decoded by InfluenceDiagrams' `decodeValue`, and the new records use its key and array helpers (`get`, `getArr`) and BayesianNetworks' exact-size `object`;
- the text is parsed by CategoricalBayesianNetworks' strict parser (`OpenNet.CertificateSyntax.parse`), which rejects duplicate keys, trailing input and non-integer number tokens.

Every object must have exactly its listed keys: a missing key fails, and since a JSON object here has distinct keys, an extra key changes the size and fails too. Key order is ignored.

Results. For each new record, `decodeX_eq_ok` states that decoding succeeds with `x` exactly when the JSON is `x`'s layout key for key (`XMatches`): flags (`decodeFlag_eq_ok`), the numeric mode, metadata, observability, chance tables, utilities, policies and the result. `decodeCertificate_eq_ok` states the same for the whole certificate, with the `network` and the utility scope rows specified by the reused readers (the network reader has no such characterisation in its own project; `networkBody_fields` records what its success means). `read_eq_ok` adds the parser.

Trusted, not proved: the parser itself (shared with CategoricalBayesianNetworks) and the Julia exporter.

```
set_option autoImplicit false

namespace OpenInfluence.RewardCertificate.Json

open Lean (Json)
open OpenNet.RawCertificate
open BayesianNetworksProofs.Raw (JsonObject object object_eq_ok bind_eq_ok pure_eq_ok
  throw_ne_ok natJson)
open InfluenceDiagramsProofs.DVECertificate (get get_eq_ok getArr getArr_eq_ok ArrayMatches
  strOf strOf_eq_ok natOf natOf_eq_ok decodeValue decodeValue_eq_ok ValueMatches)
```

Constant strings

```
/-- A string that must equal `expected`. -/
def constant (expected : String) (j : Json) : Except String Unit := do
  let s ← strOf j
  if s = expected then pure () else throw s!"expected \"{expected}\", found \"{s}\""

theorem constant_eq_ok {expected : String} {j : Json} {u : Unit} :
    constant expected j = .ok u ↔ j = .str expected := by
```

```
  unfold constant
  rw [bind_eq_ok]
  constructor
  · rintro ⟨s, hs, h⟩
    rw [strOf_eq_ok] at hs
    subst hs
    split_ifs at h with he
    · rw [he]
  · rintro rfl
    exact ⟨expected, strOf_eq_ok.2 rfl, by rw [if_pos rfl]; rfl⟩
```

Flags and numeric mode

```
/-- The JSON spelling of a flag. -/
def flagString : Observability → String
  | .observed ⇒ "observed"
  | .hidden ⇒ "hidden"

def decodeFlag (j : Json) : Except String Observability := do
  let s ← strOf j
  if s = "observed" then pure .observed
  else if s = "hidden" then pure .hidden
  else throw s!"unknown observability flag \"{s}\""

theorem decodeFlag_eq_ok {j : Json} {b : Observability} :
    decodeFlag j = .ok b ↔ j = .str (flagString b) := by
  unfold decodeFlag
  rw [bind_eq_ok]
  constructor
  · rintro ⟨s, hs, h⟩
    rw [strOf_eq_ok] at hs
    subst hs
    split_ifs at h with h1 h2
    · rw [pure_eq_ok] at h
      subst h h1
      rfl
    · rw [pure_eq_ok] at h
      subst h h2
      rfl
  · rintro rfl
    refine ⟨_, strOf_eq_ok.2 rfl, ?_⟩
    cases b <;> simp [flagString] <;> rfl

def decodeMode (j : Json) : Except String NumericMode := do
  let s ← strOf j
  if s = "binary64_exact" then pure .binary64Exact
  else if s = "rational_exact" then pure .rationalExact
  else throw s!"unknown numeric mode \"{s}\""

theorem decodeMode_eq_ok {j : Json} {m : NumericMode} :
    decodeMode j = .ok m ↔ j = .str m.toString := by
  unfold decodeMode
  rw [bind_eq_ok]
  constructor
  · rintro ⟨s, hs, h⟩
    rw [strOf_eq_ok] at hs
    subst hs
    split_ifs at h with h1 h2
    · rw [pure_eq_ok] at h
      subst h h1
      rfl
    · rw [pure_eq_ok] at h
      subst h h2
      rfl
```

```
  · rintro rfl
    refine ⟨_, strOf_eq_ok.2 rfl, ?_⟩
    cases m <;> simp [NumericMode.toString] <;> rfl
```

Metadata and observability

```
def decodeMetadata (j : Json) : Except String Metadata := do
  let o ← object "metadata" 4 j
  let p ← get o "producer" strOf
  let pv ← get o "producer_version" strOf
  let jv ← get o "julia_version" strOf
  let d ← get o "description" strOf
  pure ⟨p, pv, jv, d⟩

def MetadataMatches (j : Json) (m : Metadata) : Prop :=
  ∃ o : JsonObject, j = .obj o ∧ o.size = 4 ∧ o["producer"]? = some (.str m.producer) ∧
    o["producer_version"]? = some (.str m.producerVersion) ∧
    o["julia_version"]? = some (.str m.juliaVersion) ∧
    o["description"]? = some (.str m.description)

theorem decodeMetadata_eq_ok {j : Json} {m : Metadata} :
    decodeMetadata j = .ok m ↔ MetadataMatches j m := by
  unfold decodeMetadata MetadataMatches
  simp only [bind_eq_ok, object_eq_ok, get_eq_ok fun _ _ ⇒ strOf_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, _, ⟨_, h1, rfl⟩, _, ⟨_, h2, rfl⟩, _, ⟨_, h3, rfl⟩, _, ⟨_, h4, rfl⟩,
      rfl⟩
    exact ⟨o, rfl, hs, h1, h2, h3, h4⟩
  · rintro ⟨o, rfl, hs, h1, h2, h3, h4⟩
    exact ⟨o, ⟨rfl, hs⟩, _, ⟨_, h1, rfl⟩, _, ⟨_, h2, rfl⟩, _, ⟨_, h3, rfl⟩, _, ⟨_, h4, rfl⟩,
      rfl⟩

/-- A flag array: element `k` is the spelling of flag `k`. -/
def FlagsMatch (j : Json) (l : List Observability) : Prop :=
  ArrayMatches (fun _ v b ⇒ v = .str (flagString b)) j l

def decodeObservability (j : Json) :
    Except String (List Observability × List Observability) := do
  let o ← object "observability" 2 j
  let i ← getArr o "inputs" (fun _ ⇒ decodeFlag)
  let out ← getArr o "outputs" (fun _ ⇒ decodeFlag)
  pure (i, out)

def ObservabilityMatches (j : Json) (ins outs : List Observability) : Prop :=
  ∃ o : JsonObject, j = .obj o ∧ o.size = 2 ∧
    (∃ v, o["inputs"]? = some v ∧ FlagsMatch v ins) ∧
    (∃ v, o["outputs"]? = some v ∧ FlagsMatch v outs)

theorem decodeObservability_eq_ok {j : Json} {ins outs : List Observability} :
    decodeObservability j = .ok (ins, outs) ↔ ObservabilityMatches j ins outs := by
  unfold decodeObservability ObservabilityMatches FlagsMatch
  simp only [bind_eq_ok, object_eq_ok, getArr_eq_ok fun _ _ _ ⇒ decodeFlag_eq_ok, pure_eq_ok,
    Prod.mk.injEq]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, _, hi, _, ho, rfl, rfl⟩
    exact ⟨o, rfl, hs, hi, ho⟩
  · rintro ⟨o, rfl, hs, hi, ho⟩
    exact ⟨o, ⟨rfl, hs⟩, _, hi, _, ho, rfl, rfl⟩
```

Tables, utilities, policies and the result

```
/-- A cell array: element `k` is the layout of cell `k`. -/
def CellsMatch (j : Json) (l : List Value) : Prop :=
```

```
  ArrayMatches (fun _ ⇒ ValueMatches) j l

def decodeChanceTable (j : Json) : Except String ChanceTable := do
  let o ← object "chance table" 2 j
  let g ← get o "mechanism" natOf
  let cells ← getArr o "cells" (fun _ ⇒ decodeValue)
  pure ⟨g, cells⟩

def ChanceTableMatches (j : Json) (t : ChanceTable) : Prop :=
  ∃ o : JsonObject, j = .obj o ∧ o.size = 2 ∧ o["mechanism"]? = some (natJson t.mechanism) ∧
    ∃ v, o["cells"]? = some v ∧ CellsMatch v t.cells

theorem decodeChanceTable_eq_ok {j : Json} {t : ChanceTable} :
    decodeChanceTable j = .ok t ↔ ChanceTableMatches j t := by
  unfold decodeChanceTable ChanceTableMatches CellsMatch
  simp only [bind_eq_ok, object_eq_ok, get_eq_ok fun _ _ ⇒ natOf_eq_ok,
    getArr_eq_ok fun _ _ _ ⇒ decodeValue_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, _, ⟨_, hg, rfl⟩, _, hc, rfl⟩
    exact ⟨o, rfl, hs, hg, hc⟩
  · rintro ⟨o, rfl, hs, hg, hc⟩
    exact ⟨o, ⟨rfl, hs⟩, _, ⟨_, hg, rfl⟩, _, hc, rfl⟩

/-- The scope rows of a utility, read by CategoricalBayesianNetworks' input-row reader. -/
def decodeScopeRows (n : ℕ) (j : Json) : Except String (List (InputRow n)) :=
  JSON.list (JSON.inputRow n) j

def decodeUtility (n : ℕ) (j : Json) : Except String (Utility n) := do
  let o ← object "utility" 3 j
  let name ← get o "name" strOf
  let rows ← get o "scopeRows" (decodeScopeRows n)
  let cells ← getArr o "cells" (fun _ ⇒ decodeValue)
  pure ⟨name, rows, cells⟩

def UtilityMatches (n : ℕ) (j : Json) (u : Utility n) : Prop :=
  ∃ o : JsonObject, j = .obj o ∧ o.size = 3 ∧ o["name"]? = some (.str u.name) ∧
    (∃ v, o["scopeRows"]? = some v ∧ decodeScopeRows n v = .ok u.scopeRows) ∧
    ∃ v, o["cells"]? = some v ∧ CellsMatch v u.cells

theorem decodeUtility_eq_ok {n : ℕ} {j : Json} {u : Utility n} :
    decodeUtility n j = .ok u ↔ UtilityMatches n j u := by
  unfold decodeUtility UtilityMatches CellsMatch
  simp only [bind_eq_ok, object_eq_ok, get_eq_ok fun _ _ ⇒ strOf_eq_ok,
    get_eq_ok (M := fun v x ⇒ decodeScopeRows n v = .ok x) fun _ _ ⇒ Iff.rfl,
    getArr_eq_ok fun _ _ _ ⇒ decodeValue_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, _, ⟨_, hn, rfl⟩, _, hr, _, hc, rfl⟩
    exact ⟨o, rfl, hs, hn, hr, hc⟩
  · rintro ⟨o, rfl, hs, hn, hr, hc⟩
    exact ⟨o, ⟨rfl, hs⟩, _, ⟨_, hn, rfl⟩, _, hr, _, hc, rfl⟩

def decodePolicy (j : Json) : Except String Policy := do
  let o ← object "policy" 2 j
  let d ← get o "decision" strOf
  let a ← getArr o "actions" (fun _ ⇒ natOf)
  pure ⟨d, a⟩

def PolicyMatches (j : Json) (p : Policy) : Prop :=
  ∃ o : JsonObject, j = .obj o ∧ o.size = 2 ∧ o["decision"]? = some (.str p.decision) ∧
    ∃ v, o["actions"]? = some v ∧ ArrayMatches (fun _ v a ⇒ v = natJson a) v p.actions

theorem decodePolicy_eq_ok {j : Json} {p : Policy} :
    decodePolicy j = .ok p ↔ PolicyMatches j p := by
  unfold decodePolicy PolicyMatches
  simp only [bind_eq_ok, object_eq_ok, get_eq_ok fun _ _ ⇒ strOf_eq_ok,
    getArr_eq_ok fun _ _ _ ⇒ natOf_eq_ok, pure_eq_ok]
```

```
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, _, ⟨_, hd, rfl⟩, _, ha, rfl⟩
    exact ⟨o, rfl, hs, hd, ha⟩
  · rintro ⟨o, rfl, hs, hd, ha⟩
    exact ⟨o, ⟨rfl, hs⟩, _, ⟨_, hd, rfl⟩, _, ha, rfl⟩

def decodeResult (j : Json) : Except String Result := do
  let o ← object "result" 3 j
  get o "arithmetic" (constant "binary64")
  let k ← getArr o "K" (fun _ ⇒ decodeValue)
  let w ← getArr o "W" (fun _ ⇒ decodeValue)
  pure ⟨k, w⟩

def ResultMatches (j : Json) (r : Result) : Prop :=
  ∃ o : JsonObject, j = .obj o ∧ o.size = 3 ∧ o["arithmetic"]? = some (.str "binary64") ∧
    (∃ v, o["K"]? = some v ∧ CellsMatch v r.K) ∧ ∃ v, o["W"]? = some v ∧ CellsMatch v r.W

theorem decodeResult_eq_ok {j : Json} {r : Result} :
    decodeResult j = .ok r ↔ ResultMatches j r := by
  unfold decodeResult ResultMatches CellsMatch
  simp only [bind_eq_ok, object_eq_ok, get_eq_ok fun _ _ ⇒ constant_eq_ok,
    getArr_eq_ok fun _ _ _ ⇒ decodeValue_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, _, ⟨_, ha, rfl⟩, _, hk, _, hw, rfl⟩
    exact ⟨o, rfl, hs, ha, hk, hw⟩
  · rintro ⟨o, rfl, hs, ha, hk, hw⟩
    exact ⟨o, ⟨rfl, hs⟩, (), ⟨_, ha, rfl⟩, _, hk, _, hw, rfl⟩
```

The network

```
/-- The body of `OpenNet.RawCertificate/v1` without its `format` key, decoded by
CategoricalBayesianNetworks' readers exactly as its `network` does. -/
def networkBody (j : Json) : Except String Network := do
  JSON.keys j ["variableCount", "variableData", "mechanisms", "inputs", "outputs"]
  let n ← JSON.field j "variableCount" JSON.natural
  let vars ← JSON.field j "variableData" (JSON.list JSON.variableData)
  if h : vars.length = n then
    return {
      variableCount := n
      variableData := fun i ⇒ vars[i.val]'(h.symm ▸ i.isLt)
      mechanisms := ← JSON.field j "mechanisms" (JSON.list (JSON.mechanism n))
      inputs := ← JSON.field j "inputs" (JSON.list (JSON.boundary n))
      outputs := ← JSON.field j "outputs" (JSON.list (JSON.boundary n)) }
  else
    throw "variableCount does not match variableData length"

/-- What a successful network decoding means: exactly the five keys, and every field is what
the reused reader returns. -/
theorem networkBody_fields {j : Json} {N : Network} (h : networkBody j = .ok N) :
    JSON.keys j ["variableCount", "variableData", "mechanisms", "inputs", "outputs"] = .ok () ∧
    JSON.field j "variableCount" JSON.natural = .ok N.variableCount ∧
    (∃ vars : List VariableData, JSON.field j "variableData" (JSON.list JSON.variableData) =
      .ok vars ∧ ∃ hl : vars.length = N.variableCount,
        ∀ i : Fin N.variableCount, N.variableData i = vars[i.val]'(hl.symm ▸ i.isLt)) ∧
    JSON.field j "mechanisms" (JSON.list (JSON.mechanism N.variableCount)) = .ok N.mechanisms ∧
    JSON.field j "inputs" (JSON.list (JSON.boundary N.variableCount)) = .ok N.inputs ∧
    JSON.field j "outputs" (JSON.list (JSON.boundary N.variableCount)) = .ok N.outputs := by
  unfold networkBody at h
  simp only [bind_eq_ok] at h
  obtain ⟨u, hk, n, hn, vars, hv, h⟩ := h
  cases u
  split at h
  · rename_i hl
    simp only [bind_eq_ok, pure_eq_ok] at h
```

```
    obtain ⟨ms, hm, is, hi, os, ho, rfl⟩ := h
    exact ⟨hk, hn, ⟨vars, hv, hl, fun _ ⇒ rfl⟩, hm, hi, ho⟩
  · exact absurd h throw_ne_ok
```

The certificate

```
/-- The top-level keys, in the format's order. -/
def certificateKeys : List String :=
  ["format", "numeric_mode", "metadata", "network", "observability", "chance_tables",
    "utilities", "strategy", "result"]

def decodeCertificate (j : Json) : Except String Certificate := do
  let o ← object "certificate" 9 j
  get o "format" (constant formatName)
  let mode ← get o "numeric_mode" decodeMode
  let md ← get o "metadata" decodeMetadata
  let N ← get o "network" networkBody
  let obs ← get o "observability" decodeObservability
  let tables ← getArr o "chance_tables" (fun _ ⇒ decodeChanceTable)
  let utils ← getArr o "utilities" (fun _ ⇒ decodeUtility N.variableCount)
  let strat ← getArr o "strategy" (fun _ ⇒ decodePolicy)
  let res ← get o "result" decodeResult
  pure ⟨mode, md, N, obs.1, obs.2, tables, utils, strat, res⟩

/-- The certificate `c` is the JSON `j`, key for key: the new records by their `Matches`
predicates, the network and the utility scope rows by the reused readers. -/
def CertificateMatches (j : Json) (c : Certificate) : Prop :=
  ∃ o : JsonObject, j = .obj o ∧ o.size = 9 ∧ o["format"]? = some (.str formatName) ∧
    o["numeric_mode"]? = some (.str c.numericMode.toString) ∧
    (∃ v, o["metadata"]? = some v ∧ MetadataMatches v c.metadata) ∧
    (∃ v, o["network"]? = some v ∧ networkBody v = .ok c.network) ∧
    (∃ v, o["observability"]? = some v ∧ ObservabilityMatches v c.inputFlags c.outputFlags) ∧
    (∃ v, o["chance_tables"]? = some v ∧
      ArrayMatches (fun _ ⇒ ChanceTableMatches) v c.chanceTables) ∧
    (∃ v, o["utilities"]? = some v ∧
      ArrayMatches (fun _ ⇒ UtilityMatches c.network.variableCount) v c.utilities) ∧
    (∃ v, o["strategy"]? = some v ∧ ArrayMatches (fun _ ⇒ PolicyMatches) v c.strategy) ∧
    ∃ v, o["result"]? = some v ∧ ResultMatches v c.result

theorem decodeObservability_eq_ok' {j : Json} {p : List Observability × List Observability} :
    decodeObservability j = .ok p ↔ ObservabilityMatches j p.1 p.2 := by
  obtain ⟨a, b⟩ := p
  exact decodeObservability_eq_ok

/-- **Faithfulness of the decoder**: decoding succeeds with `c` exactly when the JSON is
`c`'s layout. -/
theorem decodeCertificate_eq_ok {j : Json} {c : Certificate} :
    decodeCertificate j = .ok c ↔ CertificateMatches j c := by
  unfold decodeCertificate CertificateMatches
  simp only [bind_eq_ok, object_eq_ok, get_eq_ok fun _ _ ⇒ constant_eq_ok,
    get_eq_ok fun _ _ ⇒ decodeMode_eq_ok, get_eq_ok fun _ _ ⇒ decodeMetadata_eq_ok,
    get_eq_ok (M := fun v x ⇒ networkBody v = .ok x) fun _ _ ⇒ Iff.rfl,
    get_eq_ok fun _ _ ⇒ decodeObservability_eq_ok',
    getArr_eq_ok fun _ _ _ ⇒ decodeChanceTable_eq_ok,
    getArr_eq_ok fun _ _ _ ⇒ decodePolicy_eq_ok,
    get_eq_ok fun _ _ ⇒ decodeResult_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, _, ⟨_, hf, rfl⟩, mode, ⟨_, hm, rfl⟩, md, ⟨_, hmeta, hmeta'⟩, N,
      ⟨_, hn, hn'⟩, obs, ⟨_, hobs, hobs'⟩, tables, ht, utils, hu, strat, hst, res,
      ⟨_, hr, hr'⟩, rfl⟩
    rw [getArr_eq_ok fun _ _ _ ⇒ decodeUtility_eq_ok] at hu
    exact ⟨o, rfl, hs, hf, hm, ⟨_, hmeta, hmeta'⟩, ⟨_, hn, hn'⟩, ⟨_, hobs, hobs'⟩, ht, hu,
      hst, ⟨_, hr, hr'⟩⟩
  · obtain ⟨mode, md, N, ins, outs, tables, utils, strat, res⟩ := c
```

```
    rintro ⟨o, rfl, hs, hf, hm, ⟨_, hmeta, hmeta'⟩, ⟨_, hn, hn'⟩, ⟨_, hobs, hobs'⟩, ht, hu,
      hst, ⟨_, hr, hr'⟩⟩
    refine ⟨o, ⟨rfl, hs⟩, (), ⟨_, hf, rfl⟩, mode, ⟨_, hm, rfl⟩, md, ⟨_, hmeta, hmeta'⟩, N,
      ⟨_, hn, hn'⟩, (ins, outs), ⟨_, hobs, hobs'⟩, tables, ht, utils, ?_, strat, hst, res,
      ⟨_, hr, hr'⟩, rfl⟩
    rw [getArr_eq_ok fun _ _ _ ⇒ decodeUtility_eq_ok]
    exact hu

/-- Parse with CategoricalBayesianNetworks' strict parser, then decode. -/
def read (text : String) : Except String Certificate := do
  let j ← OpenNet.CertificateSyntax.parse text
  decodeCertificate j

theorem read_eq_ok {text : String} {c : Certificate} :
    read text = .ok c ↔ ∃ j, OpenNet.CertificateSyntax.parse text = .ok j ∧
      CertificateMatches j c := by
  unfold read
  rw [bind_eq_ok]
  simp only [decodeCertificate_eq_ok]

/-- A read certificate is the decoding of the parsed text, and its network is what the reused
network readers return on the `network` value. -/
theorem read_preserves_decoded {text : String} {c : Certificate} (h : read text = .ok c) :
    ∃ j, OpenNet.CertificateSyntax.parse text = .ok j ∧ decodeCertificate j = .ok c ∧
      ∃ o : JsonObject, j = .obj o ∧ ∃ v, o["network"]? = some v ∧
        networkBody v = .ok c.network := by
  obtain ⟨j, hj, hm⟩ := read_eq_ok.1 h
  have hd := decodeCertificate_eq_ok.2 hm
  obtain ⟨o, rfl, -, -, -, -, ⟨v, hv, hn⟩, -⟩ := hm
  exact ⟨_, hj, hd, o, rfl, v, hv, hn⟩

end OpenInfluence.RewardCertificate.Json
```

Reward certificates: the exact reward kernel of the certificate's numbers

```
import CategoricalInfluenceDiagramsProofs.Certificate.Record
```

This module computes, over $\mathbb{Q}$ and by explicit enumeration, the reward kernel `(K, W)` that the certificate's own numbers determine for its own strategy. Nothing here assumes that the certificate has been checked; `Check.lean` compares the recorded result with these values and `Sound.lean` proves that, on an accepted certificate, they are the values of `interpret σ f` from `Interpretation.lean`.

Cell layout. A table with axes of lengths $d_1, \ldots, d_k$ is a flat list of cells in lexicographic order, the last axis fastest. `flatIndex` takes the list of pairs (d_i, a_i) of axis lengths and 0-based coordinates and returns $a_1 (d_2 \cdots d_k) + \cdots + a_k$; `flatIndex_lt` bounds it by the product of the lengths, `flatIndex_append` splits off the last axis (so a chance table is a list of rows, one per parent coordinate) and `flatIndex_injective` shows that distinct in-range coordinates have distinct cells. A missing cell reads as `0` (`cellQ`); the checker requires every table to have exactly the product of its axis lengths as its cell count. The axis length of a variable is CategoricalBayesianNetworks' `dimension` of its attributes, which is its state count on a checked network (`dimension_checked`).

The enumeration. Fix 0-based states `x` of the input rows and `z` of the generators (all generators, chance and decision). The value of a variable (`value`) is `z g` if `g` is its generator, otherwise `x i` if it is the variable of input row `i`. Then:

- a decision generator weighs `1` if its state is the action its policy (looked up by its decision name in the strategy) picks at the values of its information inputs, and `0` otherwise (`actionOf`; an action out of range reads as the first state, which the checker excludes);
- a chance generator weighs its own table's cell at its parents' values and its own state (`tableOf`, `chanceAt`);
- the reward is the sum over the utilities of each utility's cell at its scope's values;
- `exactK x y` sums the product of the weights over all `z` whose output rows read `y`, and `exactW x y` sums that product times the reward.

The result coordinate of `(x, y)` is `resultIndex x y`, the flat index of the input rows' coordinates followed by the

output rows'. The largest result errors (`kError`, `wError`) and the largest deviation of a chance row sum from one (`rowDeviation`) are computed here too.

```
set_option autoImplicit false

namespace OpenInfluence.RewardCertificate

open OpenNet.RawCertificate
open InfluenceDiagramsProofs.DVECertificate (Value.toRat)
```

Flat indices

```
/-- The lexicographic (last axis fastest) index of the coordinates `aᵢ` on axes of lengths
`dᵢ`, given as the pairs `(dᵢ, aᵢ)`. -/
def flatIndex : List (ℕ × ℕ) → ℕ
  | [] ⇒ 0
  | (_, a) :: rest ⇒ a * (rest.map Prod.fst).prod + flatIndex rest

theorem flatIndex_lt : ∀ l : List (ℕ × ℕ), (∀ p ∈ l, p.2 < p.1) →
    flatIndex l < (l.map Prod.fst).prod
  | [], _ ⇒ by simp [flatIndex]
  | (d, a) :: rest, h ⇒ by
    have ha : a < d := h (d, a) List.mem_cons_self
    have hr := flatIndex_lt rest (fun p hp ⇒ h p (List.mem_cons_of_mem _ hp))
    simp only [flatIndex, List.map_cons, List.prod_cons]
    calc a * (rest.map Prod.fst).prod + flatIndex rest
        < a * (rest.map Prod.fst).prod + (rest.map Prod.fst).prod := by omega
      _ = (a + 1) * (rest.map Prod.fst).prod := by ring
      _ ≤ d * (rest.map Prod.fst).prod := Nat.mul_le_mul_right _ ha

theorem flatIndex_append (l : List (ℕ × ℕ)) (d s : ℕ) :
    flatIndex (l ++ [(d, s)]) = flatIndex l * d + s := by
  induction l with
  | nil ⇒ simp [flatIndex]
  | cons p rest ih ⇒
    obtain ⟨d', a⟩ := p
    simp only [List.cons_append, flatIndex, List.map_append, List.map_cons, List.map_nil,
      List.prod_append, List.prod_cons, List.prod_nil, mul_one, ih]
    ring

theorem flatIndex_injective : ∀ l l' : List (ℕ × ℕ), l.map Prod.fst = l'.map Prod.fst →
    (∀ p ∈ l, p.2 < p.1) → (∀ p ∈ l', p.2 < p.1) → flatIndex l = flatIndex l' → l = l'
  | [], [], _, _, _, _ ⇒ rfl
  | [], _ :: _, h, _, _, _ ⇒ by simp at h
  | _ :: _, [], h, _, _, _ ⇒ by simp at h
  | (d, a) :: rest, (d', a') :: rest', hd, h, h', he ⇒ by
    simp only [List.map_cons, List.cons.injEq] at hd
    obtain ⟨rfl, hd⟩ := hd
    have hr := flatIndex_lt rest (fun p hp ⇒ h p (List.mem_cons_of_mem _ hp))
    have hr' := flatIndex_lt rest' (fun p hp ⇒ h' p (List.mem_cons_of_mem _ hp))
    rw [hd] at hr
    simp only [flatIndex, hd] at he
    set P := (rest'.map Prod.fst).prod
    have hP : 0 < P := by omega
    have hq : a = a' := by
      have e1 : (a * P + flatIndex rest) / P = a := by
        rw [Nat.add_comm, Nat.add_mul_div_right _ _ hP, Nat.div_eq_of_lt hr, zero_add]
      have e2 : (a' * P + flatIndex rest') / P = a' := by
        rw [Nat.add_comm, Nat.add_mul_div_right _ _ hP, Nat.div_eq_of_lt hr', zero_add]
      rw [← e1, ← e2, he]
    subst hq
    have hrest : flatIndex rest = flatIndex rest' := by omega
    rw [flatIndex_injective rest rest' hd (fun p hp ⇒ h p (List.mem_cons_of_mem _ hp))
      (fun p hp ⇒ h' p (List.mem_cons_of_mem _ hp)) hrest]
```

```
/-- The exact value of cell `k`, or `0` when there is none. -/
def cellQ (cells : List Value) (k : ℕ) : ℚ := ((cells[k]?).map Value.toRat).getD 0

theorem cellQ_nonneg {cells : List Value} (h : ∀ v ∈ cells, 0 ≤ v.toRat) (k : ℕ) :
    0 ≤ cellQ cells k := by
  unfold cellQ
  cases hk : cells[k]? with
  | none ⇒ simp
  | some v ⇒ simpa using h v (List.mem_of_getElem? hk)
```

Labels, tables and policies

```
/-- The axes of a table on the inputs of a label: their lengths paired with the values. -/
def inputAxes (l : LabelAttrs) (vals : List ℕ) : List (ℕ × ℕ) :=
  (l.inputs.map dimension).zip vals

/-- The number of parent rows of a chance table on `l`. -/
def rowCount (l : LabelAttrs) : ℕ := (l.inputs.map dimension).prod

/-- The cell of a chance table on `l` at the parent values `vals` and child state `s`. -/
def chanceAt (cells : List Value) (l : LabelAttrs) (vals : List ℕ) (s : ℕ) : ℚ :=
  cellQ cells (flatIndex (inputAxes l vals ++ [(dimension l.result, s)]))

theorem chanceAt_row (cells : List Value) (l : LabelAttrs) (vals : List ℕ) (s : ℕ) :
    chanceAt cells l vals s =
      cellQ cells (flatIndex (inputAxes l vals) * dimension l.result + s) := by
  unfold chanceAt
  rw [flatIndex_append]

/-- The cell of a utility table on `l` at the scope values `vals`. -/
def utilityAt (cells : List Value) (l : ULabel) (vals : List ℕ) : ℚ :=
  cellQ cells (flatIndex ((l.inputs.map dimension).zip vals))

namespace Certificate

variable (c : Certificate)

/-- A generator index. -/
abbrev Gen := Fin c.network.mechanisms.length

/-- A utility index. -/
abbrev UIdx := Fin c.utilities.length

/-- Whether generator `g` is a decision: its reference is a `policy` reference. -/
def isDecisionGen (g : c.Gen) : Bool := isPolicy (c.network.mechanism g).kernelRef

/-- The table of a generator: the cells of the first chance-table record that names it. -/
def tableOf (g : c.Gen) : List Value :=
  ((c.chanceTables.find? fun t ⇒ decide (t.mechanism = g.val)).map ChanceTable.cells).getD []

/-- The policy record of a decision name. -/
def policyOf (name : String) : Option Policy :=
  c.strategy.find? fun p ⇒ decide (p.decision = name)

/-- The raw (1-based) action that the policy of `l`'s decision name records at the
information values `vals`, or `0` when there is none. -/
def rawAction (l : LabelAttrs) (vals : List ℕ) : ℕ :=
  match policyName? l.kernelRef with
  | some name ⇒
    match c.policyOf name with
    | some p ⇒ p.actions.getD (flatIndex (inputAxes l vals)) 0
    | none ⇒ 0
  | none ⇒ 0
```

```
/-- The 0-based action state of a decision with label `l` at the information values `vals`;
an out-of-range action reads as `0` (the checker excludes it). -/
def actionOf (l : LabelAttrs) (vals : List ℕ) : ℕ :=
  if 1 ≤ c.rawAction l vals ∧ c.rawAction l vals ≤ dimension l.result then
    c.rawAction l vals - 1
  else 0

theorem actionOf_lt (l : LabelAttrs) (vals : List ℕ) : c.actionOf l vals < dimension l.result := by
  unfold actionOf
  have hpos : 0 < dimension l.result := lt_of_lt_of_le Nat.zero_lt_one (le_max_left _ _)
  split_ifs with h
  · omega
  · exact hpos

/-- The scope rows of utility `u`, in position order. -/
def scopeRows (u : c.UIdx) : List (InputRow c.network.variableCount) :=
  orderRows InputRow.position c.utilities[u].scopeRows

/-- The scope variables of utility `u`, in position order. -/
def scope (u : c.UIdx) : List (Fin c.network.variableCount) :=
  (c.scopeRows u).map InputRow.varId

/-- The label of utility `u`: its name and its scope's attributes. -/
def utilityLabel (u : c.UIdx) : ULabel :=
  ⟨c.utilities[u].name, (c.scope u).map c.network.attrs⟩
```

Coordinates

```
/-- 0-based states of the input rows. -/
abbrev InCoord := ∀ i : Fin c.network.inputs.length, Fin (dimension c.network.inputs[i].attrs)

/-- 0-based states of the output rows. -/
abbrev OutCoord :=
  ∀ j : Fin c.network.outputs.length, Fin (dimension c.network.outputs[j].attrs)

/-- 0-based states of the generators (the generated variables). -/
abbrev GenCoord := ∀ g : c.Gen, Fin (dimension (c.network.label g).result)

/-- The generator of a variable, if any. -/
def genOf (v : Fin c.network.variableCount) : Option c.Gen :=
  (List.finRange _).find? fun g ⇒ decide (c.network.target g = v)

/-- The input row of a variable, if any. -/
def inputOf (v : Fin c.network.variableCount) : Option (Fin c.network.inputs.length) :=
  (List.finRange _).find? fun i ⇒ decide (c.network.input i = v)

/-- The 0-based state of variable `v`: its generator's state, else its input row's. -/
def value (x : c.InCoord) (z : c.GenCoord) (v : Fin c.network.variableCount) : ℕ :=
  match c.genOf v with
  | some g ⇒ (z g).val
  | none ⇒
    match c.inputOf v with
    | some i ⇒ (x i).val
    | none ⇒ 0
```

The enumeration

```
/-- The weight of generator `g` at `(x, z)`. -/
def genWeight (x : c.InCoord) (z : c.GenCoord) (g : c.Gen) : ℚ :=
  if c.isDecisionGen g then
    (if (z g).val = c.actionOf (c.network.label g) ((c.network.parents g).map (c.value x z))
      then 1 else 0)
  else chanceAt (c.tableOf g) (c.network.label g) ((c.network.parents g).map (c.value x z))
```

```
    (z g).val

/-- The product of the generator weights. -/
def joint (x : c.InCoord) (z : c.GenCoord) : ℚ := ∏ g, c.genWeight x z g

/-- The total utility at `(x, z)`. -/
def reward (x : c.InCoord) (z : c.GenCoord) : ℚ :=
  ∑ u : c.UIdx, utilityAt c.utilities[u].cells (c.utilityLabel u) ((c.scope u).map (c.value x z))

/-- The output rows read `y` at `(x, z)`. -/
def Reads (x : c.InCoord) (z : c.GenCoord) (y : c.OutCoord) : Prop :=
  ∀ j, (y j).val = c.value x z (c.network.output j)

instance (x : c.InCoord) (z : c.GenCoord) (y : c.OutCoord) : Decidable (c.Reads x z y) := by
  unfold Reads
  infer_instance

/-- The exact probability component at `(x, y)`. -/
def exactK (x : c.InCoord) (y : c.OutCoord) : ℚ :=
  ∑ z : c.GenCoord, if c.Reads x z y then c.joint x z else 0

/-- The exact reward component at `(x, y)`. -/
def exactW (x : c.InCoord) (y : c.OutCoord) : ℚ :=
  ∑ z : c.GenCoord, if c.Reads x z y then c.joint x z * c.reward x z else 0
```

Result coordinates

```
/-- The axis lengths of the input rows, then of the output rows. -/
def resultDims : List ℕ :=
  c.network.inputs.map (fun r ⇒ dimension r.attrs) ++
    c.network.outputs.map (fun r ⇒ dimension r.attrs)

/-- The result axes at `(x, y)`: the input rows' lengths and states, then the output rows'. -/
def resultAxes (x : c.InCoord) (y : c.OutCoord) : List (ℕ × ℕ) :=
  List.ofFn (fun i ⇒ (dimension c.network.inputs[i].attrs, (x i).val)) ++
    List.ofFn (fun j ⇒ (dimension c.network.outputs[j].attrs, (y j).val))

/-- The cell of `(x, y)` in the recorded `K` and `W`. -/
def resultIndex (x : c.InCoord) (y : c.OutCoord) : ℕ := flatIndex (c.resultAxes x y)

theorem resultAxes_fst (x : c.InCoord) (y : c.OutCoord) :
    (c.resultAxes x y).map Prod.fst = c.resultDims := by
  unfold resultAxes resultDims
  rw [List.map_append, List.map_ofFn, List.map_ofFn]
  congr 1
  · exact List.ext_get (by simp) (fun k h1 h2 ⇒ by simp)
  · exact List.ext_get (by simp) (fun k h1 h2 ⇒ by simp)

theorem resultAxes_lt (x : c.InCoord) (y : c.OutCoord) :
    ∀ p ∈ c.resultAxes x y, p.2 < p.1 := by
  intro p hp
  unfold resultAxes at hp
  rcases List.mem_append.1 hp with hp | hp <;>
    obtain ⟨k, rfl⟩ := List.mem_ofFn.1 hp <;> exact Fin.isLt _

theorem resultIndex_lt (x : c.InCoord) (y : c.OutCoord) :
    c.resultIndex x y < c.resultDims.prod := by
  rw [← c.resultAxes_fst x y]
  exact flatIndex_lt _ (c.resultAxes_lt x y)

/-- Distinct boundary coordinates have distinct result cells. -/
theorem resultIndex_injective {x x' : c.InCoord} {y y' : c.OutCoord}
    (h : c.resultIndex x y = c.resultIndex x' y') : x = x' ∧ y = y' := by
  have he := flatIndex_injective _ _ ((c.resultAxes_fst x y).trans (c.resultAxes_fst x' y').symm)
```

```
    (c.resultAxes_lt x y) (c.resultAxes_lt x' y') h
  unfold resultAxes at he
  have hl : (List.ofFn fun i ⇒ (dimension c.network.inputs[i].attrs, (x i).val)).length =
      (List.ofFn fun i ⇒ (dimension c.network.inputs[i].attrs, (x' i).val)).length := by simp
  obtain ⟨h1, h2⟩ := List.append_inj he hl
  refine ⟨funext fun i ⇒ Fin.ext ?_, funext fun j ⇒ Fin.ext ?_⟩
  · have := congrArg (fun l ⇒ l[i.val]?) h1
    simpa using this
  · have := congrArg (fun l ⇒ l[j.val]?) h2
    simpa using this

/-- The result cell of a pair of coordinates. -/
def resultCell (p : c.InCoord × c.OutCoord) : Fin c.resultDims.prod :=
  ⟨c.resultIndex p.1 p.2, c.resultIndex_lt p.1 p.2⟩

theorem card_coords : Fintype.card (c.InCoord × c.OutCoord) = c.resultDims.prod := by
  rw [Fintype.card_prod, Fintype.card_pi, Fintype.card_pi]
  simp only [Fintype.card_fin]
  unfold resultDims
  rw [List.prod_append, ← List.ofFn_getElem_eq_map, ← List.ofFn_getElem_eq_map, List.prod_ofFn,
    List.prod_ofFn]
  rfl

/-- **The cell layout is a bijection** between pairs of boundary coordinates and the cells
of a table with the result's axes. -/
theorem resultCell_bijective : Function.Bijective c.resultCell :=
  (Fintype.bijective_iff_injective_and_card _).2
    ⟨fun p q e ⇒ by
      obtain ⟨h1, h2⟩ := c.resultIndex_injective (congrArg Fin.val e)
      exact Prod.ext h1 h2, by rw [card_coords, Fintype.card_fin]⟩
```

Diagnostics

```
/-- The sum of row `r` of generator `g`'s chance table. -/
def rowSum (g : c.Gen) (r : ℕ) : ℚ :=
  ∑ s ∈ Finset.range (dimension (c.network.label g).result),
    cellQ (c.tableOf g) (r * dimension (c.network.label g).result + s)

/-- The largest `|row sum - 1|` over the chance generators' rows. -/
def rowDeviation : ℚ :=
  (Finset.univ.filter fun g ⇒ c.isDecisionGen g = false).fold max 0 fun g ⇒
    (Finset.range (rowCount (c.network.label g))).fold max 0 fun r ⇒ |c.rowSum g r - 1|

/-- The largest error of the recorded `K`. -/
def kError : ℚ :=
  (Finset.univ : Finset (c.InCoord × c.OutCoord)).fold max 0 fun p ⇒
    |cellQ c.result.K (c.resultIndex p.1 p.2) - c.exactK p.1 p.2|

/-- The largest error of the recorded `W`. -/
def wError : ℚ :=
  (Finset.univ : Finset (c.InCoord × c.OutCoord)).fold max 0 fun p ⇒
    |cellQ c.result.W (c.resultIndex p.1 p.2) - c.exactW p.1 p.2|

theorem kError_le {τ : ℚ} (h : c.kError ≤ τ) (x : c.InCoord) (y : c.OutCoord) :
    |cellQ c.result.K (c.resultIndex x y) - c.exactK x y| ≤ τ :=
  ((Finset.fold_max_le _).1 h).2 (x, y) (Finset.mem_univ _)

theorem wError_le {τ : ℚ} (h : c.wError ≤ τ) (x : c.InCoord) (y : c.OutCoord) :
    |cellQ c.result.W (c.resultIndex x y) - c.exactW x y| ≤ τ :=
  ((Finset.fold_max_le _).1 h).2 (x, y) (Finset.mem_univ _)

end Certificate

end OpenInfluence.RewardCertificate
```

Reward certificates: a faster exact check

```
import CategoricalInfluenceDiagramsProofs.Certificate.Exact
```

`exactK x y` and `exactW x y` (`Exact.lean`) each sum over every generator coordinate `z`, so checking all recorded cells costs `|X| · |Y| · |Z|` weight evaluations, for the input, output and generator coordinates. But the output coordinate is a function of `(x, z)`: at most one `y` is read. This module accumulates, for each input coordinate `x`, the whole row of `K` and `W` in one pass over `z` (`accRow`), into an array indexed by the flat index of the output coordinate (`outIndex`), so all cells cost `|X| · |Z|` evaluations.

- `readOut? x z` is the output coordinate read at `(x, z)`, or `none` when some output row's value is out of range; `Reads x z y` holds exactly when it is `some y` (`reads_iff`).
- `accRow_getD`: the cell of `accRow x` at `outIndex y` is `(exactK x y, exactW x y)`. The accumulation is a `Multiset.foldl` over the generator coordinates, whose step commutes, so no enumeration order is assumed.
- `kErrorFast`, `wErrorFast` take the largest error row by row, and are within `τ` exactly when `kError`, `wError` are (`kErrorFast_le_iff`, `wErrorFast_le_iff`). The checker uses them (`Check.lean`), so acceptance, and every soundness theorem, is unchanged; they are in fact equal to `kError`, `wError` (`kErrorFast_eq`, `wErrorFast_eq`), and the command line reports them.

```
set_option autoImplicit false

namespace OpenInfluence.RewardCertificate

open OpenNet.RawCertificate
open InfluenceDiagramsProofs.DVECertificate (Value.toRat)
```

Accumulating into an array

```
/-- Add `w` to cell `k` of `a`, if there is one. -/
def addAt (a : Array (ℚ × ℚ)) (k : Option ℕ) (w : ℚ × ℚ) : Array (ℚ × ℚ) :=
  match k with
  | some i ⇒ a.modify i (· + w)
  | none ⇒ a

theorem size_addAt (a : Array (ℚ × ℚ)) (k : Option ℕ) (w : ℚ × ℚ) :
    (addAt a k w).size = a.size := by
  cases k <;> simp [addAt]

theorem getD_addAt (a : Array (ℚ × ℚ)) (k : Option ℕ) (w : ℚ × ℚ) (j : ℕ) (hj : j < a.size) :
    (addAt a k w).getD j 0 = a.getD j 0 + if k = some j then w else 0 := by
  cases k with
  | none ⇒ simp [addAt]
  | some i ⇒
    simp only [addAt, Array.getD_eq_getD_getElem?, Array.getElem?_modify, Option.some.injEq]
    by_cases h : i = j
    · subst h
      simp [Array.getElem?_eq_getElem hj]
    · simp [h]

theorem addAt_comm (a : Array (ℚ × ℚ)) (k k' : Option ℕ) (w w' : ℚ × ℚ) :
    addAt (addAt a k w) k' w' = addAt (addAt a k' w') k w := by
  apply Array.ext
  · simp [size_addAt]
  · intro j h1 h2
    have hj : j < a.size := by simpa [size_addAt] using h1
    have e1 := getD_addAt (addAt a k w) k' w' j (by simpa [size_addAt] using hj)
    have e2 := getD_addAt (addAt a k' w') k w j (by simpa [size_addAt] using hj)
    rw [getD_addAt a k w j hj] at e1
    rw [getD_addAt a k' w' j hj] at e2
    simp only [Array.getD_eq_getD_getElem?, Array.getElem?_eq_getElem h1,
      Array.getElem?_eq_getElem h2, Option.getD_some] at e1 e2
    rw [e1, e2]
    abel
```

```
/-- One accumulation step: add `w z` into cell `key z`. -/
def accStep {α : Type} (key : α → Option ℕ) (w : α → ℚ × ℚ) (b : Array (ℚ × ℚ)) (z : α) :
    Array (ℚ × ℚ) :=
  addAt b (key z) (w z)

instance {α : Type} (key : α → Option ℕ) (w : α → ℚ × ℚ) : RightCommutative (accStep key w) :=
  ⟨fun b z z' ⇒ addAt_comm b (key z) (key z') (w z) (w z')⟩

/-- Accumulate `w z` into cell `key z` over a multiset of `z`; the steps commute, so no order
of the multiset is assumed. -/
def accumulate {α : Type} (key : α → Option ℕ) (w : α → ℚ × ℚ) (a : Array (ℚ × ℚ))
    (s : Multiset α) : Array (ℚ × ℚ) :=
  s.foldl (accStep key w) a

theorem size_accumulate {α : Type} (key : α → Option ℕ) (w : α → ℚ × ℚ) (s : Multiset α) :
    ∀ a : Array (ℚ × ℚ), (accumulate key w a s).size = a.size := by
  induction s using Multiset.induction_on with
  | empty ⇒ intro a; simp [accumulate]
  | cons z s ih ⇒
    intro a
    simp only [accumulate, Multiset.foldl_cons] at ih ⊢
    rw [ih, accStep, size_addAt]

theorem getD_accumulate {α : Type} (key : α → Option ℕ) (w : α → ℚ × ℚ) (s : Multiset α)
    (j : ℕ) : ∀ a : Array (ℚ × ℚ), j < a.size →
    (accumulate key w a s).getD j 0 =
      a.getD j 0 + (s.map fun z ⇒ if key z = some j then w z else 0).sum := by
  induction s using Multiset.induction_on with
  | empty ⇒ intro a _; simp [accumulate]
  | cons z s ih ⇒
    intro a hj
    simp only [accumulate, Multiset.foldl_cons, Multiset.map_cons, Multiset.sum_cons] at ih ⊢
    rw [ih _ (by rwa [accStep, size_addAt]), accStep, getD_addAt a (key z) (w z) j hj]
    abel

namespace Certificate

variable (c : Certificate)
```

The output coordinate read at `(x, z)`

```
/-- The output coordinate read at `(x, z)`, if every output row's value is in range. -/
def readOut? (x : c.InCoord) (z : c.GenCoord) : Option c.OutCoord :=
  if h : ∀ j : Fin c.network.outputs.length,
      c.value x z (c.network.output j) < dimension c.network.outputs[j].attrs then
    some fun j ⇒ ⟨c.value x z (c.network.output j), h j⟩
  else none

theorem reads_iff (x : c.InCoord) (z : c.GenCoord) (y : c.OutCoord) :
    c.Reads x z y ↔ c.readOut? x z = some y := by
  unfold Reads readOut?
  constructor
  · intro h
    have hr : ∀ j : Fin c.network.outputs.length,
        c.value x z (c.network.output j) < dimension c.network.outputs[j].attrs :=
      fun j ⇒ h j ▸ (y j).isLt
    rw [dif_pos hr]
    exact congrArg some (funext fun j ⇒ Fin.ext (h j).symm)
  · intro h j
    by_cases hr : ∀ j : Fin c.network.outputs.length,
        c.value x z (c.network.output j) < dimension c.network.outputs[j].attrs
    · rw [dif_pos hr] at h
      rw [← Option.some.inj h]
```

```
    · rw [dif_neg hr] at h
      exact absurd h (by simp)

/-- The flat index of an output coordinate (last row fastest). -/
def outIndex (y : c.OutCoord) : ℕ :=
  flatIndex (List.ofFn fun j ⇒ (dimension c.network.outputs[j].attrs, (y j).val))

/-- The number of output coordinates. -/
def outCount : ℕ := (c.network.outputs.map fun r ⇒ dimension r.attrs).prod

theorem outAxes_fst (y : c.OutCoord) :
    (List.ofFn fun j ⇒ (dimension c.network.outputs[j].attrs, (y j).val)).map Prod.fst =
      c.network.outputs.map fun r ⇒ dimension r.attrs :=
  List.ext_get (by simp) (fun k h1 h2 ⇒ by simp)

theorem outIndex_lt (y : c.OutCoord) : c.outIndex y < c.outCount := by
  unfold outIndex outCount
  rw [← c.outAxes_fst y]
  refine flatIndex_lt _ fun p hp ⇒ ?_
  obtain ⟨k, rfl⟩ := List.mem_ofFn.1 hp
  exact Fin.isLt _

theorem outIndex_injective {y y' : c.OutCoord} (h : c.outIndex y = c.outIndex y') : y = y' := by
  have he := flatIndex_injective _ _ ((c.outAxes_fst y).trans (c.outAxes_fst y').symm)
    (fun p hp ⇒ by obtain ⟨k, rfl⟩ := List.mem_ofFn.1 hp; exact Fin.isLt _)
    (fun p hp ⇒ by obtain ⟨k, rfl⟩ := List.mem_ofFn.1 hp; exact Fin.isLt _) h
  funext j
  have := congrArg (fun l ⇒ l[j.val]?) he
  exact Fin.ext (by simpa using this)
```

A row of `K` and `W` in one pass

```
/-- The row of `(K, W)` at the input coordinate `x`, accumulated in one pass over the
generator coordinates. -/
def accRow (x : c.InCoord) : Array (ℚ × ℚ) :=
  accumulate (fun z ⇒ (c.readOut? x z).map c.outIndex)
    (fun z ⇒ (c.joint x z, c.joint x z * c.reward x z)) (Array.replicate c.outCount 0)
    (Finset.univ : Finset c.GenCoord).val

theorem key_eq_iff (x : c.InCoord) (z : c.GenCoord) (y : c.OutCoord) :
    (c.readOut? x z).map c.outIndex = some (c.outIndex y) ↔ c.Reads x z y := by
  rw [c.reads_iff]
  constructor
  · intro h
    obtain ⟨y', hy', he⟩ := Option.map_eq_some_iff.1 h
    rw [hy', c.outIndex_injective he]
  · intro h
    rw [h]
    rfl

/-- **One pass gives the row**: the cell of `accRow x` at `outIndex y` is
`(exactK x y, exactW x y)`. -/
theorem accRow_getD (x : c.InCoord) (y : c.OutCoord) :
    (c.accRow x).getD (c.outIndex y) 0 = (c.exactK x y, c.exactW x y) := by
  unfold accRow
  rw [getD_accumulate _ _ _ _ _ (by simpa using c.outIndex_lt y)]
  have h0 : (Array.replicate c.outCount (0 : ℚ × ℚ)).getD (c.outIndex y) 0 = 0 := by
    simp [Array.getD_eq_getD_getElem?, c.outIndex_lt y]
  rw [h0, zero_add]
  change (Multiset.map _ Finset.univ.val).sum = _
  rw [← Finset.sum_eq_multiset_sum]
  simp only [c.key_eq_iff]
  ext
  · simp only [Prod.fst_sum, apply_ite Prod.fst, Prod.fst_zero, exactK]
```

```
  · simp only [Prod.snd_sum, apply_ite Prod.snd, Prod.snd_zero, exactW]
```

The errors, row by row

```
/-- The largest error of the recorded `K`, computed row by row. -/
def kErrorFast : ℚ :=
  (Finset.univ : Finset c.InCoord).fold max 0 fun x ⇒
    let row := c.accRow x
    (Finset.univ : Finset c.OutCoord).fold max 0 fun y ⇒
      |cellQ c.result.K (c.resultIndex x y) - (row.getD (c.outIndex y) 0).1|

/-- The largest error of the recorded `W`, computed row by row. -/
def wErrorFast : ℚ :=
  (Finset.univ : Finset c.InCoord).fold max 0 fun x ⇒
    let row := c.accRow x
    (Finset.univ : Finset c.OutCoord).fold max 0 fun y ⇒
      |cellQ c.result.W (c.resultIndex x y) - (row.getD (c.outIndex y) 0).2|

theorem kErrorFast_le_iff (τ : ℚ) : c.kErrorFast ≤ τ ↔ c.kError ≤ τ := by
  unfold kErrorFast kError
  simp only [Finset.fold_max_le, Finset.mem_univ, true_implies, c.accRow_getD, Prod.forall]
  constructor
  · rintro ⟨h0, h⟩
    exact ⟨h0, fun x y ⇒ (h x).2 y⟩
  · rintro ⟨h0, h⟩
    exact ⟨h0, fun x ⇒ ⟨h0, fun y ⇒ h x y⟩⟩

theorem wErrorFast_le_iff (τ : ℚ) : c.wErrorFast ≤ τ ↔ c.wError ≤ τ := by
  unfold wErrorFast wError
  simp only [Finset.fold_max_le, Finset.mem_univ, true_implies, c.accRow_getD, Prod.forall]
  constructor
  · rintro ⟨h0, h⟩
    exact ⟨h0, fun x y ⇒ (h x).2 y⟩
  · rintro ⟨h0, h⟩
    exact ⟨h0, fun x ⇒ ⟨h0, fun y ⇒ h x y⟩⟩

/-- The fast error of `K` is the error of `K`. -/
theorem kErrorFast_eq : c.kErrorFast = c.kError :=
  le_antisymm ((c.kErrorFast_le_iff _).2 le_rfl) ((c.kErrorFast_le_iff _).1 le_rfl)

/-- The fast error of `W` is the error of `W`. -/
theorem wErrorFast_eq : c.wErrorFast = c.wError :=
  le_antisymm ((c.wErrorFast_le_iff _).2 le_rfl) ((c.wErrorFast_le_iff _).1 le_rfl)

end Certificate

end OpenInfluence.RewardCertificate
```

Reward certificates: the checker

```
import CategoricalInfluenceDiagramsProofs.Certificate.Fast
```

`check τ c` decides whether the decoded certificate `c` is accepted with tolerance `τ`. It runs a fixed list of stages (`stages`) in order and accepts when every stage passes; `firstFailure` names the first stage that fails (`check_iff_firstFailure`), and `check_iff_accepted` states acceptance as the conjunction `Accepted` of the stage properties:

1. **network**: CategoricalBayesianNetworks' `Network.check`, unchanged (positions, states, the input/target partition, attributed boundary rows, ranks);
2. **flag counts**: one flag per input row and per output row;
3. **decision names**: a decision generator's reference is `policy` of its own name;

4. **information locality** (rule 3 of ADR 0018): no decision generator has, among its information inputs, the variable of an input row whose flag is not `observed`;
5. **chance tables**: the records name exactly the chance generators, in generator order (so no decision generator has a chance table);
6. **utility scope positions**: each utility's scope positions are `1, …, k`;
7. **strategy names**: one policy record per distinct decision name, in order of first appearance among the decision generators;
8. **strategy shapes**: decision generators with equal names have equal information axis lengths and equal action counts;
9. **cell counts**: every chance table, utility table, policy and the recorded `K` and `W` have the product of their axis lengths as their number of cells;
10. **value shapes**: chance and utility cells are `{"f64"}` in `binary64_exact` mode and `{"q", "f64"}` in `rational_exact` mode, result cells are `{"f64"}`, and every value is finite, reduced, and its word the nearest-even rounding of its rational (InfluenceDiagrams' `ValueOk`);
11. **label consistency**: chance generators with equal labels have equal tables, and utilities with equal labels (name and scope attributes) equal tables; the decoded signature keys tables by label (`Sound.lean`), so this is what makes them well defined;
12. **chance nonnegativity**: every chance cell is `≥ 0`;
13. **actions in range**: every policy cell is a state position `1, …, n` of its decision's action variable;
14. **exact row sums**: every chance row sums to exactly `1` over `ℚ`. A binary64 model whose rows are not exactly normalised fails here: it is outside the exact semantics, and only the largest deviation (`rowDeviation`) is reported;
15. **result K**, 16. **result W**: every recorded cell is within `τ` of `exactK`, `exactW` (`kError ≤ τ`, `wError ≤ τ`). The checker decides them by `kErrorFast` and `wErrorFast` (`Fast.lean`), which accumulate each row of `K` and `W` in one pass over the generator coordinates and are within `τ` exactly when `kError` and `wError` are.

Stages 3, 5 to 9 and 13 check the format's conventions; the soundness theorems use stages 1, 2, 4, 11, 12, 14, 15 and 16. Not checked: that the chance mechanisms precede the decisions in `network.mechanisms` (the format's export order) and the metadata strings.

```
set_option autoImplicit false

namespace OpenInfluence.RewardCertificate

open OpenNet.RawCertificate
open InfluenceDiagramsProofs.DVECertificate (Value.toRat)

/-- The first occurrence of each element, in order (Julia's `unique`). -/
def firstOccurrences (l : List String) : List String :=
  l.foldl (fun acc x ⇒ if x ∈ acc then acc else acc ++ [x]) []

/-- A cell that is a binary64 word alone. -/
def Value.isF64 : Value → Bool
  | .f64 _ ⇒ true
  | _ ⇒ false

/-- A cell that is a rational with its binary64 word. -/
def Value.isQF64 : Value → Bool
  | .qf64 _ _ _ ⇒ true
  | _ ⇒ false

/-- The shape a chance or utility cell must have in a numeric mode. -/
def modeShape : NumericMode → Value → Bool
  | .binary64Exact, v ⇒ v.isF64
  | .rationalExact, v ⇒ v.isQF64

namespace Certificate

variable (c : Certificate) (τ : ℚ)

def FlagCounts : Prop :=
  c.inputFlags.length = c.network.inputs.length ∧ c.outputFlags.length = c.network.outputs.length

def DecisionNames : Prop :=
```

```
  ∀ g : c.Gen, c.isDecisionGen g = true →
    (c.network.mechanism g).kernelRef = .policy (c.network.mechanism g).name

/-- **Information locality** (rule 3 of ADR 0018, with the recorded flags): every
information input of a decision generator is either not the variable of an input row, or
the variable of an input row flagged `observed`. -/
def InformationLocal : Prop :=
  ∀ g : c.Gen, c.isDecisionGen g = true → ∀ v ∈ c.network.parents g,
    ∀ i : Fin c.network.inputs.length, c.network.input i = v →
      c.inputFlags[i.val]? = some .observed

/-- The chance generators, as 0-based indices in generator order. -/
def chanceGens : List ℕ :=
  ((List.finRange c.network.mechanisms.length).filter fun g ⇒ !c.isDecisionGen g).map Fin.val

def ChanceIndex : Prop := c.chanceTables.map ChanceTable.mechanism = c.chanceGens

def ScopePositions : Prop :=
  ∀ u : c.UIdx, (c.scopeRows u).map InputRow.position = List.range' 1 (c.scopeRows u).length

/-- The distinct decision names, in order of first appearance among the generators. -/
def decisionNameList : List String :=
  firstOccurrences ((List.finRange c.network.mechanisms.length).filterMap fun g ⇒
    policyName? (c.network.mechanism g).kernelRef)

def StrategyNames : Prop := c.strategy.map Policy.decision = c.decisionNameList

def StrategyShapes : Prop :=
  ∀ g g' : c.Gen, c.isDecisionGen g = true →
    policyName? (c.network.mechanism g).kernelRef = policyName? (c.network.mechanism g').kernelRef →
      (c.network.label g).inputs.map dimension = (c.network.label g').inputs.map dimension ∧
        dimension (c.network.label g).result = dimension (c.network.label g').result

/-- The first decision generator with a given name. -/
def firstDecision (name : String) : Option c.Gen :=
  (List.finRange _).find? fun g ⇒
    decide (policyName? (c.network.mechanism g).kernelRef = some name)

def CellCounts : Prop :=
  (∀ g : c.Gen, c.isDecisionGen g = false →
    (c.tableOf g).length = rowCount (c.network.label g) * dimension (c.network.label g).result) ∧
  (∀ u : c.UIdx, c.utilities[u].cells.length = ((c.utilityLabel u).inputs.map dimension).prod) ∧
  (∀ p ∈ c.strategy, ∃ g : c.Gen, c.firstDecision p.decision = some g ∧
    p.actions.length = rowCount (c.network.label g)) ∧
  c.result.K.length = c.resultDims.prod ∧ c.result.W.length = c.resultDims.prod

/-- Every chance and utility cell. -/
def tableValues : List Value :=
  c.chanceTables.flatMap ChanceTable.cells ++ c.utilities.flatMap Utility.cells

open InfluenceDiagramsProofs.DVECertificate.Spec (ValueOk) in
def ValuesOk : Prop :=
  (∀ v ∈ c.tableValues, modeShape c.numericMode v = true ∧ ValueOk v) ∧
  (∀ v ∈ c.result.K ++ c.result.W, v.isF64 = true ∧ ValueOk v)

def LabelsConsistent : Prop :=
  (∀ g g' : c.Gen, c.isDecisionGen g = false → c.network.label g = c.network.label g' →
    c.tableOf g = c.tableOf g') ∧
  (∀ u u' : c.UIdx, c.utilityLabel u = c.utilityLabel u' →
    c.utilities[u].cells = c.utilities[u'].cells)

def ChanceNonneg : Prop := ∀ t ∈ c.chanceTables, ∀ v ∈ t.cells, 0 ≤ v.toRat

def ActionsInRange : Prop :=
  ∀ p ∈ c.strategy, ∀ g : c.Gen, c.firstDecision p.decision = some g →
    ∀ a ∈ p.actions, 1 ≤ a ∧ a ≤ dimension (c.network.label g).result
```

```
def RowsExact : Prop :=
  ∀ g : c.Gen, c.isDecisionGen g = false → ∀ r < rowCount (c.network.label g), c.rowSum g r = 1

instance : Decidable c.FlagCounts := by unfold FlagCounts; infer_instance
instance : Decidable c.DecisionNames := by unfold DecisionNames; infer_instance
instance : Decidable c.InformationLocal := by unfold InformationLocal; infer_instance
instance : Decidable c.ChanceIndex := by unfold ChanceIndex; infer_instance
instance : Decidable c.ScopePositions := by unfold ScopePositions; infer_instance
instance : Decidable c.StrategyNames := by unfold StrategyNames; infer_instance
instance : Decidable c.StrategyShapes := by unfold StrategyShapes; infer_instance
instance : Decidable c.CellCounts := by unfold CellCounts; infer_instance
instance : Decidable c.ValuesOk := by unfold ValuesOk; infer_instance
instance : Decidable c.LabelsConsistent := by unfold LabelsConsistent; infer_instance
instance : Decidable c.ChanceNonneg := by unfold ChanceNonneg; infer_instance
instance : Decidable c.ActionsInRange := by unfold ActionsInRange; infer_instance
instance : Decidable c.RowsExact := by unfold RowsExact; infer_instance

/-- The stages of the checker, in order, each a name and a deferred decision. -/
def stages : List (String × (Unit → Bool)) :=
  [("network validity (Network.check: positions, states, partition, boundary rows, ranks)",
      fun _ ⇒ c.network.check),
    ("flag counts (one observability flag per boundary row)", fun _ ⇒ decide c.FlagCounts),
    ("decision names (a decision's reference is policy of its own name)",
      fun _ ⇒ decide c.DecisionNames),
    ("information locality (a decision reads a hidden input)",
      fun _ ⇒ decide c.InformationLocal),
    ("chance tables (one record per chance generator, in order)",
      fun _ ⇒ decide c.ChanceIndex),
    ("utility scope positions (1, ..., k)", fun _ ⇒ decide c.ScopePositions),
    ("strategy names (one policy per distinct decision name, in order)",
      fun _ ⇒ decide c.StrategyNames),
    ("strategy shapes (equally named decisions differ in shape)",
      fun _ ⇒ decide c.StrategyShapes),
    ("cell counts (a table's cells are not the product of its axis lengths)",
      fun _ ⇒ decide c.CellCounts),
    ("value shapes (numeric mode, finite words, companions)", fun _ ⇒ decide c.ValuesOk),
    ("label consistency (equal labels with different tables)",
      fun _ ⇒ decide c.LabelsConsistent),
    ("chance nonnegativity (a negative chance cell)", fun _ ⇒ decide c.ChanceNonneg),
    ("actions in range (a policy cell is not an action state position)",
      fun _ ⇒ decide c.ActionsInRange),
    ("exact row sums (a chance row does not sum to exactly 1: outside exact semantics)",
      fun _ ⇒ decide c.RowsExact),
    ("result K (a recorded K cell is not within the tolerance of the exact value)",
      fun _ ⇒ decide (c.kErrorFast ≤ τ)),
    ("result W (a recorded W cell is not within the tolerance of the exact value)",
      fun _ ⇒ decide (c.wErrorFast ≤ τ))]

/-- **The checker**: every stage passes. -/
def check : Bool := (c.stages τ).all fun s ⇒ s.2 ()

/-- The name of the first failing stage, if any. -/
def firstFailure : Option String := ((c.stages τ).find? fun s ⇒ !(s.2 ())).map Prod.fst

theorem check_iff_firstFailure : c.check τ = true ↔ c.firstFailure τ = none := by
  unfold check firstFailure
  rw [Option.map_eq_none_iff, List.find?_eq_none, List.all_eq_true]
  simp

/-- What acceptance means, stage by stage. -/
structure Accepted : Prop where
  valid : c.network.Valid
  flagCounts : c.FlagCounts
  decisionNames : c.DecisionNames
  informationLocal : c.InformationLocal
```

```
  chanceIndex : c.ChanceIndex
  scopePositions : c.ScopePositions
  strategyNames : c.StrategyNames
  strategyShapes : c.StrategyShapes
  cellCounts : c.CellCounts
  valuesOk : c.ValuesOk
  labelsConsistent : c.LabelsConsistent
  chanceNonneg : c.ChanceNonneg
  actionsInRange : c.ActionsInRange
  rowsExact : c.RowsExact
  kError : c.kError ≤ τ
  wError : c.wError ≤ τ

theorem check_iff_accepted : c.check τ = true ↔ c.Accepted τ := by
  unfold check stages
  simp only [List.all_cons, List.all_nil, Bool.and_true, Bool.and_eq_true, decide_eq_true_eq,
    kErrorFast_le_iff, wErrorFast_le_iff]
  constructor
  · rintro ⟨h1, h2, h3, h4, h5, h6, h7, h8, h9, h10, h11, h12, h13, h14, h15, h16⟩
    exact ⟨c.network.check_sound h1, h2, h3, h4, h5, h6, h7, h8, h9, h10, h11, h12, h13, h14,
      h15, h16⟩
  · rintro ⟨h1, h2, h3, h4, h5, h6, h7, h8, h9, h10, h11, h12, h13, h14, h15, h16⟩
    exact ⟨(Network.check_iff _).2 h1, h2, h3, h4, h5, h6, h7, h8, h9, h10, h11, h12, h13, h14,
      h15, h16⟩

/-- **Acceptance gives a valid network with information locality.** -/
theorem check_sound {c : Certificate} {τ : ℚ} (h : c.check τ = true) :
    c.network.Valid ∧ c.InformationLocal :=
  let a := (c.check_iff_accepted τ).1 h
  ⟨a.valid, a.informationLocal⟩

end Certificate

end OpenInfluence.RewardCertificate
```

Reward certificates: soundness

```
import CategoricalInfluenceDiagramsProofs.Certificate.Check
```

This module proves what acceptance by `check τ c` (`Check.lean`) means, in terms of the definitions of `Syntax.lean` and `Interpretation.lean` that the functoriality and contextual-identity theorems are stated for.

The decoded diagram.

- `sig` is one influence-diagram signature for every certificate: CategoricalBayesianNetworks' raw signature (types are variable attributes, labels are generator attributes), with a label a decision exactly when its reference is a `policy` reference, and with utility labels `ULabel` (a name and the scope's attributes).
- `diagram` is the `IDNet` of an accepted certificate: its net is `Network.normalize`, the checked net of CategoricalBayesianNetworks (what its `checkedNet` returns for a raw certificate), and its utility generators are the certificate's utilities, each reading its scope variables in position order.
- `model` is the `IDInterpretation`: the states of a type are `Fin (dimension t)`, as in CategoricalBayesianNetworks' `LocalTables` (on a valid network `dimension` is the state count, and state `k` is the state of position `k + 1`). The chance table of a label is the table of the first generator with that label, read at the parents' states and the child state (labels with no generator get the point mass at the first state); it is nonnegative because every chance cell is, and normalised because every chance row sums to exactly one (`chance_normalized` is where the exact row sums are needed). The utility of a label is the table of the first utility with that label.
- `strategyOf` is the `Strategy`: a decision label's policy is the policy record of its decision name, read at the information states (`actionOf`).

The theorems.

- `check_sound` (in `Check.lean`): acceptance gives a valid network and information locality; `diagram_informationLocal` restates information locality on the decoded `IDNet`: no decision gener-

ator reads an input port whose flag is not `observed`.

- `interpret_K_eq`, `interpret_W_eq`: on an accepted certificate (in fact under the four hypotheses nonnegativity, exact row sums, label consistency and network validity), for all 0-based states `x` of the input rows and `y` of the output rows, `interpret σ f` evaluated at the corresponding boundary assignments is exactly `exactK x y` and `exactW x y` (`Exact.lean`), the enumeration over the certificate's own tables.
- `reward_certificate_sound_K`, `reward_certificate_sound_W`: on an accepted certificate, for **every** pair of boundary assignments `x`, `y` of the decoded diagram, the recorded cell at `boundaryIndex x y` exists and is within `τ` of `(interpret σ f).K x y` (respectively `.W x y`). `boundaryIndex x y` is the flat index of the 0-based states `(x i).2` of the input rows followed by those `(y j).2` of the output rows (`boundaryIndex_eq`), and it is a bijection between pairs of boundary assignments and the recorded cells (`boundaryIndex_bijective`; `resultCell_bijective` in `Exact.lean` for the coordinates).

Not proved. That the certificate's diagram is the one Julia built, or that a Julia composite is the Lean composite of its parts (ADR 0019, decision 5): the theorems are about the decoded record. The reading of state `k` as the state of position `k + 1` is the format's convention, not a theorem. Binary64 certificates whose chance rows do not sum to exactly one are rejected, so nothing is claimed for them.

```
set_option autoImplicit false

noncomputable section

namespace OpenInfluence.RewardCertificate

open OpenNet OpenNet.RawCertificate
open InfluenceDiagramsProofs.DVECertificate (Value.toRat)

/-- The influence-diagram signature of every reward certificate. -/
abbrev sig : IDSignature where
  toSignature := RawCertificate.signature
  isDecision l := isPolicy l.kernelRef
  ULabel := ULabel
  uinputs := ULabel.inputs

/-- A typed value: a type (variable attributes) and a 0-based state. -/
abbrev TypedValue := Σ t : VariableAttrs, Fin (dimension t)

/-- The states of a list of typed values. -/
def vals (p : List TypedValue) : List ℕ := p.map fun s ⇒ s.2.val

theorem sigma_eq_iff {a b : TypedValue} (h : a.1 = b.1) : a = b ↔ a.2.val = b.2.val := by
  obtain ⟨t, s⟩ := a
  obtain ⟨t', s'⟩ := b
  simp only at h
  subst h
  simp [Fin.ext_iff]

theorem sigma_mk_eq_iff {t : VariableAttrs} (s s' : Fin (dimension t)) :
    (⟨t, s⟩ : TypedValue) = ⟨t, s'⟩ ↔ s.val = s'.val := by
  simp [Fin.ext_iff]

theorem zip_typed (p : List TypedValue) :
    ((p.map Sigma.fst).map dimension).zip (vals p) = p.map fun s ⇒ (dimension s.1, s.2.val) := by
  induction p with
  | nil ⇒ rfl
  | cons s p ih ⇒ simp only [vals, List.map_cons, List.zip_cons_cons] at ih ⊢; rw [ih]

theorem dimension_pos (t : VariableAttrs) : 0 < dimension t :=
  lt_of_lt_of_le Nat.zero_lt_one (le_max_left _ _)

theorem row_lt {l : LabelAttrs} {p : List TypedValue} (hp : p.map Sigma.fst = l.inputs) :
    flatIndex (inputAxes l (vals p)) < rowCount l := by
  unfold inputAxes rowCount
  rw [← hp, zip_typed]
  have h := flatIndex_lt (p.map fun s ⇒ (dimension s.1, s.2.val)) (by
    intro q hq
    obtain ⟨s, -, rfl⟩ := List.mem_map.1 hq
```

```
    exact s.2.isLt)
  simpa [List.map_map] using h

theorem find?_unique {α : Type} {l : List α} {p : α → Bool} {a : α} (ha : a ∈ l)
    (hp : p a = true) (hu : ∀ b ∈ l, p b = true → b = a) : l.find? p = some a := by
  cases h : l.find? p with
  | none ⇒
    rw [List.find?_eq_none] at h
    exact absurd hp (h a ha)
  | some b ⇒ rw [hu b (List.mem_of_find?_eq_some h) (List.find?_some h)]

namespace Certificate

variable (c : Certificate)
```

Tables by label

```
/-- The chance table of a label: the table of the first generator with that label. -/
def labelTable (l : LabelAttrs) : Option (List Value) :=
  ((List.finRange _).find? fun g ⇒ decide (c.network.label g = l)).map c.tableOf

/-- The chance cell of a label at the parent states `vs` and the child state `s`; the point
mass at the first state when no generator has the label. -/
def chanceEntry (l : LabelAttrs) (vs : List ℕ) (s : ℕ) : ℚ :=
  match c.labelTable l with
  | some cells ⇒ chanceAt cells l vs s
  | none ⇒ if s = 0 then 1 else 0

/-- The table of a utility label: the table of the first utility with that label. -/
def ulabelCells (l : ULabel) : Option (List Value) :=
  ((List.finRange _).find? fun u ⇒ decide (c.utilityLabel u = l)).map
    fun u ⇒ c.utilities[u].cells

/-- The utility of a label at the scope states `vs`; `0` when no utility has the label. -/
def utilityEntry (l : ULabel) (vs : List ℕ) : ℚ :=
  match c.ulabelCells l with
  | some cells ⇒ utilityAt cells l vs
  | none ⇒ 0

theorem labelTable_spec {l : LabelAttrs} {cells : List Value}
    (h : c.labelTable l = some cells) : ∃ g, c.network.label g = l ∧ c.tableOf g = cells := by
  unfold labelTable at h
  obtain ⟨g, hg, rfl⟩ := Option.map_eq_some_iff.1 h
  have hp := List.find?_some hg
  exact ⟨g, by simpa using hp, rfl⟩

theorem labelTable_label (hcons : c.LabelsConsistent) (g : c.Gen)
    (hg : c.isDecisionGen g = false) : c.labelTable (c.network.label g) = some (c.tableOf g) := by
  cases h : c.labelTable (c.network.label g) with
  | none ⇒
    exfalso
    unfold labelTable at h
    rw [Option.map_eq_none_iff, List.find?_eq_none] at h
    exact h g (List.mem_finRange g) (by simp)
  | some cells ⇒
    obtain ⟨g', hl, rfl⟩ := c.labelTable_spec h
    rw [hcons.1 g g' hg hl.symm]

theorem ulabelCells_label (hcons : c.LabelsConsistent) (u : c.UIdx) :
    c.ulabelCells (c.utilityLabel u) = some c.utilities[u].cells := by
  cases h : c.ulabelCells (c.utilityLabel u) with
  | none ⇒
    exfalso
    unfold ulabelCells at h
```

```
    rw [Option.map_eq_none_iff, List.find?_eq_none] at h
    exact h u (List.mem_finRange u) (by simp)
  | some cells ⇒
    unfold ulabelCells at h
    obtain ⟨u', hu', rfl⟩ := Option.map_eq_some_iff.1 h
    have hp := List.find?_some hu'
    have hp' : c.utilityLabel u' = c.utilityLabel u := by simpa using hp
    rw [hcons.2 u u' hp'.symm]

theorem utilityEntry_label (hcons : c.LabelsConsistent) (u : c.UIdx) (vs : List ℕ) :
    c.utilityEntry (c.utilityLabel u) vs =
      utilityAt c.utilities[u].cells (c.utilityLabel u) vs := by
  unfold utilityEntry
  rw [c.ulabelCells_label hcons u]

theorem tableOf_nonneg (hnn : c.ChanceNonneg) (g : c.Gen) : ∀ v ∈ c.tableOf g, 0 ≤ v.toRat := by
  unfold tableOf
  cases h : c.chanceTables.find? fun t ⇒ decide (t.mechanism = g.val) with
  | none ⇒ simp
  | some t ⇒ exact hnn t (List.mem_of_find?_eq_some h)

theorem chanceEntry_nonneg (hnn : c.ChanceNonneg) (l : LabelAttrs) (vs : List ℕ) (s : ℕ) :
    0 ≤ c.chanceEntry l vs s := by
  unfold chanceEntry
  cases h : c.labelTable l with
  | none ⇒ dsimp only; split_ifs <;> norm_num
  | some cells ⇒
    obtain ⟨g, -, rfl⟩ := c.labelTable_spec h
    exact cellQ_nonneg (c.tableOf_nonneg hnn g) _

theorem chanceEntry_normalized (hrows : c.RowsExact) (l : LabelAttrs) (p : List TypedValue)
    (hl : isPolicy l.kernelRef = false) (hp : p.map Sigma.fst = l.inputs) :
    ∑ s : Fin (dimension l.result), c.chanceEntry l (vals p) s.val = 1 := by
  unfold chanceEntry
  cases h : c.labelTable l with
  | none ⇒
    dsimp only
    rw [Fin.sum_univ_eq_sum_range (fun s ⇒ if s = 0 then (1 : ℚ) else 0)]
    rw [Finset.sum_ite_eq' (Finset.range _) 0 (fun _ ⇒ (1 : ℚ))]
    simp [dimension_pos]
  | some cells ⇒
    obtain ⟨g, rfl, rfl⟩ := c.labelTable_spec h
    dsimp only
    simp only [chanceAt_row]
    rw [Fin.sum_univ_eq_sum_range (fun s ⇒ cellQ (c.tableOf g)
      (flatIndex (inputAxes (c.network.label g) (vals p)) *
        dimension (c.network.label g).result + s))]
    exact hrows g hl _ (row_lt hp)
```

The interpretation, the strategy and the diagram

```
/-- The `IDInterpretation` of the certificate's tables. -/
def model (hnn : c.ChanceNonneg) (hrows : c.RowsExact) : IDInterpretation sig where
  states t := Fin (dimension t)
  finite _ := inferInstance
  chance := fun (l : LabelAttrs) (p : List TypedValue) (s : TypedValue) ⇒
    if s.1 = l.result then ((c.chanceEntry l (vals p) s.2.val : ℚ) : ℝ) else 0
  chance_nonneg := by
    intro l p s _
    change 0 ≤ (if s.1 = l.result then ((c.chanceEntry l (vals p) s.2.val : ℚ) : ℝ) else 0)
    split_ifs
    · exact_mod_cast c.chanceEntry_nonneg hnn l _ _
    · exact le_refl 0
  chance_normalized := by
```

```
    intro l p hl hp
    change ∑ s : Fin (dimension l.result), (if l.result = l.result then
      ((c.chanceEntry l (vals p) s.val : ℚ) : ℝ) else 0) = 1
    simp only [if_true]
    exact_mod_cast c.chanceEntry_normalized hrows l p hl hp
  utility := fun (l : ULabel) (p : List TypedValue) ⇒ ((c.utilityEntry l (vals p) : ℚ) : ℝ)

/-- The strategy of the certificate: each decision label reads the policy of its name. -/
def strategyOf (hnn : c.ChanceNonneg) (hrows : c.RowsExact) : (c.model hnn hrows).Strategy :=
  fun (l : LabelAttrs) _ (p : List TypedValue) ⇒
    (⟨c.actionOf l (vals p), c.actionOf_lt l (vals p)⟩ : Fin (dimension l.result))

/-- The decoded open influence diagram: CategoricalBayesianNetworks' checked net of the
certificate's network, with the certificate's utilities. -/
def diagram (hc : c.network.check = true) : IDNet (S := sig) c.network.dom c.network.cod where
  net := c.network.normalize hc
  UGen := c.UIdx
  ufinite := inferInstance
  ulabel := c.utilityLabel
  uparents u := (c.scope u).map (c.network.toLegged (c.network.check_sound hc)).varsEquiv.symm
  uparents_typed u := by
    rw [List.map_map]
    exact congrArg (fun k ⇒ (c.scope u).map k)
      (funext (c.network.toLegged (c.network.check_sound hc)).decode_typed)

/-- The input boundary with the recorded flags (a missing flag reads as hidden). -/
def domFlag (i : Fin c.network.inputs.length) : Observability :=
  (c.inputFlags[i.val]?).getD .hidden

/-- The decoded input interface with its observability flags. -/
def observedDom : ObservedInterface sig.toSignature := ⟨c.network.dom, c.domFlag⟩

/-- **Information locality on the decoded diagram**: if the network is valid and the
certificate's information locality holds, no decision generator of the decoded `IDNet`
reads an input port whose flag is not `observed`. -/
theorem diagram_informationLocal_of (hc : c.network.check = true) (hl : c.InformationLocal) :
    ∀ a : (c.diagram hc).net.Gen, (c.diagram hc).IsDecision a →
      ∀ p : c.network.dom.Port, Sum.inl p ∈ (c.diagram hc).info a →
        c.observedDom.flag p = .observed := by
  intro a ha p hp
  change Sum.inl p ∈ (c.network.parents a).map
    (c.network.toLegged (c.network.check_sound hc)).varsEquiv.symm at hp
  obtain ⟨v, hv, he⟩ := List.mem_map.1 hp
  have hvp : v = c.network.input p := by
    rw [Equiv.symm_apply_eq] at he
    exact he
  subst hvp
  change (c.inputFlags[p.val]?).getD .hidden = .observed
  rw [hl a ha _ hv p rfl]
  rfl
```

The bridge: `interpret` is the exact enumeration

```
section Bridge

variable (hnn : c.ChanceNonneg) (hrows : c.RowsExact) (hc : c.network.check = true)

/-- The boundary assignment of 0-based input-row states. -/
def inAssign (x : c.InCoord) :
    ((c.model hnn hrows).interp (c.strategyOf hnn hrows)).Boundary c.network.dom :=
  Interpretation.Assignment.ofPi _ x

/-- The boundary assignment of 0-based output-row states. -/
def outAssign (y : c.OutCoord) :
```

```
    ((c.model hnn hrows).interp (c.strategyOf hnn hrows)).Boundary c.network.cod :=
  Interpretation.Assignment.ofPi _ y

/-- Generator states are the internal assignments of the decoded net. -/
def genEquiv : c.GenCoord ≃
    ((c.model hnn hrows).interp (c.strategyOf hnn hrows)).Internal (c.diagram hc).net :=
  Interpretation.Assignment.piEquiv ((c.model hnn hrows).interp (c.strategyOf hnn hrows))
    (sig.result ∘ (c.diagram hc).net.label)

theorem valuation_value (x : c.InCoord) (z : c.GenCoord) (v : Fin c.network.variableCount) :
    (((c.model hnn hrows).interp (c.strategyOf hnn hrows)).valuation (c.diagram hc).net
      (c.inAssign hnn hrows x) (c.genEquiv hnn hrows hc z)
      ((c.network.toLegged (c.network.check_sound hc)).varsEquiv.symm v)).2.val =
      c.value x z v := by
  have hv := c.network.check_sound hc
  rcases hv.2.2.2.2.2.1 v with ⟨i, rfl⟩ | ⟨g, rfl⟩
  · have hs : (c.network.toLegged hv).varsEquiv.symm (c.network.input i) = Sum.inl i := by
      rw [Equiv.symm_apply_eq]
      rfl
    rw [hs]
    have hg : c.genOf (c.network.input i) = none := by
      unfold genOf
      rw [List.find?_eq_none]
      intro g _ hg
      exact hv.2.2.2.2.1 i g (of_decide_eq_true hg).symm
    have hi : c.inputOf (c.network.input i) = some i :=
      find?_unique (List.mem_finRange i) (by simp)
        (fun i' _ h' ⇒ hv.2.2.2.1 (of_decide_eq_true h'))
    simp only [value, hg, hi]
    rfl
  · have hs : (c.network.toLegged hv).varsEquiv.symm (c.network.target g) = Sum.inr g := by
      rw [Equiv.symm_apply_eq]
      rfl
    rw [hs]
    have hg : c.genOf (c.network.target g) = some g :=
      find?_unique (List.mem_finRange g) (by simp)
        (fun g' _ h' ⇒ hv.2.2.1 (of_decide_eq_true h'))
    simp only [value, hg]
    rfl

theorem vals_map (x : c.InCoord) (z : c.GenCoord) (vs : List (Fin c.network.variableCount)) :
    vals ((vs.map (c.network.toLegged (c.network.check_sound hc)).varsEquiv.symm).map
      (((c.model hnn hrows).interp (c.strategyOf hnn hrows)).valuation (c.diagram hc).net
        (c.inAssign hnn hrows x) (c.genEquiv hnn hrows hc z)).value) =
      vs.map (c.value x z) := by
  unfold vals
  refine List.map_map.trans (List.map_map.trans ?_)
  exact List.map_congr_left fun v _ ⇒ c.valuation_value hnn hrows hc x z v

theorem weight_eq (hcons : c.LabelsConsistent) (x : c.InCoord) (z : c.GenCoord) (g : c.Gen) :
    ((c.model hnn hrows).interp (c.strategyOf hnn hrows)).weight ((c.diagram hc).net.label g)
      (((c.model hnn hrows).interp (c.strategyOf hnn hrows)).parents (c.diagram hc).net
        (c.inAssign hnn hrows x) (c.genEquiv hnn hrows hc z) g)
      ((c.genEquiv hnn hrows hc z) g) = ((c.genWeight x z g : ℚ) : ℝ) := by
  have hp := c.vals_map hnn hrows hc x z (c.network.parents g)
  unfold genWeight
  by_cases hd : c.isDecisionGen g = true
  · have hd' : sig.isDecision ((c.diagram hc).net.label g) = true := hd
    rw [IDInterpretation.interp_weight_decision (c.model hnn hrows) (c.strategyOf hnn hrows) _
      hd', if_pos hd]
    have key : ((c.genEquiv hnn hrows hc z) g = ⟨sig.result ((c.diagram hc).net.label g),
        c.strategyOf hnn hrows _ hd' (((c.model hnn hrows).interp
          (c.strategyOf hnn hrows)).parents (c.diagram hc).net (c.inAssign hnn hrows x)
          (c.genEquiv hnn hrows hc z) g))⟩ ↔
        (z g).val = c.actionOf (c.network.label g)
          ((c.network.parents g).map (c.value x z)) := by
```

```
      change (⟨(c.network.label g).result, z g⟩ : TypedValue) =
        ⟨(c.network.label g).result, ⟨c.actionOf (c.network.label g) (vals _), _⟩⟩ ↔ _
      rw [sigma_mk_eq_iff]
      exact iff_of_eq (congrArg (fun vs ⇒ (z g).val = c.actionOf (c.network.label g) vs) hp)
    split_ifs with h1 h2 h2
    · simp
    · exact absurd (key.1 h1) h2
    · exact absurd (key.2 h2) h1
    · simp
  · have hd2 : c.isDecisionGen g = false := by simpa using hd
    have hd' : sig.isDecision ((c.diagram hc).net.label g) = false := hd2
    rw [IDInterpretation.interp_weight_chance (c.model hnn hrows) (c.strategyOf hnn hrows) _
      hd', if_neg hd]
    change (if (c.network.label g).result = (c.network.label g).result then
      ((c.chanceEntry (c.network.label g) (vals (((c.network.parents g).map
        (c.network.toLegged (c.network.check_sound hc)).varsEquiv.symm).map _))
        (z g).val : ℚ) : ℝ) else 0) = _
    rw [if_pos rfl]
    have e1 : c.chanceEntry (c.network.label g) ((c.network.parents g).map (c.value x z))
        (z g).val = chanceAt (c.tableOf g) (c.network.label g)
          ((c.network.parents g).map (c.value x z)) (z g).val := by
      unfold chanceEntry
      rw [c.labelTable_label hcons g hd2]
    exact congrArg (fun q : ℚ ⇒ (q : ℝ))
      ((congrArg (fun vs ⇒ c.chanceEntry (c.network.label g) vs (z g).val) hp).trans e1)

theorem joint_eq (hcons : c.LabelsConsistent) (x : c.InCoord) (z : c.GenCoord) :
    ((c.model hnn hrows).interp (c.strategyOf hnn hrows)).joint (c.diagram hc).net
      (c.inAssign hnn hrows x) (c.genEquiv hnn hrows hc z) = ((c.joint x z : ℚ) : ℝ) := by
  unfold Interpretation.joint joint
  push_cast
  exact Finset.prod_congr rfl fun g _ ⇒ c.weight_eq hnn hrows hc hcons x z g

theorem reward_eq (hcons : c.LabelsConsistent) (x : c.InCoord) (z : c.GenCoord) :
    (c.model hnn hrows).reward (c.strategyOf hnn hrows) (c.diagram hc)
      (c.inAssign hnn hrows x) (c.genEquiv hnn hrows hc z) = ((c.reward x z : ℚ) : ℝ) := by
  unfold IDInterpretation.reward reward
  push_cast
  refine Finset.sum_congr rfl fun u _ ⇒ ?_
  change ((c.utilityEntry (c.utilityLabel u) (vals (((c.scope u).map
    (c.network.toLegged (c.network.check_sound hc)).varsEquiv.symm).map _)) : ℚ) : ℝ) = _
  exact congrArg (fun q : ℚ ⇒ (q : ℝ))
    ((congrArg (fun vs ⇒ c.utilityEntry (c.utilityLabel u) vs)
      (c.vals_map hnn hrows hc x z (c.scope u))).trans (c.utilityEntry_label hcons u _))

theorem read_iff (x : c.InCoord) (z : c.GenCoord) (y : c.OutCoord) :
    c.outAssign hnn hrows y = ((c.model hnn hrows).interp (c.strategyOf hnn hrows)).read
      (c.diagram hc).net (c.inAssign hnn hrows x) (c.genEquiv hnn hrows hc z) ↔
      c.Reads x z y := by
  rw [Interpretation.Assignment.ext_iff, funext_iff]
  apply forall_congr'
  intro j
  rw [← c.valuation_value hnn hrows hc x z (c.network.output j)]
  exact sigma_eq_iff ((((c.model hnn hrows).interp (c.strategyOf hnn hrows)).read
    (c.diagram hc).net (c.inAssign hnn hrows x) (c.genEquiv hnn hrows hc z)).typed j).symm

/-- **`K` is the exact enumeration**: at the boundary assignments of 0-based states `x`,
`y`, the probability component of `interpret σ f` is `exactK x y`. -/
theorem interpret_K_eq (hcons : c.LabelsConsistent) (x : c.InCoord) (y : c.OutCoord) :
    ((c.model hnn hrows).interpret (c.strategyOf hnn hrows) (c.diagram hc)).K
      (c.inAssign hnn hrows x) (c.outAssign hnn hrows y) = ((c.exactK x y : ℚ) : ℝ) := by
  change ((c.model hnn hrows).interp (c.strategyOf hnn hrows)).kernel (c.diagram hc).net
    (c.inAssign hnn hrows x) (c.outAssign hnn hrows y) = _
  rw [IDInterpretation.kernel_eq_sum]
  unfold exactK
  push_cast
```

```
  symm
  refine Fintype.sum_equiv (c.genEquiv hnn hrows hc) _ _ fun z ⇒ ?_
  by_cases hr : c.Reads x z y
  · rw [if_pos hr, ite_cond_eq_true _ _ (eq_true ((c.read_iff hnn hrows hc x z y).2 hr)),
      c.joint_eq hnn hrows hc hcons]
  · rw [if_neg hr,
      ite_cond_eq_false _ _ (eq_false (fun e ⇒ hr ((c.read_iff hnn hrows hc x z y).1 e))),
      Rat.cast_zero]

/-- **`W` is the exact enumeration**: at the boundary assignments of 0-based states `x`,
`y`, the reward component of `interpret σ f` is `exactW x y`. -/
theorem interpret_W_eq (hcons : c.LabelsConsistent) (x : c.InCoord) (y : c.OutCoord) :
    ((c.model hnn hrows).interpret (c.strategyOf hnn hrows) (c.diagram hc)).W
      (c.inAssign hnn hrows x) (c.outAssign hnn hrows y) = ((c.exactW x y : ℚ) : ℝ) := by
  change (c.model hnn hrows).rewardTable (c.strategyOf hnn hrows) (c.diagram hc)
    (c.inAssign hnn hrows x) (c.outAssign hnn hrows y) = _
  unfold IDInterpretation.rewardTable exactW
  push_cast
  symm
  refine Fintype.sum_equiv (c.genEquiv hnn hrows hc) _ _ fun z ⇒ ?_
  by_cases hr : c.Reads x z y
  · rw [if_pos hr, ite_cond_eq_true _ _ (eq_true ((c.read_iff hnn hrows hc x z y).2 hr)),
      c.joint_eq hnn hrows hc hcons, c.reward_eq hnn hrows hc hcons, Rat.cast_mul]
  · rw [if_neg hr,
      ite_cond_eq_false _ _ (eq_false (fun e ⇒ hr ((c.read_iff hnn hrows hc x z y).1 e))),
      Rat.cast_zero]

end Bridge
```

Acceptance

```
section Accepted

variable {c} {τ : ℚ}

theorem accepted_of_check (h : c.check τ = true) : c.Accepted τ := (c.check_iff_accepted τ).1 h

theorem network_check_of_check (h : c.check τ = true) : c.network.check = true :=
  (Network.check_iff _).2 (accepted_of_check h).valid

/-- The `IDInterpretation` of an accepted certificate. -/
def modelOf (h : c.check τ = true) : IDInterpretation sig :=
  c.model (accepted_of_check h).chanceNonneg (accepted_of_check h).rowsExact

/-- The strategy of an accepted certificate. -/
def strategyOfCheck (h : c.check τ = true) : (modelOf h).Strategy :=
  c.strategyOf (accepted_of_check h).chanceNonneg (accepted_of_check h).rowsExact

/-- The `IDNet` of an accepted certificate. -/
def diagramOf (h : c.check τ = true) : IDNet (S := sig) c.network.dom c.network.cod :=
  c.diagram (network_check_of_check h)

/-- The fixed-strategy semantics of the decoded diagram, `interpret σ f`. -/
def semantics (h : c.check τ = true) :
    RewardKernel (((modelOf h).interp (strategyOfCheck h)).Boundary c.network.dom)
      (((modelOf h).interp (strategyOfCheck h)).Boundary c.network.cod) :=
  (modelOf h).interpret (strategyOfCheck h) (diagramOf h)

/-- The result cell of a pair of boundary assignments: the flat index of the input rows'
0-based states, then the output rows'. -/
def boundaryIndex (h : c.check τ = true)
    (x : ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.dom)
    (y : ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.cod) : ℕ :=
  flatIndex (List.ofFn (fun i ⇒ (dimension c.network.inputs[i].attrs, (x i).2.val)) ++
```

```
    List.ofFn (fun j ⇒ (dimension c.network.outputs[j].attrs, (y j).2.val)))

theorem cast_val {t t' : VariableAttrs} (e : t = t') (s : Fin (dimension t)) :
    (cast (congrArg (fun u ⇒ Fin (dimension u)) e) s).val = s.val := by
  subst e
  rfl

theorem boundaryIndex_eq (h : c.check τ = true)
    (x : ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.dom)
    (y : ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.cod) :
    boundaryIndex h x y = c.resultIndex (Interpretation.Assignment.toPi _ x)
      (Interpretation.Assignment.toPi _ y) := by
  unfold boundaryIndex resultIndex resultAxes
  congr 2
  · exact List.ofFn_inj.2 (funext fun i ⇒ Prod.ext rfl (cast_val (x.typed i) (x i).2).symm)
  · exact List.ofFn_inj.2 (funext fun j ⇒ Prod.ext rfl (cast_val (y.typed j) (y j).2).symm)

theorem inAssign_toPi (h : c.check τ = true)
    (x : ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.dom) :
    c.inAssign (accepted_of_check h).chanceNonneg (accepted_of_check h).rowsExact
      (Interpretation.Assignment.toPi _ x) = x :=
  (Interpretation.Assignment.piEquiv _ _).apply_symm_apply x

theorem outAssign_toPi (h : c.check τ = true)
    (y : ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.cod) :
    c.outAssign (accepted_of_check h).chanceNonneg (accepted_of_check h).rowsExact
      (Interpretation.Assignment.toPi _ y) = y :=
  (Interpretation.Assignment.piEquiv _ _).apply_symm_apply y

theorem boundaryIndex_lt_K (h : c.check τ = true)
    (x : ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.dom)
    (y : ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.cod) :
    boundaryIndex h x y < c.result.K.length := by
  rw [boundaryIndex_eq, (accepted_of_check h).cellCounts.2.2.2.1]
  exact c.resultIndex_lt _ _

theorem boundaryIndex_lt_W (h : c.check τ = true)
    (x : ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.dom)
    (y : ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.cod) :
    boundaryIndex h x y < c.result.W.length := by
  rw [boundaryIndex_eq, (accepted_of_check h).cellCounts.2.2.2.2]
  exact c.resultIndex_lt _ _

/-- Distinct pairs of boundary assignments have distinct result cells. -/
theorem boundaryIndex_injective (h : c.check τ = true)
    {x x' : ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.dom}
    {y y' : ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.cod}
    (he : boundaryIndex h x y = boundaryIndex h x' y') : x = x' ∧ y = y' := by
  rw [boundaryIndex_eq, boundaryIndex_eq] at he
  obtain ⟨hx, hy⟩ := c.resultIndex_injective he
  exact ⟨(Interpretation.Assignment.piEquiv _ _).symm.injective hx,
    (Interpretation.Assignment.piEquiv _ _).symm.injective hy⟩

/-- **The cell correspondence**: `boundaryIndex` is a bijection between pairs of boundary
assignments of the decoded diagram and the cells of the recorded `K` (and `W`, which has
the same length). -/
theorem boundaryIndex_bijective (h : c.check τ = true) :
    Function.Bijective (fun p : ((modelOf h).interp (strategyOfCheck h)).Boundary
        c.network.dom × ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.cod ⇒
      (⟨boundaryIndex h p.1 p.2, boundaryIndex_lt_K h p.1 p.2⟩ : Fin c.result.K.length)) := by
  constructor
  · intro p q e
    obtain ⟨h1, h2⟩ := boundaryIndex_injective h (congrArg Fin.val e)
    exact Prod.ext h1 h2
  · intro k
    have hk : k.val < c.resultDims.prod := (accepted_of_check h).cellCounts.2.2.2.1 ▸ k.isLt
```

```
    obtain ⟨⟨x, y⟩, hxy⟩ := c.resultCell_bijective.2 ⟨k.val, hk⟩
    refine ⟨(c.inAssign (accepted_of_check h).chanceNonneg (accepted_of_check h).rowsExact x,
      c.outAssign (accepted_of_check h).chanceNonneg (accepted_of_check h).rowsExact y),
      Fin.ext ?_⟩
    change boundaryIndex h _ _ = k.val
    rw [boundaryIndex_eq]
    exact congrArg Fin.val hxy

theorem cellQ_getElem {cells : List Value} {k : ℕ} (hk : k < cells.length) :
    cellQ cells k = cells[k].toRat := by
  simp [cellQ, List.getElem?_eq_getElem hk]

/-- **Soundness for `K`.** If `check τ c` accepts, then for every pair of boundary
assignments `x`, `y` of the decoded diagram, the recorded `K` cell at `boundaryIndex x y`
exists and is within `τ` of `(interpret σ f).K x y`. -/
theorem reward_certificate_sound_K (h : c.check τ = true)
    (x : ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.dom)
    (y : ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.cod) :
    ∃ v, c.result.K[boundaryIndex h x y]? = some v ∧
      |(v.toRat : ℝ) - (semantics h).K x y| ≤ τ := by
  have a := accepted_of_check h
  have hk := boundaryIndex_lt_K h x y
  refine ⟨c.result.K[boundaryIndex h x y], List.getElem?_eq_getElem hk, ?_⟩
  have he := c.interpret_K_eq a.chanceNonneg a.rowsExact (network_check_of_check h)
    a.labelsConsistent (Interpretation.Assignment.toPi _ x) (Interpretation.Assignment.toPi _ y)
  rw [inAssign_toPi, outAssign_toPi] at he
  change |(c.result.K[boundaryIndex h x y].toRat : ℝ) -
    ((modelOf h).interpret (strategyOfCheck h) (diagramOf h)).K x y| ≤ τ
  unfold modelOf strategyOfCheck diagramOf
  rw [he, ← cellQ_getElem hk, boundaryIndex_eq]
  exact_mod_cast c.kError_le a.kError _ _

/-- **Soundness for `W`.** If `check τ c` accepts, then for every pair of boundary
assignments `x`, `y` of the decoded diagram, the recorded `W` cell at `boundaryIndex x y`
exists and is within `τ` of `(interpret σ f).W x y`. -/
theorem reward_certificate_sound_W (h : c.check τ = true)
    (x : ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.dom)
    (y : ((modelOf h).interp (strategyOfCheck h)).Boundary c.network.cod) :
    ∃ v, c.result.W[boundaryIndex h x y]? = some v ∧
      |(v.toRat : ℝ) - (semantics h).W x y| ≤ τ := by
  have a := accepted_of_check h
  have hk := boundaryIndex_lt_W h x y
  refine ⟨c.result.W[boundaryIndex h x y], List.getElem?_eq_getElem hk, ?_⟩
  have he := c.interpret_W_eq a.chanceNonneg a.rowsExact (network_check_of_check h)
    a.labelsConsistent (Interpretation.Assignment.toPi _ x) (Interpretation.Assignment.toPi _ y)
  rw [inAssign_toPi, outAssign_toPi] at he
  change |(c.result.W[boundaryIndex h x y].toRat : ℝ) -
    ((modelOf h).interpret (strategyOfCheck h) (diagramOf h)).W x y| ≤ τ
  unfold modelOf strategyOfCheck diagramOf
  rw [he, ← cellQ_getElem hk, boundaryIndex_eq]
  exact_mod_cast c.wError_le a.wError _ _

/-- **Information locality on the decoded diagram of an accepted certificate.** -/
theorem diagram_informationLocal (h : c.check τ = true) :
    ∀ a : (diagramOf h).net.Gen, (diagramOf h).IsDecision a →
      ∀ p : c.network.dom.Port, Sum.inl p ∈ (diagramOf h).info a →
        c.observedDom.flag p = .observed :=
  c.diagram_informationLocal_of _ (accepted_of_check h).informationLocal

/-- The decoded diagram's `K` is stochastic, so the semantics is a morphism of `FinRew`. -/
theorem semantics_stochastic (h : c.check τ = true) : (semantics h).Stochastic :=
  (modelOf h).interpret_stochastic (strategyOfCheck h) (diagramOf h)

end Accepted

end Certificate
```

```
end OpenInfluence.RewardCertificate
```